

THE PRINCIPAL ML SYSTEMS HANDBOOK



TRAINING / INFERENCE / CUDA
DISTRIBUTED SYSTEMS / TECHNICAL LEADERSHIP



YURY KIRPICHEV

FIRST EDITION - 2026

A field guide for senior engineers who must connect model behavior, accelerator performance, distributed architecture, and organizational judgment into one coherent system.

This handbook is built around a demanding premise: **the interview is not the subject**. The subject is the ability to design, explain, debug, and lead machine learning systems under real constraints. Interview readiness follows from that mastery.

The material is organized around the questions that repeatedly surface in Staff and Principal loops: How do you design a training recipe? Why does a serving system miss its latency target? What makes one CUDA kernel faster than another? When should a team trade quality for capacity? How do you turn technical ambiguity into an executable cross-org decision?

HOW TO USE THIS BOOK

For a fast interview pass, read the chapter opener, the decision rules, the pitfalls, and the review drill. For durable mastery, reproduce the derivations, implement the kernels, and defend every architecture against at least two credible alternatives.

The Principal answer pattern

A strong answer usually moves through six layers:

- 1 **Frame the objective.** State the user outcome, workload, SLO, quality bar, and constraints before naming a technique.
- 2 **Build a quantitative model.** Estimate bytes, FLOPs, memory, communication, queueing, or sample complexity at the right level of fidelity.
- 3 **Choose the bottleneck.** Separate symptoms from the resource or coordination limit that actually controls performance.
- 4 **Compare alternatives.** Explain why a reasonable competing design loses under the stated conditions.
- 5 **Design the measurement loop.** Define offline, online, guardrail, and operational metrics with a rollback boundary.
- 6 **Name the next uncertainty.** A Principal answer makes the remaining risk visible and proposes the cheapest experiment that can retire it.

Editorial conventions

- Color-coded boxes separate the compact **Interview Insight**, the attractive but incomplete **Common Pitfall**, and the governing **Principal Decision**.
- Equations use implementation-oriented notation; code stays small enough for a whiteboard, then expands into production considerations, and every technical chapter closes with a transfer-focused Principal interview review.

Contents

Part I - Foundations for ML Systems Decisions 8

 Optimization as a System 9

 Transformers as an Architecture of Tradeoffs 13

 Measurement, Evaluation, and Experimental Judgment 17

Part II - Training, Data, and Distillation 20

 Designing a Training Recipe 21

 Data Curation and Synthetic Data 25

 Distillation, KL Divergence, and Model Sizing 29

 Post-Training and Alignment 33

Part III - LLM Inference Systems 36

 Request Lifecycle, Metrics, and Workload Models 38

 Prefill, Decode, and Performance Modeling 44

 KV Cache, Paging, and Prefix Reuse 51

 Scheduling, Batching, and Admission Control 57

 Sampling, Structured Output, and Speculative Decoding 63

 Parallel, Replicated, and Disaggregated Serving 70

 Quantization, Compression, and Adapter Serving 76

 Production Architecture, Capacity, and Reliability 82

Part IV - CUDA and Accelerator Programming 90

 GPU Execution, Memory, and Resource Accounting 93

 Host-Device Orchestration, Streams, and CUDA Graphs 100

Hierarchical Matrix Multiplication	105
Reductions, Prefix Scans, and Histograms	112
Softmax, Normalization, Top-K, and Sampling	117
FlashAttention and IO-Aware Exact Attention	123
CUDA Kernels for LLM Inference	129
Kernel Engineering, Profiling, and Correctness	135
Part V - Distributed ML Systems	142
Communication Models, Collectives, and Topology	143
Data Parallelism, ZeRO, and Fully Sharded Training	149
Tensor, Sequence, and Context Parallelism	154
Pipeline Parallelism and Hybrid Plans	159
Mixture-of-Experts and Sparse Communication	165
Distributed Inference and Stateful Placement	171
Distributed Checkpoints, Recovery, and Elasticity	177
Cluster Scheduling, Observability, and Distributed Diagnosis	183
Part VI - Coding and System Design	191
Streaming Algorithms and Top-K	192
ML Algorithms in Production Code	195
A Principal System Design Method	198
RAG, Vector Search, and Evaluation Pipelines	201
Part VII - Technical Leadership and the Interview Masterclass	204
Executive Technical Communication	205
Strategy, Vision, and the First 90 Days	208
Behavioral Stories at Principal Level	211

Recruiter-Derived Interview Masterclass 214

Part VIII - Field Reference 218

Formula and Capacity Sheet 219

CUDA Interview Checklist 222

Rapid-Fire Question Bank 225

Glossary and Decision Index 229

A note on scope

This first edition is a cohesive, recruiter-derived core: foundations, training, inference, CUDA, distributed systems, coding, system design, and leadership. It is intentionally opinionated. The goal is not to catalog every paper. The goal is to teach a reusable method for reaching sound decisions when details change.

The examples assume transformer-style models and GPU-like accelerators, but the reasoning applies more broadly. Hardware names, model families, and framework APIs will evolve. Arithmetic intensity, dependency structure, failure isolation, data provenance, and organizational incentives will not.

Reader contract

Do not memorize the final recommendation without the assumptions that produced it. A technique is rarely "best" in isolation. It is best for a workload, budget, failure model, and organization. If any of those move, reopen the decision.

The one-page interview operating system

Phase	What to say	What the interviewer learns
Clarify	Workload, scale, latency, quality, cost, failure tolerance	You refuse to optimize an undefined system
Model	Dominant bytes, FLOPs, state, communication, queueing	You reason quantitatively
Propose	Baseline architecture with explicit ownership	You can make a decision
Stress	Hot keys, long tails, partial failure, distribution shift	You expect reality
Measure	Success, guardrails, observability, rollback	You close the loop
Extend	Two alternatives and the condition that flips the choice	You understand the design space

THE STANDARD OF EVIDENCE

Use an equation when it changes the decision, a benchmark when implementation details dominate, and an experiment when uncertainty is cheaper to remove than to debate. Do not use precision theater to hide an unknown workload.

PART I - FOUNDATIONS FOR ML SYSTEMS DECISIONS

Part I - Foundations for ML Systems Decisions



The foundation is not a list of algorithms. It is a way to connect statistical objectives to the data, precision, memory, and execution constraints that determine whether a model can be trained and served.

CHAPTER 01

01

Optimization as a System

Optimization as a System

Training is a feedback control problem implemented by a distributed numerical program. The optimizer, data order, precision policy, parallelism plan, and observability stack jointly define the trajectory.

Start from the update, not the optimizer name

For parameters θ , stochastic optimization applies an update from a noisy gradient estimate. Plain SGD is:

$$\theta_{t+1} = \theta_t - \eta_t g_t$$

The practical questions are hidden inside the symbols. What examples produced g_t ? How much variance does the batch remove? In what precision was the reduction accumulated? Was the gradient clipped before or after data-parallel synchronization? Does η_t refer to the nominal learning rate or the effective rate after adaptive normalization?

Momentum adds a state variable that filters short-term gradient noise:

$$m_t = \beta m_{t-1} + (1 - \beta) g_t$$

$$\theta_{t+1} = \theta_t - \eta_t m_t$$

Adam tracks both first and second moments. AdamW decouples weight decay from the adaptive gradient step, avoiding the coordinate-dependent regularization produced by inserting L2 penalty directly into Adam's gradient.

SAY THE INVARIANT FIRST

SGD chooses one global scale per step; Adam chooses a coordinate-wise scale from gradient history. AdamW then applies shrinkage independently of that adaptive scale. The right choice depends on optimization geometry, batch regime, memory budget, and generalization evidence.

Batch size is a systems knob

Larger batches improve hardware utilization and reduce gradient variance, but they also change the number of parameter updates per token. A comparison that holds epochs constant but not optimizer steps can attribute a training difference to the optimizer when the real cause is schedule mismatch.

Define global batch as:

$$B_{\text{global}} = B_{\text{micro}} * \text{accumulation_steps} * \text{data_parallel_replicas}$$

Increasing any factor may preserve the nominal batch while changing activation memory, communication cadence, or pipeline bubbles. Gradient accumulation reduces all-reduce frequency per token but lengthens the interval before parameters move. At very large batch, the noise scale can fall below the level that helped exploration or regularization.

Learning-rate scaling

Linear scaling is a useful initial hypothesis when the batch grows and the optimization regime is stable. Square-root scaling is more conservative under high gradient noise. Neither is a law. Use a short sweep around the predicted rate and compare loss versus **tokens and wall-clock**, not steps alone.

Failure signal	Likely mechanism	First check
Loss spikes early	Update norm too large or precision overflow	Grad norm, loss scale, warmup
Stable loss, weak validation	Overfitting or data mismatch	Split integrity, duplication, regularization
Throughput high, convergence slow	Batch too large or stale updates	Tokens per update, optimizer steps
Replica losses diverge	Synchronization or nondeterministic data	Collective health, seed and sampler state
Late NaNs	Accumulated optimizer-state corruption	Moment statistics, checkpoint bisection

Precision policy is part of the algorithm

Mixed precision is not "turning on FP16." A production policy specifies:

- storage dtype for parameters and optimizer states;
- compute dtype for matrix multiply and reductions;
- accumulation dtype for sensitive sums;
- loss scaling behavior;
- master-weight policy;

- overflow detection and recovery;
- which operations are forced into higher precision.

BF16 preserves the exponent range of FP32 with fewer mantissa bits, which usually makes it easier to train than FP16 without dynamic loss scaling. FP16 provides more mantissa precision at a narrower range. FP8 increases throughput and reduces bandwidth but requires scale selection, granularity, delayed or current statistics, and careful protection of outliers.

PRECISION IS NOT ONLY A SPEED SETTING

If a candidate says "use lower precision" without naming accumulation, scaling, outliers, and validation criteria, the answer is incomplete. Numerical policy can change convergence and therefore total training cost.

Initialization, normalization, and residual scale

Deep residual networks work because signal and gradient magnitudes remain controlled across many layers. Initialization sets the first scale; normalization and residual parameterization determine how it evolves.

Pre-norm transformers place normalization before the sublayer, improving gradient flow for deep networks. Post-norm can provide different representation dynamics but is harder to stabilize at scale. RMSNorm removes mean subtraction and reduces work; LayerNorm controls both mean and variance. The performance difference is usually small relative to matmul, but the training dynamics and fusion opportunities matter.

For a residual update $x_{l+1} = x_l + f_l(x_l)$, depth-aware scaling prevents the sum of many residual branches from growing without bound. Techniques differ in where they introduce factors, but the design question is the same: how should the variance of the residual stream behave as depth increases?

A reproducible training step

PYTHON

```
def train_step(model, batch, optimizer, scaler, clip_norm):
    optimizer.zero_grad(set_to_none=True)
    with autocast(dtype="bfloat16"):
        logits = model(batch.tokens)
        loss = cross_entropy(logits[:, :-1], batch.tokens[:, 1:])

    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    grad_norm = clip_grad_norm_(model.parameters(), clip_norm)
    scaler.step(optimizer)
    scaler.update()
    return {"loss": loss.detach(), "grad_norm": grad_norm}
```

The interview discussion begins after the code: Where is accumulation? Which values are synchronized? Is clipping global across shards? How are token masks normalized? What happens

when one worker overflows? Does a skipped step advance the data sampler or learning-rate schedule?

Scaling laws as resource allocation

Scaling laws relate reducible loss to model size, data, and compute over a particular regime. Their value is not the exact exponent; it is the discipline of treating architecture and data allocation as an optimization problem under a compute budget.

A compute-optimal recipe balances marginal return from more parameters against marginal return from more training tokens. Real products add constraints absent from the ideal curve: inference cost, memory capacity, data rights, latency, fine-tuning behavior, and the value of rapid iteration.

WHEN THE SCALING-LAW OPTIMUM IS NOT THE PRODUCT OPTIMUM

A smaller, better-trained model may win when serving dominates lifetime cost. A larger model trained on fewer tokens may win when capability at the frontier matters more than utilization. State which lifetime objective you are optimizing.

Principal Interview Review

- 1 Explain why doubling global batch without changing the schedule is not a controlled optimizer comparison.
- 2 Design a numerical policy for FP8 training. Name the values you would keep in BF16 or FP32.
- 3 A run is 18 percent faster per step but reaches the target loss 25 percent later. What should the team report?
- 4 How would you localize a NaN that appears after 30,000 steps on one of 256 workers?
- 5 When would you prefer SGD with momentum to AdamW for a large model?

A Principal answer connects the loss curve to the execution trace. It does not stop at optimizer folklore.

CHAPTER 02

02

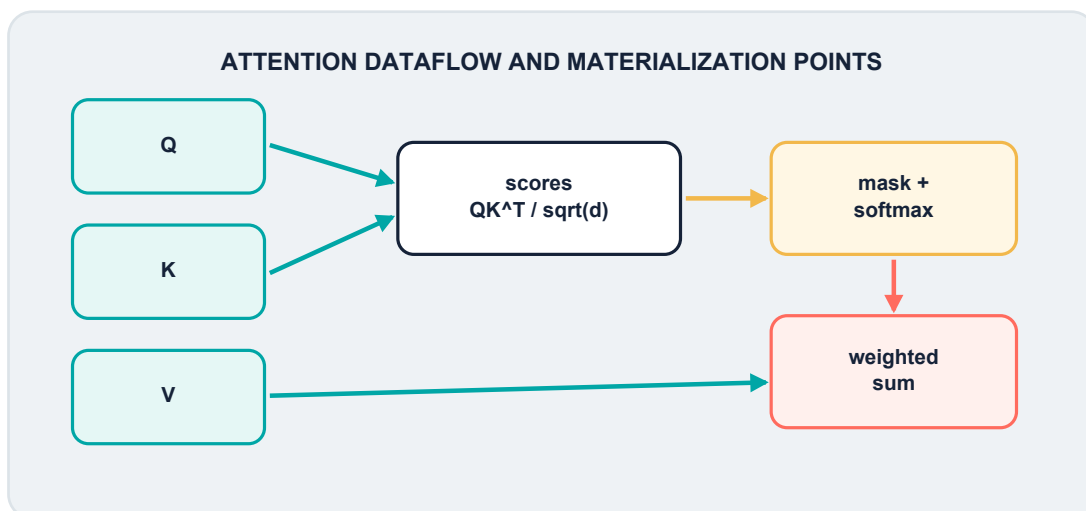
Transformers as an Architecture of Tradeoffs

Transformers as an Architecture of Tradeoffs

A transformer is a collection of decisions about communication, state, and inductive bias. Attention is important, but the systems interview is about why each variant moves cost or quality.

The block and its cost centers

A decoder block alternates token mixing and channel mixing. Self-attention allows each token to read other positions; the feed-forward network transforms each position independently. Residual connections preserve an information highway; normalization stabilizes scale.



Attention is a sequence of projections, a score computation, a normalization, and a weighted reduction. Efficient kernels avoid materializing the large score matrix in HBM.

For sequence length S , hidden width D , heads H , and head dimension $d = D/H$, projection FLOPs scale roughly with $S \cdot D^2$, while naive attention score and value products scale with $S^2 \cdot D$. At short context, projections and MLP dominate. At long context, quadratic attention becomes the controlling term unless sparsity, locality, or recurrence changes the dependency graph.

The feed-forward expansion is often four times the hidden width in a dense baseline, although gated variants change constants. Because those layers contain much of the parameter count and FLOPs, a discussion focused only on attention misses the dominant compute for many workloads.

Multi-head, multi-query, and grouped-query attention

Standard multi-head attention stores separate K and V projections for every query head. Multi-query attention shares one K/V head; grouped-query attention shares K/V within groups.

During training, reducing K/V heads modestly changes projection parameters. During autoregressive serving, it can dramatically reduce KV -cache bytes and memory traffic. The quality cost depends on model size, data, and head grouping.

Variant	Query heads	K/V heads	KV -cache cost	Typical use
MHA	H	H	Highest	Maximum flexibility
GQA	H	G , where $1 < G < H$	Medium	Quality-throughput balance
MQA	H	1	Lowest	Memory-constrained decoding

WHY GQA EXISTS

GQA is a serving-aware architecture decision. It preserves multiple K/V subspaces while reducing the state that must be read for every generated token.

Position: RoPE, ALiBi, and context extension

Transformers require a notion of order because attention itself is permutation equivariant. RoPE rotates query and key components by position-dependent angles, making the dot product depend on relative offset. ALiBi adds head-specific linear distance biases to attention scores.

Context extension is not achieved by changing one configuration value. A model trained mostly on short sequences may not learn behavior at long offsets. RoPE scaling changes the mapping between position and phase; interpolation or frequency-aware schemes trade local resolution against extrapolation. Long-context training also changes memory, parallelism, data construction, and evaluation.

Long-context evaluation

Needle-in-a-haystack retrieval is a diagnostic, not a complete evaluation. Include:

- retrieval across distance and distractor density;
- multi-hop composition across separated evidence;
- instruction retention over long interaction histories;
- long-form generation coherence;
- latency, prefill memory, and KV-cache pressure;
- degradation on ordinary short-context tasks.

Mixture of Experts

MoE increases parameter count without activating all parameters per token. A router selects a small number of experts. The promise is higher capacity per unit of compute; the cost is routing instability, load imbalance, all-to-all communication, and operational complexity.

Let E be experts and k active experts per token. Compute follows k , but storage follows E . Communication follows token movement across the expert-parallel group. If one expert receives too many tokens, capacity limits cause drops or padding and lower utilization.

Typical objectives combine task loss with an auxiliary load-balancing term. The auxiliary term should be monitored as a means, not a product metric. Perfectly uniform routing can be incompatible with specialization.

SPARSE DOES NOT MEAN CHEAP

An MoE layer may have low arithmetic cost but high network cost. The design is attractive only when expert placement, token dispatch, batching, and capacity produce high accelerator utilization.

Attention kernel choices

Naive attention materializes $S \times S$ scores in HBM. FlashAttention tiles Q , K , and V into on-chip memory, computes softmax statistics online, and writes only the final output plus small normalization state. It is exact attention with a different IO schedule, not an approximation.

For backward, the kernel recomputes selected intermediates because recomputation FLOPs are cheaper than reading a quadratic matrix from HBM. This is a recurring accelerator principle: spend abundant arithmetic to avoid scarce data movement.

Architecture review template

When an interviewer asks you to choose a transformer variant, organize the answer by lifecycle:

- 1 Training:** FLOPs, activation memory, stability, parallelism, and data.
- 2 Serving prefill:** long-sequence compute, batching, attention kernel, and time to first token.
- 3 Serving decode:** weight and KV bytes, batch shape, scheduling, and time per output token.
- 4 Quality:** capacity, context behavior, specialization, and fine-tuning response.

- 5 Operations:** kernel availability, quantization, observability, fallback, and model portability.

Principal Interview Review

- 1** Why can a change from MHA to GQA improve decode throughput more than prefill throughput?
- 2** Explain FlashAttention without saying "it is faster attention."
- 3** Design an evaluation that detects long-context gains without hiding short-context regression.
- 4** When can MoE be slower than a dense model with similar active FLOPs?
- 5** Why is an architecture decision also a serving decision?

CHAPTER 03

03

Measurement, Evaluation, and Experimental Judgment

Measurement, Evaluation, and Experimental Judgment

Evaluation is the contract between an optimization and the product. If the contract is weak, a locally correct system can still fail.

Separate four kinds of metrics

Capability metrics measure whether the model can perform tasks: exact match, pass rate, calibrated score, factuality, or human preference. **System metrics** measure latency, throughput, utilization, queueing, and cost. **Guardrail metrics** capture safety, privacy, fairness, regressions, and abuse. **Diagnostic metrics** reveal mechanisms: acceptance rate, router balance, KV hit rate, gradient norm, or cache fragmentation.

No single metric belongs to every layer. Acceptance rate is an excellent diagnostic for speculative decoding, but speedup and quality are the product outcome. Perplexity is useful for broad training progress, but it may fail to predict instruction following or code correctness.

Build the evaluation matrix

Cross user segments with task categories, difficulty, language, context length, and safety risk. Attach confidence intervals and minimum sample sizes. Preserve immutable benchmark versions and provenance. Track contamination risk explicitly.

Dimension	Example slices	Why it matters
Workload	chat, code, reasoning, summarization	Optimizations move tasks differently
Context	short, medium, long, multi-turn	Memory and attention behavior shift
Traffic	interactive, batch, burst	Queueing changes the system optimum
User	locale, domain, experience	Aggregate scores hide regressions
Risk	ordinary, sensitive, adversarial	Guardrails need targeted power

Offline does not predict online by default

Offline evaluation offers control and reproducibility. Online experiments include user adaptation, traffic mix, cache warmth, queueing, and feedback effects. A robust launch chain is:

- 1 unit and numerical checks;
- 2 offline capability and safety suite;
- 3 replay with production-like shapes;
- 4 shadow traffic without user exposure;
- 5 small canary with automatic rollback;
- 6 controlled experiment with segment analysis;
- 7 broad rollout with durable monitors.

CHOOSE THE SMALLEST HONEST EXPERIMENT

Do not demand an online A/B test for a broken kernel, and do not accept a microbenchmark for a product claim. Match the evidence surface to the decision surface.

Tail latency and coordinated omission

A latency benchmark that generates the next request only after the previous request completes reduces offered load when the system slows. It can hide queueing collapse. Use an arrival process independent of service completion, record admission rejections, and report the full latency distribution by request shape.

Always define the boundary: client-to-client, gateway-to-gateway, engine-only, or kernel-only. Time to first token, inter-token latency, and end-to-end completion latency answer different user questions.

Statistical power and practical significance

Confidence intervals quantify sampling uncertainty, not benchmark validity. A highly significant 0.1 percent lift may be irrelevant; a noisy 3 percent loss in a critical safety slice may block launch. Predefine the minimum effect worth acting on and the guardrail boundary that cannot be crossed.

Multiple comparisons inflate false positives when teams inspect many slices. Use a hierarchy: primary metrics, declared guardrails, and exploratory diagnostics. Treat exploratory findings as hypotheses for the next experiment.

A compact experiment review

TEXT

Decision: What choice will this experiment change?
Hypothesis: What causal mechanism predicts the result?
Population: Which workload and users are represented?
Metrics: Primary outcome, guardrails, diagnostics.
Power: What effect size can we reliably detect?
Execution: Randomization, duration, ramp, rollback.
Analysis: Segments, uncertainty, missing data.
Follow-up: Ship, iterate, or retire - with owner and date.

Principal Interview Review

- 1 A model improves benchmark accuracy but increases p99 latency. How do you decide?
- 2 Explain why a closed-loop load generator can lie about overload behavior.
- 3 Design evaluation for a new data-quality filter.
- 4 Which metrics would you use for an optimization that changes only KV-cache layout?
- 5 How do you respond when the aggregate metric is flat but an important segment regresses?

Measurement is not the final reporting step. It is part of architecture because it determines which failures the organization can see.

PART II - TRAINING, DATA, AND DISTILLATION

Part II - Training, Data, and Distillation



Training systems turn a capability hypothesis into a reproducible model. The hard work is not only gradient descent. It is choosing data, objectives, curriculum, precision, parallelism, checkpoints, evaluation, and stop conditions that reinforce one another.

CHAPTER 04

04

Designing a Training Recipe

Designing a Training Recipe

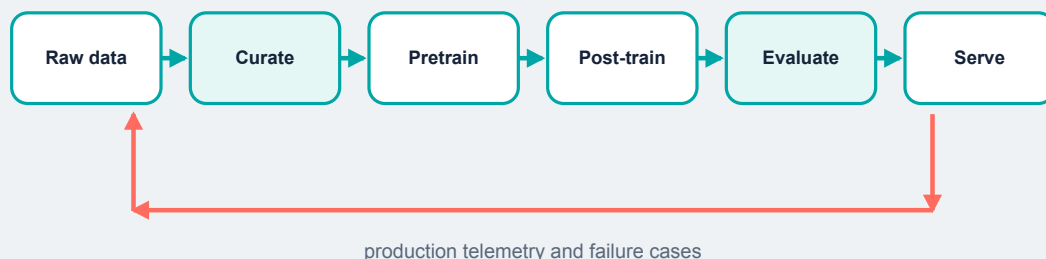
A recipe is an executable theory of how a model will acquire a capability within a compute and time budget. Begin with the target behavior and work backward to data and objective.

The recipe canvas

Before selecting hyperparameters, write a one-page canvas:

- **Product behavior:** tasks, users, languages, modalities, context, and safety boundary.
- **Model constraints:** parameter budget, serving hardware, latency, throughput, memory, and quantization target.
- **Data assets:** licensed corpora, synthetic generation, preference labels, verifier signals, and contamination risks.
- **Training stages:** pretraining, continued pretraining, supervised fine-tuning, preference or RL stage, distillation, and compression.
- **Evaluation gates:** capability, generalization, safety, calibration, and system performance.
- **Operations:** failure recovery, lineage, reproducibility, checkpoint cadence, and ownership.

THE TRAINING SYSTEM IS A CLOSED LOOP



A production recipe is a loop. Serving failures become curated examples, evaluation updates, and the next training stage.

Work backward from serving

If the product needs high-volume interactive decode, architecture and tokenizer choices should reflect KV-cache and output-length costs. If the product needs code completion, data freshness, exact syntax, repository context, and executable verification matter more than generic preference labels. If the product needs a speculative draft, target-distribution agreement and acceptance speed matter more than standalone benchmark leadership.

A common failure is to train the most capable model that fits the training cluster and discover later that it cannot meet the serving SLO. The lifecycle objective is closer to:

$$\text{value} = \text{quality_gain} - \text{training_cost} - \text{lifetime_serving_cost} - \text{operational_risk}$$

The terms are not directly commensurate, but writing them forces a complete decision.

Stage design

Pretraining builds broad representations from next-token prediction or a related self-supervised objective. **Continued pretraining** shifts domain, language, freshness, or context distribution while protecting general capability. **Supervised fine-tuning** teaches response formats and high-quality trajectories. **Preference optimization** shifts behavior among plausible outputs. **Distillation** transfers a distribution or capability into a smaller serving envelope.

Stages should be justified by what signal they add. Do not add a preference stage because it is fashionable if verified demonstrations already specify the desired answer. Do not use supervised imitation for an objective that requires exploration and delayed reward.

Curriculum and mixture

Data mixture weights determine the gradient distribution. Sampling every source in proportion to raw size lets abundant low-value sources dominate. Equal sampling overweights tiny domains and can cause memorization. Temperature sampling interpolates between these extremes.

Curriculum can mean ordering by difficulty, capability, context length, quality, or domain. It is useful when later data assumes skills learned earlier or when expensive long sequences should be introduced after the model is stable. It is harmful when the sequence creates catastrophic forgetting or a distribution cliff.

Mixture control loop

- 1 Define capability slices with leading metrics.
- 2 Estimate source-to-slice influence through ablations or data attribution proxies.
- 3 Choose a baseline mixture that covers product distribution.
- 4 Train small pilots across a sparse set of mixture changes.
- 5 Fit a response model only as complex as the evidence supports.
- 6 Update the mixture and preserve held-out confirmation data.

SYNTHETIC VOLUME IS NOT SYNTHETIC VALUE

Teacher generations reproduce teacher biases and can collapse diversity. The useful unit is a verified, novel training signal, not a generated token.

Schedules and stop conditions

Warmup protects early training when moment estimates and activation scales are unstable. Cosine or linear decay then reduces update size as the run approaches a local basin. Restart schedules can help exploration but complicate reproducibility and interpretation.

Stopping only when training loss plateaus wastes compute. Use a portfolio of signals: validation loss, target capability, data-slice trends, gradient noise, checkpoint-to-checkpoint improvement, projected value of more tokens, and schedule milestones. A predeclared review point makes it easier to stop a prestigious run whose marginal return has collapsed.

Checkpointing and recovery

A distributed checkpoint must capture parameters, optimizer states, scheduler, RNG state, data-loader position, tokenizer and configuration hashes, parallel topology assumptions, and code version. Asynchronous checkpointing reduces pause time but requires a consistency boundary. Sharded checkpoints reduce write hotspots but make restore topology and format evolution first-class design problems.

Use periodic full checkpoints plus more frequent lightweight or incremental protection. Test restore before the expensive run. A checkpoint that has never been restored is a hypothesis.

Recipe interview: a reasoning model

For a reasoning model, start with a strong base model. Construct tasks with verifiable answers, collect diverse solution trajectories, and filter by both correctness and strategy diversity. Use supervised training to establish format and basic reasoning. Add outcome- or process-based optimization only where the reward is reliable. Prevent reward hacking with hidden tests, adversarial cases, and holdout generators. Track answer accuracy, pass@k, calibration, token efficiency, and failure types.

A RECIPE ANSWER SHOULD HAVE GATES

Name what evidence permits progression from pilot to scale-up, from supervised training to online optimization, and from offline evaluation to serving. Gates show that you can operate the program, not merely describe it.

Principal Interview Review

- 1 Design a recipe for a code model that must run on a fixed single-GPU serving tier.
- 2 When would continued pretraining be safer than changing the base mixture from the start?
- 3 How would you decide whether to spend the next budget increment on data, model size, or more steps?
- 4 What belongs in a resumable checkpoint for a sharded optimizer?
- 5 How do you prevent a reasoning model from learning to exploit its verifier?

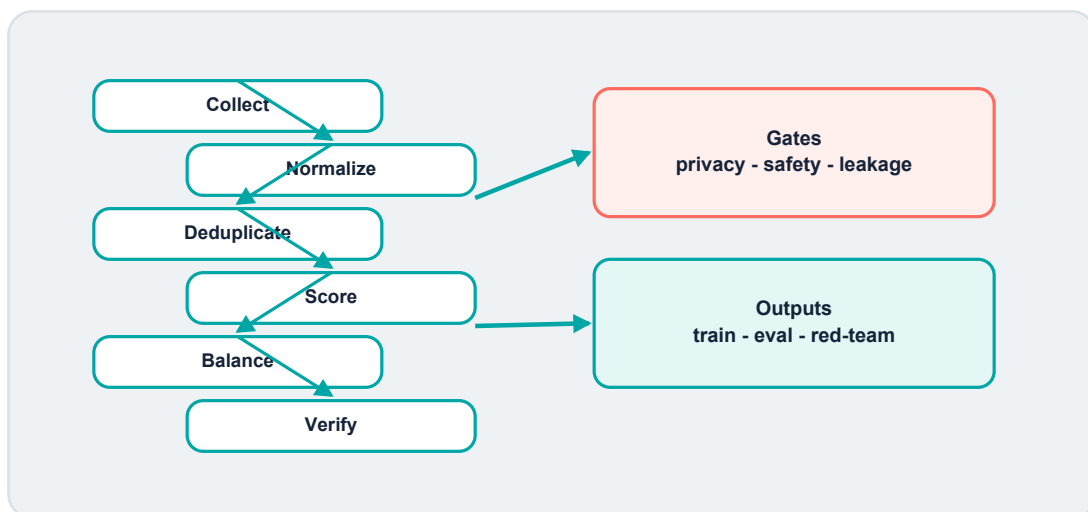
CHAPTER 05

05

Data Curation and Synthetic Data

Data Curation and Synthetic Data

Data quality is a pipeline of explicit decisions about identity, rights, utility, diversity, and risk. "Clean the data" is not a design.



A dataset is the result of transformations and gates. Every retained example should have provenance and a reason to exist.

Source contracts and provenance

For every source, record ownership or license, acquisition time, original identifier, permitted uses, deletion obligations, language, domain, and transformation lineage. Hashes enable identity checks but do not replace source IDs because normalization changes bytes.

Provenance must survive shuffling and packing. If a later safety incident requires removal, the system should identify derived examples, synthetic descendants, trained checkpoints, and evaluations affected by the source.

Normalization without erasing signal

Normalization standardizes encoding, line endings, markup, and obvious boilerplate. Aggressive normalization can destroy code indentation, mathematical structure, conversational turns, or document boundaries. Keep raw and normalized references; version transformation logic; sample diffs by domain.

Language identification should return a distribution or confidence, not only a hard label. Mixed-language documents are common. Route uncertain or code-heavy text through specialized logic instead of dropping it indiscriminately.

Deduplication at several radii

Exact duplication wastes tokens and increases memorization. Near duplication includes templated pages, mirrors, lightly edited articles, and repeated code. Semantic duplication includes paraphrases or generated variants that carry little new training signal.

Use several stages:

- 1 content hashes after conservative normalization for exact duplicates;
- 2 shingled MinHash or locality-sensitive hashing for near-duplicate text;
- 3 structural signatures for code and templates;
- 4 embedding or model-based similarity only where its cost is justified;
- 5 group-aware splitting so related documents cannot cross train and evaluation.

The cluster representative policy matters. Keeping the shortest document can remove context; keeping the longest can preserve spam. Score representatives on quality, provenance, completeness, and recency.

PYTHON

```
def assign_near_duplicate_cluster(doc, bands, index):
    signature = minhash(shingles(normalize(doc.text), width=5))
    candidates = set()
    for band_id, band_hash in split_into_bands(signature, bands):
        candidates |= index.lookup(band_id, band_hash)
    match = best_jaccard_match(doc, candidates)
    if match and match.score >= doc.threshold:
        return match.cluster_id
    return index.create_cluster(doc, signature)
```

Quality scoring

Quality is multidimensional. Signals may include language fluency, information density, coherence, citation structure, code validity, authoritativeness, originality, toxicity, and fit to target capability. A single scalar score is useful for ranking but dangerous for governance because tradeoffs become hidden.

Train scorers on carefully defined labels and test calibration by domain. Adversarially inspect high-scoring examples. A model can learn that length, formal tone, or particular sites imply quality, then systematically suppress concise or minority-domain material.

FILTER, WEIGHT, OR ROUTE

Hard-filter examples only for policy, corruption, or very low expected value. Use weights when utility is continuous. Route specialized data into domain mixtures when the data is valuable but distributionally distinct.

Contamination and benchmark leakage

Contamination can occur through direct benchmark text, solution discussions, derivative tasks, teacher prompts, or synthetic paraphrases. Search exact and fuzzy overlaps before training, but also protect the evaluation supply chain: restrict access, log joins, separate credentials, and rotate private tests.

Report contamination risk rather than pretending it is binary. Evaluate on time-split, source-isolated, and newly authored sets. For code, use hidden tests and repository-level splits. For reasoning, generate parameterized variants with independently verified answers.

Synthetic generation pipeline

Synthetic data is a programmable acquisition process:

```
seed -> generate -> verify -> score -> deduplicate -> balance -> audit ->
train
```

Seed selection controls coverage. Generation temperature and prompting control diversity. Verification may be executable, symbolic, retrieval-based, consensus-based, or human. Rejection rules must not collapse the exact rare behaviors the project wants to learn.

For coding, compile, run hidden tests, lint, and measure complexity. For mathematics, use symbolic checks and alternative solvers. For open-ended reasoning, require evidence, decompose claims, and use adjudication on uncertain samples. Teacher confidence is not independent verification.

Distribution balancing

Balancing should reflect the desired post-training workload while reserving capacity for rare but high-cost failures. If 1 percent of requests are safety-sensitive, naive frequency weighting may undertrain them; uniform category weighting may distort ordinary behavior.

Use a constrained objective: maximize expected utility on the product distribution subject to minimum performance on critical slices and limits on regression. Track the effective sample weight, not only raw counts.

Data pipeline system design

At large scale, separate immutable object storage, metadata catalog, distributed transformations, feature or score stores, and dataset manifests. Make every dataset a content-addressed manifest of shards and transformation versions. Validate row counts, token counts, language mix, duplication, score distributions, and holdout overlap before publication.

Operational requirements include idempotence, partial reruns, backpressure, deletion propagation, audit logs, budget accounting, and reproducible sampling. Data systems fail silently when teams can overwrite a dataset name without changing its visible version.

Principal Interview Review

- 1 Design a data-generation pipeline for speculative decoding.
- 2 How would you evaluate whether a new quality filter helps rather than merely shortens the dataset?
- 3 Explain train-test contamination beyond exact string matching.
- 4 A model-based quality scorer rejects dialectal text. How do you diagnose and repair the pipeline?
- 5 When should synthetic data receive lower sampling weight than human-authored data?

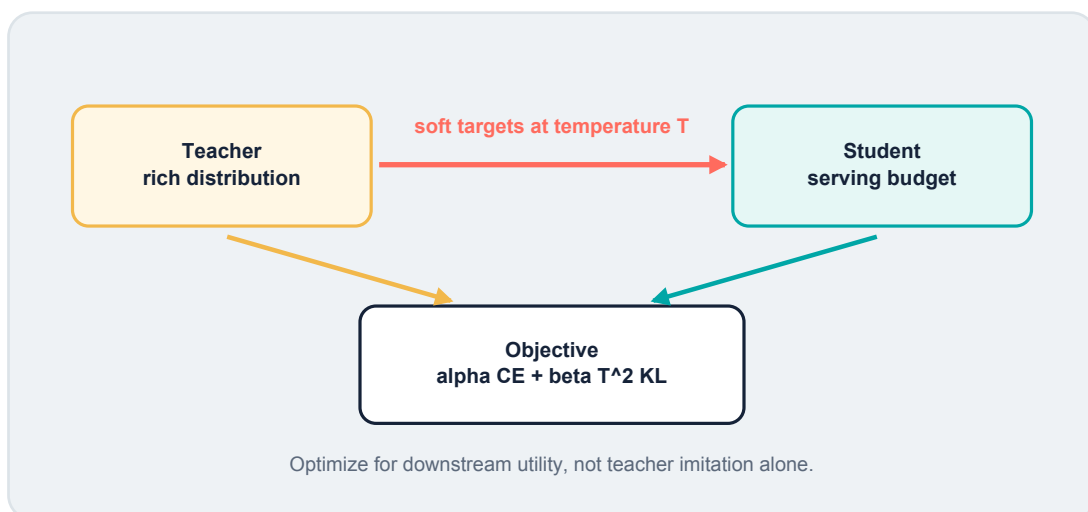
CHAPTER 06

06

Distillation, KL Divergence, and Model Sizing

Distillation, KL Divergence, and Model Sizing

Distillation is distribution transfer under a capacity and serving constraint. The central question is not "teacher or student?" It is which information should cross the capacity boundary.



The teacher provides a structured target; the student must convert it into utility inside a different architecture and budget.

Cross entropy and KL

Let teacher distribution be $p(y|x)$ and student distribution be $q(y|x)$. Cross entropy from teacher to student is:

$$H(p, q) = - \sum_y p(y|x) \log q(y|x)$$

It decomposes as $H(p) + KL(p \parallel q)$. The teacher entropy does not depend on student parameters, so minimizing teacher cross entropy is equivalent to minimizing forward KL.

$$KL(p \parallel q) = \sum_y p(y) \log(p(y) / q(y))$$

Forward KL heavily penalizes the student for assigning too little probability where the teacher has mass. It is often described as mode covering. Reverse KL, $KL(q \parallel p)$, weights regions under the student and tends to concentrate on one teacher mode when representing all modes is expensive.

These slogans are helpful but incomplete. Token distributions are conditional, the student is parameterized, the support is finite after softmax, and sequence-level behavior compounds local choices. Always connect the divergence to the actual objective and sampling process.

Temperature

With temperature T , logits z become:

$$p_i(T) = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

Higher temperature softens the distribution and exposes relationships among non-argmax tokens. Lower temperature sharpens it. In classic distillation, multiplying the KL term by T^2 compensates for the gradient scale change introduced by temperature.

PYTHON

```
def distillation_loss(student_logits, teacher_logits, labels, T=2.0, alpha=0.7):
    teacher = softmax(teacher_logits / T, dim=-1)
    student_logp = log_softmax(student_logits / T, dim=-1)
    soft = kl_div(student_logp, teacher, reduction="batchmean") * (T * T)
    hard = cross_entropy(student_logits, labels)
    return alpha * soft + (1.0 - alpha) * hard
```

The production version masks padding, normalizes by valid tokens, handles sharded vocabulary, controls teacher precision, and avoids storing full teacher logits when bandwidth dominates.

Why combine soft and hard targets

Soft targets transfer relative preference among tokens. Hard labels preserve ground truth or verified behavior when the teacher is wrong. The balance depends on teacher reliability, student capacity, domain shift, and whether labels represent a unique answer.

For generated sequence data, a hard next-token label from a teacher sample discards uncertainty. Offline logits preserve more information but are enormous. Alternatives include top-k logit storage, quantized logits, on-policy teacher queries, sequence-level ranking, hidden-state targets, or verifier-selected trajectories.

Sequence and feature distillation

Sequence distillation trains on teacher-generated outputs. It can simplify the target distribution and transfer style or reasoning patterns, but it inherits teacher errors and may reduce diversity.

Feature distillation aligns hidden representations or attention maps, which can help when architectures are compatible. Layer mapping, scale, and representational non-identifiability complicate it.

Online distillation queries the teacher on student-relevant states, reducing mismatch between a fixed corpus and the student's current failure modes. It costs teacher inference and creates a moving data distribution.

Draft-model distillation

A speculative draft is evaluated jointly with the target. The most relevant objectives are acceptance rate, average accepted prefix, draft latency, target verification cost, memory footprint, and final exactness.

Train on real product prompts and target continuations. Include high-entropy positions, domain slices, long-context states, and student failure cases. Match tokenizer and vocabulary unless the system explicitly supports a mapping; otherwise verification and KV reuse become much harder.

An effective loss can combine hard target tokens, forward KL on target logits, and emphasis on positions that control rejection. Evaluate under the actual sampling policy. A draft that agrees under greedy decoding may disagree at production temperature.

THE BEST DRAFT IS NOT THE BEST SMALL MODEL

The best draft maximizes end-to-end target throughput under memory and latency constraints. Standalone perplexity is a proxy; acceptance per microsecond is closer to the objective.

Model-size tradeoff

A larger draft costs more per proposed token but can raise acceptance. A smaller draft is cheap but may create verification waste. Let:

- $C_d(m)$ be draft cost for model size m ;
- $C_v(\gamma)$ be target verification cost for γ proposed tokens;
- $A(m, \gamma)$ be expected accepted tokens;
- C_{base} be one ordinary target decode step.

A simple speed model is:

$$\text{speedup} = A(m, \gamma) * C_{base} / (C_d(m, \gamma) + C_v(\gamma))$$

The model omits queueing and overlap but identifies the decision. Benchmark a grid of draft size and proposal length using production shapes. Watch memory capacity: loading a larger draft can reduce target batch size and erase its acceptance benefit.

Anti-distillation and extraction resistance

"Anti-distillation" covers techniques that make model extraction less effective: rate limits, query anomaly detection, output perturbation, watermarking, restricted logit access, policy controls, and legal or contractual measures. There is no free defense. Perturbing outputs can harm users and may not stop a determined extractor. Detection must distinguish legitimate high-volume customers from imitation workflows.

The Principal framing is a threat model: attacker access, budget, query adaptivity, target fidelity, and acceptable product degradation. Treat claims of protection as measured risk reduction, not impossibility.

Principal Interview Review

- 1 Explain forward versus reverse KL using behavior and gradients, not only slogans.
- 2 Why multiply a temperature-scaled KL term by T^2 ?
- 3 Design a draft-model training corpus and evaluation.
- 4 When can a 3B draft outperform a 1B draft end to end?
- 5 What defenses against extraction preserve normal user quality?

CHAPTER 07

07

Post-Training and Alignment

Post-Training and Alignment

Post-training chooses behavior among capabilities the base model can express. The objective, data, and evaluation must agree on what "better" means.

Supervised fine-tuning

SFT is appropriate when experts can demonstrate the desired behavior. It teaches format, policy, tool use, domain tone, and solution trajectories. High-quality demonstrations often beat a much larger weak corpus.

Mixing must protect broad capability. Include replay from general instruction data when specializing. Mask user or tool tokens according to the learning objective. For multi-turn data, test whether the loss incorrectly rewards copying system content or private tool outputs.

Preference data

Pairwise preferences can be collected from humans, verifiers, judges, or product behavior. Each source has bias. Human labels reflect rubric clarity and annotator context. Model judges may favor verbosity, familiar style, or their own outputs. Engagement data is confounded by ranking, position, and user intent.

Build calibration sets with expert adjudication. Measure agreement by slice. Retain ties and uncertainty rather than forcing every comparison into a binary label.

RLHF and direct preference objectives

Classic RLHF trains a reward model and optimizes a policy while constraining divergence from a reference. It supports online sampling and sequence-level reward, but it is operationally complex and vulnerable to reward hacking.

Direct Preference Optimization converts pairwise preferences into a supervised-style objective relative to a reference policy. It removes the explicit reward-model-and-RL loop, simplifying training. Other objectives alter the link function, treatment of a reference, margin, or unpaired feedback.

Do not choose by acronym. Ask:

- Is reward available only at the sequence level?
- Must the policy explore new outputs?
- How reliable and stationary is the preference signal?
- Is an explicit reward model useful for analysis or reuse?
- How much divergence from the base model is safe?

Verifiable reward and reasoning

Code execution, theorem checks, database answers, and simulated environments provide strong outcome signals. They still permit shortcut exploitation. Split generators and verifiers, randomize hidden tests, audit suspiciously short or repetitive traces, and evaluate on tasks unavailable to the training loop.

Process supervision labels intermediate steps and can improve diagnosis, but step correctness may be ambiguous and expensive to annotate. Outcome supervision is cheaper when final answers are verifiable but provides sparse credit. Hybrid designs use outcome reward plus auxiliary signals for format, tool validity, or progress.

A BETTER REWARD SCORE IS NOT AUTOMATICALLY A BETTER MODEL

Optimization changes the policy distribution, so the reward model is evaluated out of its training distribution. Track independent capability and safety metrics, divergence, response length, and exploit signatures.

Online learning and rollback

Online post-training creates feedback loops. New policy outputs influence the data later used to judge or train it. Protect a stable holdout, log exposure probabilities, version policies and reward models, and preserve the ability to reconstruct which policy generated each sample.

Use canaries and conservative trust regions. A rollback should restore model, tokenizer, tool policy, prompt templates, safety configuration, and serving parameters as one versioned release.

Principal Interview Review

- 1 When does DPO simplify a problem that does not require online RL?
- 2 Design defenses against reward hacking for a code reasoning model.
- 3 How do you audit a model judge for verbosity bias?

- 4 What belongs in an atomic rollback for an aligned assistant?
- 5 Explain why preference optimization can reduce capability even as reward rises.

PART III - LLM INFERENCE SYSTEMS

Part II - LLM Inference Systems

Inference turns a fixed model into a variable, stateful, multi-tenant service. Prompt length, output length, sampling policy, cache state, adapter choice, arrival bursts, and hardware topology change the cost of every request. The systems problem is to preserve a declared quality and latency contract while sharing expensive accelerators efficiently.

This part is self-contained. Model phases, KV state, batching, decoding semantics, parallel placement, compression, and operations are developed here from first principles; the only prerequisites are transformer basics and comfort with back-of-envelope arithmetic. Part IV writes the kernels that execute these decisions on one GPU. This part owns the request, the fleet, and the contract. Each part can be read alone.

The running service used throughout

Every section returns to one illustrative deployment so estimates stay comparable. Treat the numbers as a teaching configuration, not a product benchmark.

Item	Value in this part
Model	dense 7B-class transformer: $L = 32$ layers, hidden $D = 4096$, GQA with $H_{kv} = 8$, head dim $d = 128$
Weights	FP16, approximately 14 GB
KV cost	$2 * L * H_{kv} * d * 2 \text{ bytes} = 128 \text{ KiB}$ (about 131 kB) per cached token
Replica	one 80 GB accelerator, ~3 TB/s HBM, ~50 GB/s usable inter-node path
Workload	median prompt 2,000 tokens with a shared 500-token system prefix; median output 300 tokens; bursty arrivals
SLOs	streaming; p99 time to first token $\leq 1.5 \text{ s}$; p99 inter-token gap $\leq 100 \text{ ms}$

The model geometry deliberately matches the running example of Part IV, so a reader moving between the parts sees the same tensors from the kernel side and from the service side.

This part follows a request through that service, from API admission to token streaming and cleanup. Every chapter ends with principal-level interview questions and complete answer keys.

THE SERVING CONTRACT COMES FIRST

Before choosing an engine or optimization, define model semantics, supported request features, latency objectives, overload behavior, and the quality deviations that are permitted. A faster system that silently changes these is a different product.

Map of this part

Section	Role in the story
Lifecycle and metrics	Define the request path and the measurements every later claim uses
Prefill, decode, modeling	Build the bytes-and-FLOPs model of one request on one replica
KV cache and prefix reuse	Manage the working set that decode carries between steps
Scheduling and admission	Share one replica across requests without breaking the contract
Sampling and speculation	Preserve decoding semantics while accelerating token selection
Parallel and disaggregated serving	Place phases and state across devices and pools
Quantization and adapters	Shrink bytes and multiply variants without changing the product
Production and reliability	Operate the whole system through failures and rollouts

CHAPTER 08

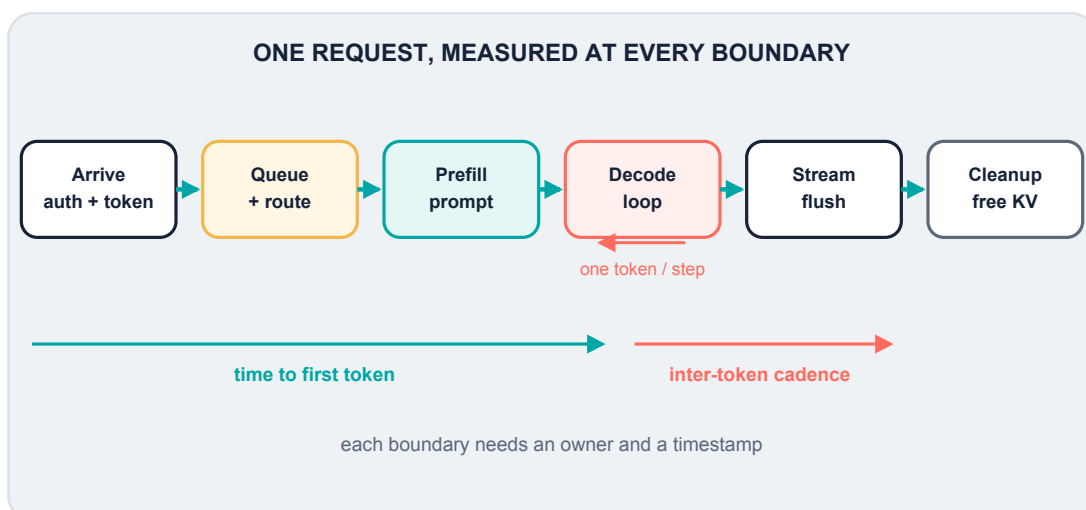
08

Request Lifecycle, Metrics, and Workload Models

Request Lifecycle, Metrics, and Workload Models

An LLM request is a distributed transaction with a long, streaming response. Correct measurement separates queueing, model phases, transport, and user-visible cadence instead of collapsing everything into one throughput number.

The map starts here because every optimization in this part is judged against these measurements. For the running service the contract is concrete: a request must show its first token within 1.5 seconds at the 99th percentile and must never let the visible gap between tokens exceed 100 milliseconds.



A request crosses gateway, queue, prefill, decode, and streaming boundaries. Each boundary needs an owner and a timestamp before any latency claim is meaningful.

The request path

A production request usually crosses these stages:

- 1 authenticate, rate-limit, and validate the request;
- 2 normalize the prompt, apply the chat template, and tokenize;
- 3 route by model, adapter, tenant, locality, and capacity;
- 4 wait in an admission or execution queue;
- 5 allocate or reuse KV state and run prefill;
- 6 sample and stream the first token;
- 7 repeat scheduling, decode, postprocessing, and streaming;
- 8 stop on an end token, length limit, constraint, cancellation, or error;
- 9 release pages, adapter references, graph slots, and accounting state.

Each boundary needs an owner and a timestamp. Without this, a regression in tokenization, routing, queueing, networking, or client backpressure may be misdiagnosed as a slow GPU.

The request contract includes more than input text. It includes model and tokenizer versions, chat template, adapter, maximum context and output, stop rules, logits processors, random seed, sampling parameters, structured-output grammar, priority, deadline, streaming mode, and any approximation policy. These fields affect both semantics and execution shape.

Latency definitions

Let a be arrival at the service boundary, s the first model-service start, f the first token visible at that boundary, and o_i the visible time of output token i .

Metric	Definition	What it reveals
Queue delay	$s - a$	Admission pressure and scheduler delay
Time to first token	$f - a$	Queueing, preprocessing, prefill, first sampling, and transport
Inter-token latency	$o_i - o_{(i-1)}$	Per-step cadence and stalls
Time per output token	Usually $(o_{\text{last}} - f) / (N - 1)$	Average generation speed after the first token
End-to-end latency	$o_{\text{last}} - a$	Total user wait for a completed response

Time per output token is an average; it can hide a one-second generation stall among many fast tokens. Report an inter-token latency distribution or a maximum-gap metric for interactive products. Also define whether the first token means sampled on the GPU, serialized by the server, received by the gateway, or observed by the client. A metric without a boundary is ambiguous.

For non-streaming clients, end-to-end latency matters most, but internal first-token and cadence metrics still locate faults. For streaming clients, a reasonable SLO often has separate thresholds for time to first token and inter-token latency.

Decompose the running service's budget before optimizing anything. Of the 1.5-second first-token allowance, transport and tokenization cost tens of milliseconds and prefill for a 2,000-token prompt costs tens to low hundreds of milliseconds on a healthy replica. The remainder - often more than a second - is queueing and scheduling slack. That allocation is why later sections spend more effort on admission, batching, and cache reuse than on shaving prefill kernels.

Throughput, goodput, and capacity

Report input tokens per second and output tokens per second separately. They exercise different phases and cannot be added without context. Requests per second is meaningful only with a declared prompt/output distribution. Per-GPU token throughput is useful for efficiency, while per-user output rate describes interactivity.

Goodput is work that satisfies the product contract. One useful definition is:

`goodput = requests meeting every required SLO / measurement interval`

A request that eventually completes but violates its time-to-first-token or token-cadence requirement consumes capacity without contributing to goodput. This makes goodput a better autoscaling and architecture objective than raw throughput under strict latency objectives.

Capacity is not the peak point on a throughput curve. It is the highest sustained arrival process for which tail latency, error rate, memory reserve, quality, and fairness remain within contract. State the percentile and duration: for example, 99th-percentile first-token latency under a thirty-minute replay with a specified burst model.

Open-loop and closed-loop load

In a **closed-loop** test, each simulated client waits for completion before sending another request. When the service slows, offered load falls. This is useful for user-session modeling but can hide overload.

In an **open-loop** test, arrivals follow a process independent of completions. Queueing is visible because offered load does not retreat when latency rises. Use open-loop tests to find sustainable capacity, but choose realistic burstiness rather than only a smooth Poisson stream.

Avoid coordinated omission: a load generator that pauses arrivals while the server is slow fails to count requests that would have arrived during the stall. Record intended arrival time separately from actual send time, and include client-side queue delay if the generator cannot keep up.

A workload is a joint distribution

Average prompt length and average output length are insufficient. The service cost depends on their joint distribution and correlations with:

- arrival time and burst size;

- model, adapter, and tenant;
- sampling or structured-output policy;
- prefix-reuse probability and shared-prefix length;
- cancellation point and client backpressure;
- speculative acceptance;
- multimodal input size;
- priority and deadline.

Long prompts and long outputs create different pressure. A long prompt is a large prefill event; a long output holds KV memory and a decode slot for many iterations. A few long-tail requests can control memory reserve and tail latency even when token-weighted averages look benign.

Build at least three workload views: a stable production replay, a stress distribution that magnifies tails, and synthetic shape sweeps for causal explanation. Preserve request correlations in replay. Independently sampling each feature can create combinations that never occur or remove the bursts that matter.

Trace decomposition

Use a request identifier across gateway, scheduler, worker, and stream transport. A useful span model includes:

```
gateway -> tokenize -> route -> queue -> prefill -> first sample ->
decode iterations -> stream flush -> cleanup
```

Within the worker, attach phase, batch composition, input/output token counts, allocated and reused KV tokens, model/adapter version, graph or engine variant, device group, and termination reason. Sample detailed traces if volume is high, but retain aggregate histograms for every request class.

Tail diagnosis should compare a slow request with the batch and node state around it. Per-request latency alone cannot reveal that a short decode waited behind a long prefill, a cache eviction triggered recomputation, or a peer stalled a tensor-parallel collective.

Benchmark protocol

A defensible benchmark declares:

- exact model, tokenizer, precision, engine, kernels, drivers, and hardware topology;
- prompt/output distributions, sampling policy, and dataset transformations;
- concurrency or arrival process, duration, warmup, and random seeds;
- inclusion boundaries for tokenization, network, queueing, and detokenization;
- cold-start and warm steady-state results;
- latency percentiles, inter-token gaps, throughput, goodput, errors, and memory;

- output correctness or quality checks.

Run both a component benchmark and an end-to-end replay. Component results explain a mechanism; the replay proves that scheduling, transport, memory pressure, and application features do not erase the gain.

TOKENS PER SECOND CAN BE MANUFACTURED

Changing prompt/output mix, ignoring queue time, omitting rejected requests, reducing sampling work, or reporting a saturated batch with no latency constraint can raise tokens per second without improving a real service.

Principal Interview Review

- 1 Define time to first token, inter-token latency, time per output token, and end-to-end latency at precise boundaries.
- 2 Why is goodput often a better capacity objective than raw token throughput?
- 3 Design a load test that exposes overload without coordinated omission.
- 4 What workload correlations must a production replay preserve?
- 5 A GPU trace looks healthy, but user latency regressed. How do you localize the problem?
- 6 What belongs in a reproducible inference benchmark report?

Answer Key

1. Latency metrics

At the chosen service boundary, time to first token is arrival until the first emitted token. Inter-token latency is the gap between consecutive visible tokens. Time per output token is usually the average generation interval after the first token, while end-to-end latency is arrival until the final token or terminal response. State whether the boundary is worker, gateway, or client and whether tokenization and transport are included. Report an inter-token distribution because its average hides generation stalls.

2. Goodput

Raw throughput counts work even when requests miss latency, error, fairness, or quality requirements. Goodput counts only requests that satisfy every required SLO, so it captures the useful capacity of the system. At overload, batching can raise raw tokens per second while queue delay causes nearly every interactive request to miss its first-token target. Autoscaling against goodput or SLO attainment avoids rewarding that state.

3. Load testing overload

Use an open-loop arrival generator whose intended send times do not depend on response completion. Replay realistic prompt/output correlations and burstiness, increase offered load across a sweep, and measure queue growth, tail latency, goodput, errors, and memory reserve. Timestamp intended and actual send times so generator delay is visible. Run long enough to expose slow leaks and output-length tails, and include a closed-loop session test separately when it reflects product behavior.

4. Workload correlations

Preserve at least prompt versus output length, request features versus shape, tenant/model/adaptor locality, prefix sharing, arrival bursts, cancellation behavior, and speculative acceptance. Long prompts may cluster in batch jobs, a specific adaptor may have longer outputs, or a shared system prompt may create most prefix hits. Breaking these relationships changes queueing, memory lifetime, cache behavior, and batch compatibility.

5. Healthy GPU, slow users

Trace from the user boundary inward. Split latency into gateway, tokenization, routing, queue, model phases, sampling, serialization, network flush, and client backpressure. Correlate the slow request with scheduler state, batch composition, cache events, collective peers, and node health. A well-utilized GPU can coexist with excessive queueing or stream buffering; utilization proves activity, not useful latency.

6. Reproducible report

Record the model/tokenizer and semantic settings, hardware and topology, software stack and versions, precision and engine configuration, complete workload distribution, arrival process, warmup, measurement boundary, seeds, and all exclusions. Report latency percentiles and gaps, input/output throughput, goodput, errors, memory, power if relevant, and quality validation. Include raw traces or enough artifacts to reproduce them, plus cold and steady-state behavior.

CHAPTER 09

09

Prefill, Decode, and Performance Modeling

Prefill, Decode, and Performance Modeling

A request moves through phases with different shapes and bottlenecks. Prefill exposes token parallelism and often high arithmetic intensity; decode is sequential across time and repeatedly reads weights and growing KV state.

The lifecycle defined where time can go; this section models how much the model itself must cost. The tool is the same resource ledger used throughout the book: count the bytes each phase must move and the FLOPs it must execute, and let the running service make the quantities concrete.

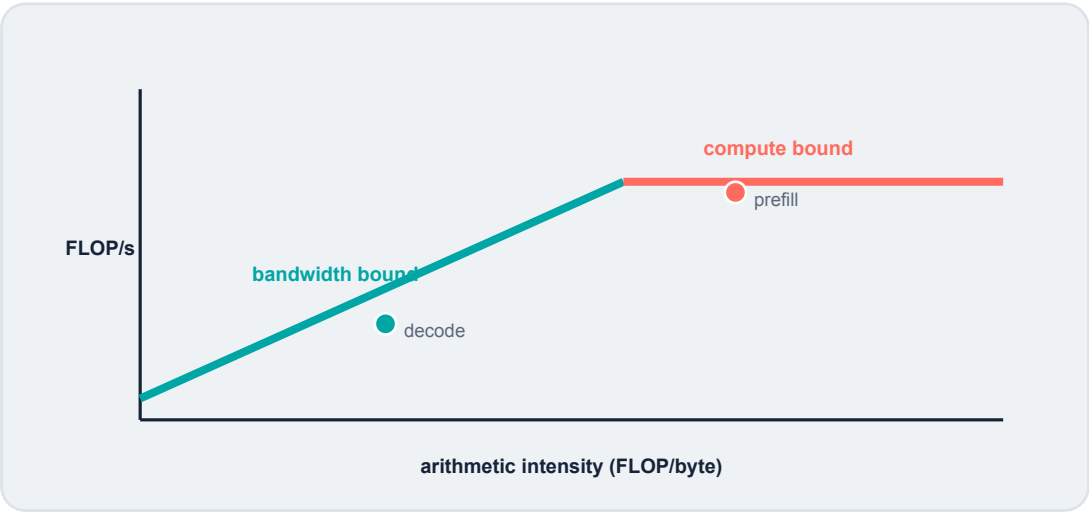
Phase anatomy

Text-only autoregressive serving has two principal model phases:

- **Prefill** processes all uncached prompt tokens, produces the first-token state, and materializes their KV cache.
- **Decode** adds one token per active sequence per iteration, reads prior KV state, samples a token, and appends new KV.

Multimodal models may add an encoder phase before prefill. Image, audio, or video preprocessing and encoding can dominate first-token latency, expand into many model tokens, and require a different accelerator profile. Treat encode, prefill, and decode as distinct stages even when one engine executes all three.

Property	Prefill	Decode
Token parallelism	Many prompt positions	One new position per sequence
Typical linear shape	Large token dimension	Small token dimension, often batch-sized
Common limit	Compute or attention IO	Weight/KV bandwidth, collectives, launch overhead
KV behavior	Creates prompt state	Reads history and appends one position
Primary user metric	Time to first token	Token cadence and output rate



Arithmetic intensity helps explain why a large prefill can approach compute throughput while small-batch decode remains limited by bytes moved per generated token.

Transformer work per phase

Each layer performs normalization, projections for attention, attention itself, an output projection, another normalization, and an MLP, with residual paths and optional routing for mixture-of-experts models.

For a dense model, most parameter FLOPs occur in linear layers. Prefill applies them to many prompt positions at once, creating GEMM shapes with substantial reuse. Decode applies them to one new position per sequence, so the effective token dimension is approximately active batch size.

Attention has a different scaling. For sequence length S and head dimension d , full causal prefill performs work proportional to $S^2 d$ but IO-aware kernels avoid materializing the full score matrix. One decode query attends to S cached keys and values, so per-step attention work and KV traffic grow approximately linearly with context length. At very long context, attention can dominate a decode step even when weight reads dominate at shorter context.

Arithmetic intensity and roofline reasoning

Arithmetic intensity is useful FLOPs divided by bytes moved from the limiting memory level. A first-order roofline is:

attainable compute rate $\leq \min(\text{peak compute}, \text{bandwidth times arithmetic intensity})$

For an unbatched decode linear layer, a weight may be fetched for only one multiply-accumulate before the next layer. Batching sequences reuses each weight across rows, raising intensity. Weight quantization reduces bytes but adds scale reads and dequantization. A fused epilogue removes intermediate reads and writes. The correct denominator is bytes actually transferred, not tensor size on paper.

Prefill matrix multiplications can become compute-bound because tiled kernels reuse weights and activations. Attention can remain IO-bound if an implementation materializes scores or makes extra HBM passes. The roofline identifies a bound, not a schedule: low achieved performance may still come from poor tiling, serialization, launch gaps, or an imbalanced distributed peer.

A lower-bound model for decode

Let W be active model weight bytes per replica or model-parallel group and BW_w sustainable bandwidth for those reads. For small-batch decode:

$t_{\text{weights}} \geq W / BW_w$

Let $K(S, B)$ be KV bytes read at current context and batch, BW_{kv} the sustainable bandwidth for that access pattern, C collective time, and O launch, scheduler, sampling, and synchronization overhead. A deliberately conservative step model is:

$t_{\text{step}} \geq \max(t_{\text{weights}}, t_{kv}, t_{\text{compute}}) + C + O$

Some traffic and compute overlap, so do not blindly sum all resource bounds. Measure which terms overlap on the real engine. The model is valuable because it predicts how batch, precision, context, and topology should move the result.

If latency is near the weight-read bound, optimizing scalar arithmetic cannot produce a large gain. Reduce bytes, increase batch reuse, improve placement, or change the model. If latency is far above every bound, examine kernel efficiency, launch gaps, collectives, and scheduling.

The running service's step floor

Plug in the numbers. Weights are approximately 14 GB in FP16 and the replica sustains roughly 3 TB/s from HBM, so a small-batch decode step cannot beat:

$t_{\text{weights}} \geq 14e9 / 3e12 = 4.7 \text{ ms}$

That is a ceiling near 210 steps per second, shared by every sequence in the batch. KV traffic starts small by comparison: at batch 8 with 4,096-token contexts, a step reads about $8 * 4096 * 131 \text{ kB} = 4.3 \text{ GB}$, roughly 1.4 ms of additional bandwidth time. The two terms cross when the batch holds about 107,000 cached tokens ($14 \text{ GB} / 131 \text{ kB}$) - for example, eight sequences of about 13,000 tokens each. Below that point the service is mostly buying weight reads; beyond it,

context length governs the step. This one number reappears in the KV, quantization, and disaggregation sections.

KV-cache accounting

For layers L , batch B , cached sequence length S , KV heads H_{kv} , head dimension d , and element bytes b :

$$KV_bytes = 2 \cdot L \cdot B \cdot S \cdot H_{kv} \cdot d \cdot b$$

The factor two represents keys and values. For heterogeneous sequence lengths, replace $B \cdot S$ with the sum of cached tokens across sequences. Add page-table metadata, partial blocks, allocator reserve, prefix-cache entries, speculative branches, and temporary workspaces.

This equation makes architectural effects explicit. Grouped-query or multi-query attention reduces H_{kv} . Lower KV precision reduces b . Sliding-window attention caps effective S for selected layers. None of these is free: each changes model quality, kernels, or both.

For the running service the equation gives $2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 131$ kB per cached token. A median request holding 2,300 tokens of context therefore occupies about 300 MB, and after weights and workspace the 80 GB replica has room for roughly 450,000 cached tokens - about 200 median conversations. Every architectural lever above moves one factor of that product.

Batch as a reuse and latency control

Increasing active decode batch can reuse weights and amortize launches, but it also:

- increases per-iteration duration;
- consumes more KV capacity;
- may raise collective message size;
- delays newly arrived work until a scheduling boundary;
- makes one iteration include more heterogeneous states.

Therefore the useful batch is SLO-dependent. Throughput-oriented jobs may use a large batch near saturation. Interactive traffic often needs a smaller iteration budget or a deadline-aware scheduler. Report the full throughput-latency curve rather than one maximum-throughput point.

Long prompts and chunked prefill

A long prefill is efficient in isolation but can monopolize an accelerator long enough to stall active decodes. Chunked prefill divides uncached prompt tokens into bounded pieces and interleaves them with decode iterations. The chunk size trades:

- smaller generation gaps versus better prefill kernel efficiency;
- earlier service for other requests versus later first token for the long request;
- stable iteration shapes versus more scheduling and state transitions.

Chunk by a measured compute or time budget rather than assuming equal token chunks cost the same. Attention work changes with prefix length, and multimodal tokens or architecture-specific layers may have different cost.

Prefix reuse changes prefill accounting

When a request reuses cached state for S_{hit} prefix tokens and has S_{new} uncached tokens, its prefill work is determined primarily by S_{new} plus attention against the reused history. A high token hit rate can reduce compute while consuming substantial cache memory. Distinguish:

- request hit rate;
- reused-token fraction;
- prefill FLOPs avoided;
- first-token time saved;
- bytes occupied and eviction cost.

Routing a request away from a useful prefix may cost more than waiting briefly for the node that holds it. Cache locality becomes a scheduling input rather than a storage statistic.

Phase-specific optimization

Prefill commonly benefits from efficient attention, large matrix shapes, fused projections and epilogues, balanced long-prompt partitioning, prefix reuse, and topology that supports compute-heavy parallelism.

Decode commonly benefits from continuous batching, lower weight and KV bytes, efficient small-row matrix operations, paged attention, launch amortization, graph capture, and carefully chosen model parallelism. At long context, decode attention and KV placement deserve separate analysis from weight-dominated layers.

NAME THE LIMITING RESOURCE BY PHASE AND SHAPE

"The model is memory-bound" is too coarse. State whether the critical shape is short-prompt prefill, long-prompt attention, small-batch decode, long-context KV reads, a collective, or host scheduling. Each points to a different intervention.

Principal Interview Review

- 1 Why can the same model be compute-bound in prefill and bandwidth-bound in decode?
- 2 Derive the KV-cache memory equation and explain every omitted overhead.
- 3 How does decode batch size change arithmetic intensity and user latency?
- 4 When does long-context decode stop being mainly a weight-streaming problem?
- 5 Design a chunked-prefill policy for interactive and batch traffic.

6 How should a multimodal request change phase accounting and capacity planning?

Answer Key

1. Different phase bounds

Prefill applies weight matrices to many token rows, so tiled GEMMs reuse weights and can reach high arithmetic intensity. Decode adds one row per sequence; at small batch, weights and KV state are moved for relatively little new computation, so bandwidth and launch overhead dominate. Long prompts can make prefill attention IO-limited, and a large decode batch can move linear layers toward compute-bound. The answer must be shape-specific rather than an absolute label for the model.

2. KV memory

For L layers, B equal-length sequences of S cached tokens, H_{kv} KV heads, dimension d , and b bytes per element, keys plus values require $2 \cdot L \cdot B \cdot S \cdot H_{kv} \cdot d \cdot b$ bytes. With variable lengths, sum cached tokens instead of multiplying by $B \cdot S$. Add partially filled pages, allocator reserve, block tables, shared-prefix reference metadata, speculative branches, quantization scales, temporary attention workspace, and replication or sharding effects.

3. Decode batch

More active sequences reuse each fetched weight across more rows and amortize launch and collective overhead, which raises arithmetic intensity and throughput. The iteration also becomes longer, consumes more KV memory, and may delay admission or token emission. Sweep batch under the actual latency SLO and context distribution; choose the largest budget that improves goodput rather than the batch with the highest unconstrained tokens per second.

4. Long-context decode

Weight traffic is roughly stable per step, while each query reads a KV history that grows with context. When measured KV read time or attention compute approaches and then exceeds the weight-streaming term, attention controls the step. GQA, KV precision, sliding windows, cache layout, page locality, and attention parallelism then matter more. Use per-layer profiling and a bytes model; total model bandwidth alone cannot identify the crossover.

5. Chunked prefill

Reserve a decode budget that maintains the declared maximum token gap, then spend remaining per-iteration compute or token budget on one or more prefill chunks. Adapt chunk size to decode pressure, estimated chunk time, prompt progress, priority, and pipeline shape. Age long prompts so they cannot starve, reserve KV before executing a chunk, and measure both cadence improvement for active decodes and added first-token latency for chunked requests.

6. Multimodal accounting

Add preprocessing and encoder time, memory, and queueing as a separate phase. Account for media-size distributions and the number of model tokens produced by the encoder, which affect later prefill and KV. The encoder may need different hardware or batching, and large media payloads affect network and host memory. Capacity tests must preserve correlations between media size, text length, output length, and model variant; text token counts alone understate cost.

CHAPTER 10

10

KV Cache, Paging, and Prefix Reuse

KV Cache, Paging, and Prefix Reuse

KV state is the working set of autoregressive inference. Its lifetime follows requests, not batches, so allocation, sharing, eviction, placement, and security are first-class serving decisions.

The previous section priced KV by the byte; this section manages its lifetime. On the running replica, 131 kB per token means one long conversation can hold hundreds of megabytes hostage, and the shared 500-token system prefix costs about 65 MB per copy - large enough that whether it is shared or duplicated is a capacity decision, not a detail.

Logical state and physical storage

Each cached token contributes a key and value vector at every attention layer. Logically, a sequence sees positions in order. Physically, those positions do not need to occupy one contiguous allocation. A serving engine can divide KV into fixed-size token blocks, allocate physical blocks on demand, and translate logical block numbers through a per-sequence table.

Paging avoids reserving the maximum possible output for every request and avoids copying the entire cache whenever a sequence grows. It also enables prefix sharing and copy-on-write branching. The attention implementation must accept block tables or an equivalent indirection, so memory management and kernel layout are designed together.

The term **page** may refer to a token block across selected layers, a per-layer allocation, or an engine-specific unit. Always state the unit, byte size, and placement. The important contract is stable logical order over movable or non-contiguous physical storage.

Fragmentation

Three forms matter:

- **Internal fragmentation** is unused space inside the last allocated block of a live sequence.
- **External fragmentation** is free memory that cannot satisfy an allocation because of size or placement constraints.
- **Reserved slack** is intentionally unallocated or pre-reserved capacity for growth and failure recovery.

With equal fixed-size blocks, external fragmentation in the KV pool is greatly reduced, but internal waste is bounded by at most one partially filled block per independent sequence or branch. Metadata, alignment, and separate pools can reintroduce constraints. Report effective usable KV capacity from the live allocator, not free bytes from a device-wide monitor.

Block-size tradeoffs

Smaller token blocks reduce tail waste, improve the granularity of prefix sharing, and make cancellation reclaim memory sooner. They increase block-table size, allocator operations, address translation, scatter in physical access, and scheduling metadata.

Larger blocks reduce metadata and can improve contiguous access, but waste more space on short or terminated sequences and require longer exact prefix matches before a whole block is reusable. The optimum depends on the sequence-length distribution, layer layout, attention kernel, and reuse pattern. Benchmark block sizes under memory pressure, not only with an empty cache.

Allocation and reservation invariants

A robust allocator maintains:

- no live sequence or in-flight kernel references a freed block;
- reference counts protect shared prefixes and speculative branches;
- copy-on-write occurs before a branch mutates shared state;
- reservation makes the next scheduled iteration memory-safe;
- cancellation and errors eventually release every owned reference;
- block tables are updated before kernels consume them, with explicit ordering;
- accounting attributes bytes to model, tenant, request, and cache class;
- migration or restore preserves token order, position state, dtype, and model identity.

Reserve before launch rather than discovering out-of-memory after some requests in a batch have advanced. For uncertain output length, reserve a bounded horizon and repeat admission checks. A reservation is a promise to the scheduler, not necessarily immediate physical zero-filling.

Prefix cache identity

KV for a prefix is reusable only when the computation that produced it is identical under the product contract. A key commonly includes:

- model weights and engine-compatible model version;

- tokenizer and exact token IDs;
- chat template and special-token policy;
- adapter or prompt-tuning state;
- positional encoding configuration and absolute offset;
- attention mode, relevant multimodal embeddings, and cache dtype;
- tenant or security namespace and cache salt;
- any hidden input that changes activations.

Text equality is insufficient because normalization or templates can change token IDs. Token equality can still be insufficient when adapter, image embedding, position, or model version differs. Cache identity should be a versioned schema with test vectors.

Lookup structures and partial hits

A trie or radix tree naturally represents shared token prefixes. A block-hash index maps full token blocks and their parent identity to cached blocks. Both can return the longest valid prefix rather than an all-or-nothing hit.

Insert only committed state. A cancelled or speculative branch may have computed tokens that were never part of the accepted sequence. Mark blocks immutable once published for reuse; append new private blocks and publish them only after the corresponding tokens and semantic state are committed.

Measure four distinct outcomes:

- 1 request-level hit rate;
- 2 reused-token fraction;
- 3 prefill time or FLOPs avoided;
- 4 net first-token improvement after lookup, routing, and transfer.

A high hit rate on tiny prefixes may save little. A lower hit rate on long system prompts can be far more valuable.

The running service makes the distinction concrete. Its shared 500-token system prefix costs about 65 MB to retain and saves 500 tokens of prefill on nearly every request. A per-user greeting of a few tokens may hit just as often while saving almost nothing and fragmenting the pool with tiny published blocks.

Eviction as value density

Least-recently-used eviction is a baseline, not a universal optimum. The value of an entry depends on expected reuse probability, recomputation cost, bytes, transfer locality, and the opportunity cost of displacing active KV.

A practical priority score may increase with saved prefill time and recent or known future reuse, and decrease with bytes and memory pressure. Pinning known system prompts can help, but

permanent pins can crowd out active requests. Set budgets by cache class and expose why an entry survived or was evicted.

Prefix caching competes with decode capacity. Under pressure, retaining a reusable prefix may force rejection of a live request or reduce batch size. Admission and eviction need one memory objective, not separate local heuristics.

Routing and cache locality

In a replicated fleet, the router chooses between queue delay and cached state. Define an estimated completion score such as:

```
cost = queue_time + transfer_time + uncached_prefill_time +  
policy_penalty
```

Sticky routing maximizes locality but can create hot spots. Pure least-loaded routing destroys reuse. A cache-aware router considers both, places common prefixes on enough replicas, and falls back when a cache owner is unhealthy or overloaded.

Do not transfer a prefix merely because it exists elsewhere. Compare transfer time and network contention with recomputation. For a short prefix or a compute-rich prefill worker, recomputing may be cheaper and simpler.

Offload and migration

KV can be tiered across accelerator memory, host memory, local storage, or remote memory. Offload is useful when pause time is long enough to hide transfer and recomputation is more expensive. It is harmful when every decode step faults pages across a slow link.

For a state object of X bytes over sustainable bandwidth BW , a lower bound is X / BW , plus registration, serialization, queueing, and synchronization. Migration should transfer only live and required blocks, preserve reference ownership, and cancel cleanly if the destination can no longer admit the request.

A session cache and an active decode cache have different latency needs. Separate tiers and policies so inactive conversations cannot evict the working set of interactive generation.

Security and side channels

Cross-request reuse creates an information boundary. A tenant must not infer another tenant's prompt through timing, hit metadata, or shared identifiers. Namespace or salt cache keys, authorize sharing explicitly, avoid returning cache-hit details to untrusted callers, and consider padding or disabling cross-tenant reuse for sensitive workloads.

Deletion requirements propagate to cache copies, offloaded tiers, and replicas. Logging raw token keys or prefix hashes can itself leak information when prompts have low entropy. Treat cache metadata under the same retention policy as model inputs.

A KV HIT IS NOT AUTOMATICALLY A WIN

The useful metric is first-token time or cost saved after lookup, routing, transfer, memory displacement, and security policy. Cache reuse that fragments capacity or creates a hot node can lower fleet goodput.

Principal Interview Review

- 1 Why does paged KV reduce waste, and what costs does it introduce?
- 2 How would you choose token-block size?
- 3 State the allocator invariants required for prefix sharing and speculative branches.
- 4 What must be included in a safe prefix-cache identity?
- 5 Design a cache-aware routing and eviction policy.
- 6 When should the system transfer, offload, recompute, or discard KV state?

Answer Key

1. Paging tradeoff

Paging allocates fixed-size blocks as sequences grow, so requests do not reserve maximum length and growth does not copy a contiguous cache. Tail waste is bounded by partially filled blocks, and blocks can be shared or released independently. Costs include block tables, allocation operations, address translation in attention, less contiguous physical access, metadata synchronization, and kernel complexity. The win must be measured with the actual block layout and memory pressure.

2. Block size

Sweep sizes against the production sequence and reuse distributions. Small blocks reduce internal fragmentation, reclaim sooner, and expose finer prefix hits; large blocks reduce metadata, allocation frequency, and translation overhead and may improve locality. Measure usable KV capacity, attention latency across context lengths, allocator CPU cost, block-table traffic, cache hit value, and cancellation reclamation. Choose by end-to-end goodput and SLOs, not fragmentation alone.

3. Sharing invariants

Published shared blocks are immutable and reference-counted. A branch increments references, performs copy-on-write before divergence, and publishes only accepted state. Scheduling reserves private growth blocks before launch. Freeing waits until no sequence or in-flight kernel can reference the block, with ordered block-table updates. Cancellation, rejection, and partial failure release every reference exactly once, and accounting remains attributable.

4. Cache identity

Use exact token IDs plus model and tokenizer versions, chat-template and special-token policy, adapter or prompt state, position configuration and offset, attention/cache mode and dtype, multimodal inputs that affect activations, and an authorization namespace or salt. Include any hidden feature that changes computed K/V. Version the identity schema so rollout cannot mix incompatible cache entries.

5. Routing and eviction

Estimate completion cost on candidate replicas from queue delay, reusable tokens, transfer or recompute time, memory pressure, and tenant policy. Replicate very hot prefixes enough to avoid affinity hot spots and use a no-cache fallback. Eviction should rank saved recompute time or known future value per byte while preserving active-request reserve and per-tenant limits. Continuously compare predicted savings with actual first-token improvement.

6. KV placement decision

Keep active working state on the accelerator. Transfer when another node can finish sooner after including network and admission cost. Offload when expected idle time can hide the round trip and recomputation is expensive. Recompute when the prefix is small, transfer links are contended, or local prefill is cheap. Discard when future reuse probability times saved work is below storage and displacement cost. Enforce deletion and tenant-isolation policy regardless of performance.

CHAPTER 11

11

Scheduling, Batching, and Admission Control

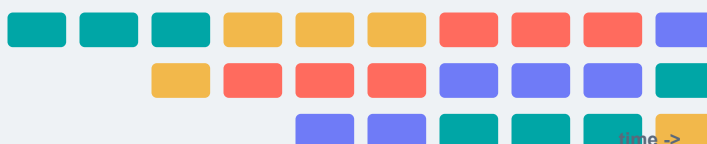
Scheduling, Batching, and Admission Control

The scheduler allocates compute time, KV memory, and latency slack across requests whose work is revealed incrementally. It is a policy engine with explicit invariants, not a queue attached to a GPU.

With phase costs and KV lifetime established, the scheduler is where they collide. Every iteration must divide the replica between the token cadence promised to active requests and the first-token latency promised to queued ones - the running service's two SLOs are exactly that tension, and its roughly 200-conversation KV capacity is the budget being divided.

Iteration-level scheduling

Autoregressive requests finish at different output lengths. A static batch keeps completed slots idle or delays their responses until the longest sequence finishes. Iteration-level scheduling rebuilds the active batch at token or bounded-chunk boundaries. Finished and cancelled requests leave; newly admitted work can enter.

STATIC BATCH**CONTINUOUS BATCH**

Continuous batching fills freed decode slots over time. The scheduler gains efficiency but must decide admission, phase mixing, fairness, and shape compatibility at every boundary.

An iteration may contain decode tokens, prefill chunks, encoder work, or verification tokens. The engine exposes feasible work units; the scheduler chooses a set under budgets for tokens, sequences, memory, execution time, graph shape, and distributed compatibility.

Scheduler state

Track per request:

- phase and tokens already processed;
- prompt remainder and generated length;
- deadline, priority, tenant weight, and arrival time;
- estimated next-iteration and remaining work;
- current KV blocks, reserved growth, and prefix ownership;
- model, adapter, sampling, grammar, and speculative mode;
- compatible engine/graph variants and device placement;
- cancellation and client-backpressure state.

Track global state such as free and reserved KV blocks, active batch composition, queue age, measured iteration-time curves, cache locality, device health, adapter residency, and SLO risk.

Token and time budgets

A maximum sequence count is a weak batching control because one prefill token and one long-context decode token have different cost. Use one or more measured budgets:

- total uncached prefill tokens;
- active decode sequences grouped by context bucket;

- predicted iteration time;
- KV blocks allocated now and at the next horizon;
- communication volume or pipeline microbatches;
- verification tokens for speculation.

Cost estimates need not be perfect, but they should be calibrated from telemetry and bounded conservatively near deadlines. When estimates drift after an engine or model update, the scheduler can violate SLOs even though kernels improved.

Phase mixing and chunked prefill

Decode-first scheduling protects token cadence but can starve new prefills under sustained generation. Prefill-first scheduling reduces queue delay for new arrivals but can create generation stalls. Chunked prefill bounds interference and creates a tunable compromise.

A robust policy first reserves enough decode work to protect imminent deadlines, then fills remaining predicted iteration time with prefill chunks. Aging increases the priority of waiting prefills. Batch jobs may use a separate queue and consume explicitly allocated slack rather than opportunistically overwhelming interactive traffic.

Pipeline-parallel execution may prefer iterations with similar compute to reduce bubbles. Chunk size should therefore consider stage balance as well as a single-device token budget.

The running service shows why the compromise is necessary. An uninterrupted 2,000-token prefill occupies the replica for tens to low hundreds of milliseconds, which alone can exceed the 100 ms inter-token budget of every active decode. Chunking that prompt into a few hundred tokens per iteration bounds each gap while adding modest total prefill time; the long request pays a slightly later first token so that dozens of active streams keep their cadence.

Fairness and priorities

FIFO is predictable but allows a huge prompt to delay many small requests.

Shortest-remaining-processing-time improves mean latency but can starve large jobs and relies on unknown output length. Earliest-deadline-first protects explicit deadlines when service time is estimable. Weighted fair service can allocate token or compute time across tenants.

Charge service in a resource-correlated unit, not requests. One long-context decode step can cost far more than a short-context step; one prefill can consume thousands of tokens. Maintain deficit or virtual-time accounting and add aging so low-priority work eventually progresses.

Priority inversion appears when a high-priority request depends on a low-priority resource: an adapter load, a shared prefix owner, a pipeline peer, or memory held by a paused request. The scheduler must either inherit priority, preempt or evict safely, or route around the dependency.

Admission control

Admission determines whether a request can enter without breaking promises already made. Estimate:

- prompt and encoder work;
- minimum and worst-credible KV growth;
- model and adapter residency;
- deadline slack and queue position;
- compatible replica capacity;
- tenant quota and global reserve.

Possible outcomes are admit, defer, route, downgrade under an explicit policy, truncate if the API allows it, or reject quickly with a retry signal. Do not admit unlimited work into an invisible queue. An honest overload response protects existing requests and gives upstream systems a chance to shed or reroute load.

Reserve memory for at least the next scheduling horizon. Reserving maximum output for every request wastes capacity; reserving nothing risks mid-generation failure. A probabilistic growth model can improve utilization, but keep a hard safety reserve and define what happens when predictions are wrong.

Cancellation and backpressure

Cancellation is a capacity feature. Propagate it from disconnected client to gateway, queue, scheduler, worker, and KV allocator. If a kernel cannot be interrupted, mark the request and release resources at the next safe boundary. Avoid publishing unaccepted speculative state or streaming tokens after cancellation.

A slow client can make stream buffers grow while the GPU continues generation. Set bounded buffers and a policy: pause the request, reduce its scheduling share, spill within limits, or cancel. Paused requests still occupy KV memory, so waiting indefinitely is not free.

Scheduler pseudocode

PYTHON

```
def schedule_tick(state, target_ms):
    candidates = state.ready_requests()
    selected = []
    predicted_ms = 0.0

    for req in deadline_and_fairness_order(candidates):
        unit = next_feasible_unit(req, state)
        if not reserve_kv(unit, state.kv_pool):
            continue
        if predicted_ms + estimate_ms(unit, selected) > target_ms:
            release_reservation(unit)
            continue
        selected.append(unit)
        predicted_ms = estimate_ms(selected)

    return group_by_engine_shape(selected)
```

The hard work lives in cost estimation, fairness ordering, memory reservation, graph compatibility, phase mixing, and rollback when a selected distributed peer becomes unavailable.

Overload and headroom

Queueing latency grows nonlinearly near saturation, and variable output lengths make service time heavy-tailed. Operate with headroom sized for bursts, node loss, and estimation error. The right utilization target is the highest level that preserves goodput and recovery reserve, not a universal percentage.

During overload, apply a declared hierarchy: protect in-flight interactive sequences, stop low-value prefills, enforce tenant quotas, shed batch traffic, reduce optional speculative or best-of work, and reject new requests before memory is exhausted. Quality-changing degradation requires product authorization and telemetry.

MAKE THE OBJECTIVE EXECUTABLE

"Maximize useful tokens subject to deadline, fairness, memory, and semantic constraints" can be translated into ordering, budgets, and invariants. "Keep the GPU busy" cannot decide whom to delay or reject.

Principal Interview Review

- 1 Design an iteration-level scheduler for mixed interactive and batch traffic.
- 2 How do you prevent long prefills from breaking token cadence without starving them?
- 3 What should admission control do when KV memory is nearly full?
- 4 How would you measure and enforce fairness across tenants with different request shapes?
- 5 Explain cancellation and slow-client handling end to end.
- 6 Why can intentionally lower accelerator utilization improve fleet goodput?

Answer Key

1. Mixed scheduler

Maintain separate objective classes but one global capacity and memory governor. At each boundary, reserve imminent interactive decodes by deadline, add aged interactive prefills in bounded chunks, and spend declared slack on batch work using weighted service accounting. Predict iteration time and KV growth, group compatible shapes, and reserve before launch. Enforce tenant quotas, propagate cancellation, and expose quick rejection during overload. Evaluate goodput and starvation, not only average throughput.

2. Long prefills

Split them into cost-bounded chunks. Reserve a decode time budget that keeps active requests below the maximum token gap, then schedule chunks in remaining capacity. Increase a waiting prompt's virtual priority with age and guarantee a minimum service share so continuous decode load cannot starve it. Adapt chunk size to measured phase cost and pipeline balance; report the added first-token latency paid by the long request.

3. Near-full KV

Stop admitting requests whose reserved growth would consume the safety margin. Reclaim cancelled and expired state, evict low-value reusable prefixes or inactive sessions, route to a replica with capacity, and shed lower-priority queued work according to policy. Protect memory promised to active requests. Do not rely on an eventual allocator failure; return a bounded retry or overload response before the pool is exhausted.

4. Fairness

Charge tenants in predicted or measured compute-time/KV-time units rather than raw request count. Use weighted deficit or virtual-time scheduling, cap burst debt, and age waiting work. Track service share, queue delay, SLO attainment, rejections, and starvation by tenant and shape class. Account for shared-prefix savings consistently so a cache hit does not accidentally grant unlimited priority or penalize the tenant twice.

5. Cancellation and backpressure

The gateway propagates cancellation with request identity; queues remove unstarted work; the scheduler marks in-flight work and excludes it at the next safe boundary; the worker stops sampling and streaming; and the allocator releases private and shared references exactly once. Stream buffers are bounded. A slow client is paused, deprioritized, or cancelled by policy, with its KV occupancy still charged. Races are tested around token commit and speculative rollback.

6. Lower utilization

Near saturation, queue delay and tail risk rise sharply, bursts cannot be absorbed, and a node failure can overload the remainder of the fleet. Huge batches may maximize hardware throughput while violating first-token and cadence SLOs. Leaving headroom can admit urgent work sooner, reduce generation stalls, preserve failure reserve, and therefore increase the number of requests completed within contract even though average device utilization is lower.

CHAPTER 12

12

Sampling, Structured Output, and Speculative Decoding

Sampling, Structured Output, and Speculative Decoding

Decoding is part of the model's externally visible semantics. Acceleration may reorganize computation, but it must preserve the declared probability distribution, constraints, random-number mapping, and termination behavior unless approximation is explicitly allowed.

Scheduling decided when a request runs; this section governs what its tokens mean and how to produce them faster without changing them. The running service's 4.7 ms weight-streaming floor is the number to beat: any acceleration that commits more than one token per floor-priced step is attacking the sequential bottleneck itself.

The token-selection pipeline

A typical step transforms model logits through an ordered pipeline:

- 1 apply vocabulary restrictions and bad-token masks;
- 2 apply repetition, frequency, presence, or custom penalties;
- 3 apply temperature or other score transformation;
- 4 truncate with top-k, nucleus top-p, minimum probability, or a product policy;
- 5 normalize and sample with a defined random stream;
- 6 test stop tokens, stop strings, grammar acceptance, and length limits;
- 7 commit token and KV state, detokenize incrementally, and stream.

Order matters because these operations generally do not commute. A fused implementation must match the reference order, tie rules, NaN behavior, and dtype policy. Tokenization creates

additional edge cases: a stop string may span tokens, and a token byte sequence may be incomplete until later output arrives.

Greedy decoding is the special case that selects an argmax under deterministic tie policy. Sampling correctness means equality of the declared distribution, not equality of one token trace unless seeds, batching, and random-number mapping are also fixed.

Randomness and reproducibility

Counter-based random generators can derive values from a key such as request seed, logical output position, branch, and sampling substep. This makes randomness independent of physical thread scheduling and easier to preserve across dynamic batches.

Exact reproducibility across engine versions may still be impossible if floating reductions, truncation boundaries, or probability transforms change. Define a level:

- distributional equivalence;
- same result for fixed seed on one engine version;
- same result across batch compositions;
- bitwise identity across versions and hardware.

Do not promise the strongest level unless the whole logits pipeline, numeric path, and RNG mapping are controlled and regression-tested.

Structured and constrained decoding

Structured output restricts the next-token set according to a grammar, schema, regular expression, or application state. A common engine compiles the constraint into a finite-state representation and maintains a state per sequence. At each step it finds allowed tokens and masks the rest before sampling.

The hard parts are often outside the model:

- tokenizer tokens may contain multiple characters or partial UTF-8 sequences;
- a grammar state can have a large or tiny allowed set;
- dynamic masks can break graph or batch compatibility;
- state transitions and detokenization must agree;
- fallback or repair changes semantics and latency.

Precompute token-to-transition tables where possible, cache grammar artifacts by versioned identity, and batch sequences with compatible constraint work without starving rare grammars. Measure compile latency separately from per-token mask cost.

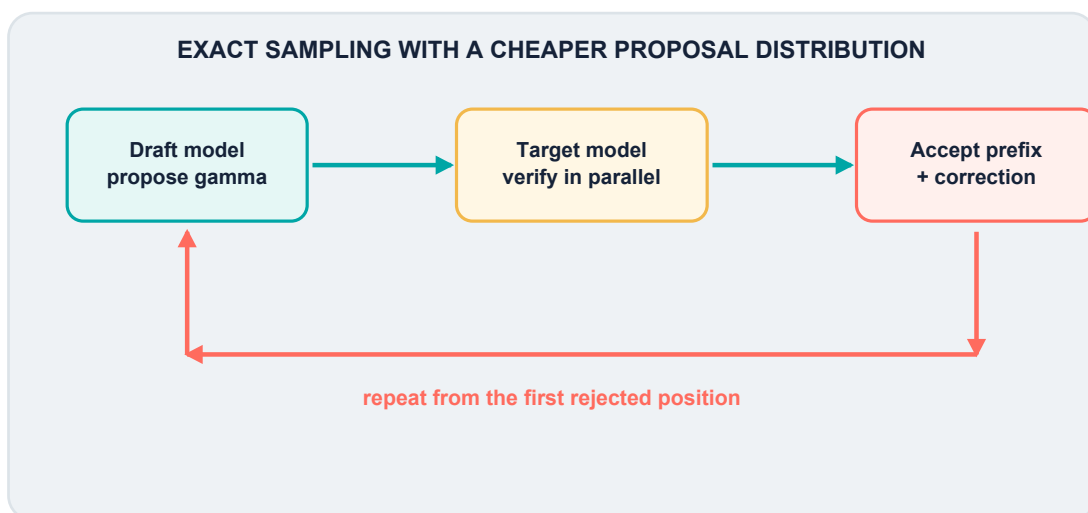
Beam search, best-of, and branch state

Beam search maintains multiple scored hypotheses and selects expansions under a length-normalized objective. Best-of samples several candidates and returns a selected one. Both multiply KV state and scheduling work. Paged copy-on-write can share the common prefix, but diverged branches need private blocks.

Define whether non-returned candidates count against token quotas and billing, how stop rules apply, and whether streaming is possible before the winner is known. Prune and release branches promptly. A scheduler that counts one API request rather than its active branches can under-reserve memory badly.

Speculative decoding

Speculative decoding uses a cheaper proposal process to reduce sequential target-model steps. A draft proposes gamma tokens. The target evaluates the proposed positions in parallel and commits an accepted prefix plus, in common formulations, one token selected from target information at the stopping point.



A proposal is useful only when the target can verify several positions more cheaply than performing the same number of ordinary sequential decode steps.

Proposal sources include a smaller standalone model, a quantized or pruned target, early-exit layers, multiple-token prediction heads, retrieval or n-gram matches, or a tree of candidates. They differ in draft cost, acceptance, memory, training requirements, and how well they batch.

Exact stochastic verification

Let target probability be $p(x)$ and proposal probability be $q(x)$ for a proposed token x . Accept the proposed token with probability:

$$a(x) = \min(1, p(x) / q(x))$$

If rejected, sample from the normalized positive residual:

$r(x)$ proportional to $\max(0, p(x) - q(x))$

This decomposition preserves the target distribution when probabilities are computed under matching histories and the correction is implemented exactly. Guard zero proposal probability, normalize stably, and apply identical target logits processors before constructing p . Vocabulary sharding may require distributed normalization or candidate exchange.

PYTHON

```
def verify_token(proposed, p, q, uniform):
    qx = max(q[proposed], 1e-30)
    accept = min(1.0, p[proposed] / qx)
    if uniform <= accept:
        return proposed, True
    residual = maximum(p - q, 0.0)
    residual = residual / residual.sum()
    return sample(residual), False
```

Top-k, top-p, temperature, penalties, and constraints define the target distribution. Applying them differently in draft and verification can be legal if the exact acceptance/correction construction uses the resulting p and q , but it changes acceptance and implementation complexity. An approximation that skips correction must be declared and evaluated as a quality change.

Expected progress and break-even

If A is the number of accepted draft tokens, the tail-sum identity gives:

$$E[A] = \sum_{k=1}^{\gamma} \Pr(A \geq k)$$

The conditional acceptance probability changes with position and context, so estimate this from traces rather than assuming independent identical trials. Useful committed tokens per cycle may include an additional target token when all or part of the proposal is accepted, depending on algorithm details.

A simple break-even comparison is:

$$\text{cycle_cost} / \text{expected_committed_tokens} < \text{ordinary_target_step_cost}$$

Cycle cost includes draft, target verification, correction sampling, branch KV work, scheduler gaps, and any lost target batch capacity. Verification latency is shape-dependent and does not usually grow linearly with γ , which is why speculation can win.

For the running service at small batch, an ordinary decode step is bounded near 4.7 ms by weight streaming. A 1B-class draft with about 2 GB of weights has a streaming floor near 0.7 ms per draft step, and target verification of several proposed positions still costs roughly one target-weight pass. Four draft steps plus one target verification therefore have an idealized floor near $4 * 0.7 + 4.7 = 7.5$ ms. If the cycle commits three output tokens on average, ordinary decoding would spend about $3 * 4.7 = 14.1$ ms on the same output work, so the idealized speedup is about 1.9x. Draft arithmetic, correction sampling, scheduler gaps, and extra KV reduce that gain; poor acceptance or lost target batch capacity can erase it.

Choosing proposal length

Long proposals amortize a target call if acceptance remains high, but waste draft and verification work after early rejection and consume more temporary KV. Choose gamma from:

- recent acceptance and draft confidence;
- target/draft cycle-time curves by batch and context;
- queue pressure and target batch opportunity cost;
- memory reserve and graph variants;
- domain, temperature, grammar, and request class.

Use a bounded policy or lookup table before a complex predictor. Optimize measured goodput and tail latency, not acceptance alone. The best length may be zero when the target is already efficiently batched.

Tree and multi-token proposals

A tree proposes alternatives at several future positions so target verification can recover useful progress even when the top draft path is wrong. Multiple-token prediction heads propose future tokens from a shared target representation. These approaches can reduce separate draft cost but require branch packing, attention masks, verification logic, and often model-specific training.

The scheduler sees a variable number of verification tokens per request. Grouping incompatible trees can waste padding, while separating every shape fragments batches and graph caches. Restrict to a small family of supported tree shapes unless telemetry justifies more variants.

Interaction with batching and KV

Different sequences accept different prefix lengths, so they advance unevenly. Commit accepted KV, discard or recycle rejected branch state, update logical positions, and ensure random streams follow logical output rather than physical verification slots.

At high target batch, ordinary decode may already reuse weights well. Draft compute can contend with the target or reduce its resident batch because of extra weights and KV. At low-latency batch, eliminating target synchronization steps is often more valuable. Benchmark the joint scheduler with real arrival pressure.

ACCEPTANCE IS NOT THE OBJECTIVE

A proposal with excellent agreement can lose if it is expensive, reduces target batch capacity, creates awkward verification shapes, or worsens queueing. Report committed output tokens per end-to-end second under the same SLO and semantics.

Principal Interview Review

- 1 Why must the order of logits processors be part of the API contract?

- 2 Design deterministic RNG mapping for continuous batching and cancellation.
- 3 How does constrained decoding affect the serving system beyond token masking?
- 4 Derive speculative acceptance and the rejection correction at a high level.
- 5 How would you choose proposal length online and decide when to disable speculation?
- 6 Compare standalone drafts, multi-token heads, and tree proposals.

Answer Key

1. Processor order

Penalties, masks, temperature, and truncation generally do not commute, so reordering changes probabilities and possibly the chosen token. The contract must fix the pipeline, numeric and tie behavior, stop handling, and constraint application. A fused or distributed path is correct only if it matches that reference or the product explicitly accepts a new distribution.

2. RNG mapping

Use a counter-based generator keyed by request seed and logical output position, plus branch or subdraw indices for rejection and best-of. Do not key randomness to batch slot, CUDA thread, or scheduling iteration because regrouping would change results. Commit counters only with logical tokens, discard uncommitted branch draws by identity, and define whether retries reproduce or intentionally resample. Test across batch reshuffles, cancellation neighbors, and speculative accept/reject paths.

3. Constraints

Constraints require compilation and caching of grammar state, per-request state transitions, tokenizer-aware allowed-token computation, dynamic masks, and defined invalid-state behavior. They change CPU work, batch compatibility, graph shapes, sampling cost, and sometimes token-length distribution. The scheduler must account for compile latency and per-step cost, while semantic tests cover UTF-8 boundaries, stop sequences, schema completion, and cancellation.

4. Exact speculation

For a proposed token with target probability $p(x)$ and draft probability $q(x)$, accept with $\min(1, p(x)/q(x))$. Accepted mass is the overlap between the two distributions. If rejected, sampling from the normalized positive residual $\max(0, p - q)$ supplies exactly the target mass not covered by accepted proposals. Apply the construction conditionally at each matching history, handle zero and normalization stably, and use the exact target postprocessed distribution.

5. Proposal control

Estimate expected committed tokens and full cycle cost for a small supported set of proposal lengths by batch, context, domain, temperature, and recent acceptance. Select the length with best predicted SLO-constrained goodput subject to memory reserve. Disable when draft plus verification per committed token exceeds ordinary target decode, when target batches are already efficient, acceptance collapses, memory is tight, or unsupported sampling/grammar semantics would require approximation.

6. Proposal designs

A standalone draft is flexible and needs no target modification, but carries separate weights/KV and consumes scheduling capacity. Multi-token heads share target computation and can draft cheaply, but require architecture-specific training and may have correlated errors. Trees provide alternative future paths and recover from top-path errors, but increase verification tokens, branch state, masks, and shape complexity. Compare end-to-end cycle cost, acceptance/progress, memory, training and deployment burden, batching, and exactness.

CHAPTER 13

13

Parallel, Replicated, and Disaggregated Serving

Parallel, Replicated, and Disaggregated Serving

A serving fleet places model parameters, KV state, and request phases across failure domains. Parallelism is useful only when its memory or throughput gain exceeds communication, synchronization, and scheduling cost at the target shapes.

One replica is no longer enough: a model may not fit one device, traffic exceeds one accelerator, and phases interfere. This section places weights, KV, and phases across devices and pools. Two communication facts drive everything below. A **collective** is a synchronized exchange, such as an all-reduce, whose time is roughly a fixed latency per step plus bytes divided by link bandwidth, and it completes at the pace of the slowest participant. Decode sends small messages every few milliseconds, so the latency term and the slowest peer dominate exactly where training-style large-message efficiency does not apply.

Replicas and model-parallel groups

Data-parallel serving replicas hold equivalent model state and process different requests. They scale aggregate capacity and isolate failures but do not make one request faster unless the request can be divided at a higher level.

Tensor parallelism shards large layer operations across devices and introduces collectives in each layer. It reduces parameters per device and can lower single-request compute time, but small decode messages may underutilize links and every step waits for the slowest peer.

Pipeline parallelism assigns layer ranges to stages. It reduces per-device weights and can overlap microbatches, but latency includes stage traversal and bubbles depend on the number and uniformity of microbatches. Dynamic continuous batches complicate stable pipeline schedules.

Expert parallelism places mixture-of-experts experts across devices. It reduces resident expert weights per device and increases aggregate capacity, but token routing creates all-to-all traffic and load imbalance. Hot experts and small per-expert token counts can dominate decode.

Part V develops these mechanisms in depth. For serving, the decision is driven by request latency, active batch, KV placement, link topology, and the model's fit, not training precedent.

Choosing the smallest group

Use the smallest model-parallel group that fits weights, KV reserve, runtime workspaces, and failure headroom while meeting latency. Increasing tensor-parallel degree can reduce local compute but adds communication and consumes more devices per replica, reducing the number of independent batches the fleet can run.

A useful comparison includes:

- per-device weight and KV bytes;
- prefill and decode latency curves by batch and context;
- collective time and exposed synchronization;
- replica count and queueing under the arrival process;
- fault blast radius and repair time;
- cost per good request or output token.

The fastest isolated request configuration may have worse fleet goodput because it creates fewer replicas and longer queues.

The running service's 14 GB model fits comfortably on one 80 GB device, so tensor parallelism is a choice, not a necessity. Splitting across two devices moves the weight-streaming floor from 4.7 ms toward 2.4 ms but inserts collectives into all 32 layers of every step and consumes two devices per replica. Unless the fleet is latency-starved at small batch, two independent replicas usually deliver more goodput than one two-way group.

Topology-aware placement

Map communication-heavy peers onto the fastest available links and keep the mapping stable enough for engine artifacts and cache locality. Know which links share bandwidth, which paths cross sockets or switches, and whether collectives contend with KV transfer or storage traffic.

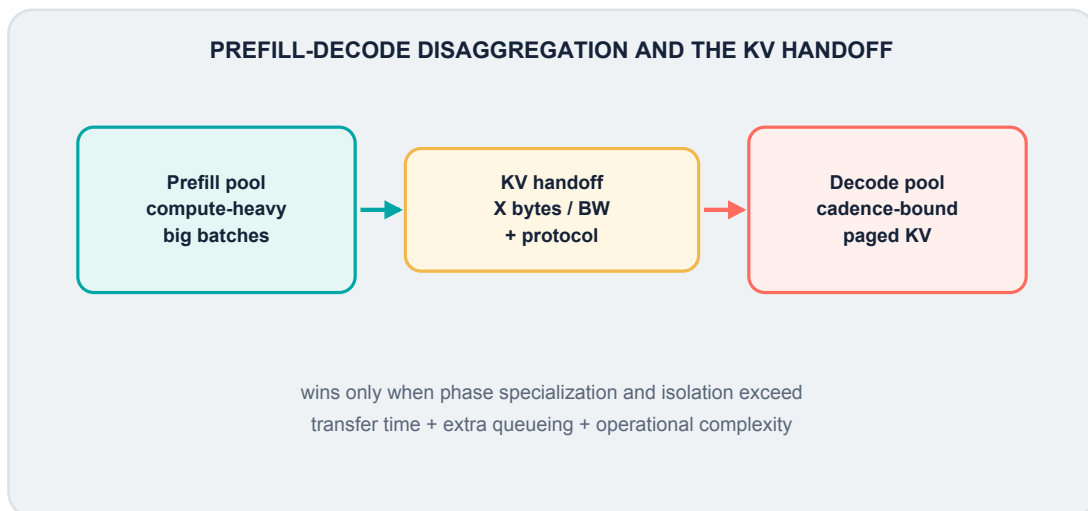
Topology faults are performance faults before they become hard errors. A degraded link or throttled peer slows every synchronized device. Record per-rank timing and collective skew; aggregate GPU utilization can hide one straggler holding the group.

Heterogeneous accelerators require separate performance models and engine variants. Route shapes to the hardware that meets their phase objective, but include transfer, queue, and fragmentation costs. A nominally faster device can lose if its pool is saturated or lacks the required adapter/precision path.

Prefill-decode disaggregation

Colocated workers run both phases and retain KV locally. This is simple and avoids a phase-boundary transfer, but long compute-heavy prefills can interfere with cadence-sensitive decode and both phases must share one parallelism and capacity plan.

Disaggregation assigns prefill and decode to different worker pools. It allows phase-specific hardware, batch, parallelism, and scaling, and it isolates interference. The handoff is KV state plus request metadata.



Prefill and decode pools specialize independently; the price is a KV handoff whose bytes, protocol, and queueing must be modeled before committing to the architecture.

For transferred KV bytes X and sustainable path bandwidth BW , a lower bound is:

$$t_{\text{handoff}} \geq X / BW + \text{fixed transfer overhead}$$

The actual path includes source readiness, registration, serialization or layout conversion, network queueing, destination placement, and synchronization. Long prompts create more state to transfer but also more prefill work that specialization may save.

For the running service, a 2,000-token prompt hands off about 260 MB of KV ($2000 * 131 \text{ kB}$). Over the assumed 50 GB/s path that is roughly 5 ms plus protocol overhead - cheap next to the prefill it frees the decode pool from repeating. The same transfer over a contended or slower path can erase the benefit, which is why the bound is a starting point and not a verdict.

DISAGGREGATE ONLY AFTER MODELING THE HANDOFF

The architecture wins when phase specialization, independent scaling, and interference isolation exceed KV transfer, additional queueing, network contention, and operational complexity at the real prompt/context distribution.

Handoff protocol

A correct handoff identifies model/adaptor version, token range, position state, KV dtype and layout, source blocks, destination allocation, and ownership. One safe sequence is:

- 1 destination admission reserves capacity;
- 2 source finishes and marks transferable committed blocks;
- 3 data moves with checksums or transport integrity;
- 4 destination confirms visibility and compatible metadata;
- 5 ownership transfers or source references are released;
- 6 decode becomes schedulable.

Retries need idempotent request and transfer identifiers. If the destination fails after receiving some blocks, cleanup must not free source state prematurely or leak destination allocations. Avoid a protocol that can generate the first token twice after a retry.

Routing with state affinity

Routing decisions occur at several levels:

- gateway to model/version pool;
- request to replica or prefill group;
- prefill result to decode group;
- resumed session to existing or migrated KV;
- expert or adapter work within a group.

Least-loaded routing is insufficient when state has locality. Estimate completion time including queue, cache hit, adapter load, transfer, and phase cost. Use bounded stickiness: prefer the state owner while its delay advantage remains positive, then migrate or recompute rather than creating an unbounded hot spot.

Independent scaling and backpressure

In a disaggregated system, prefill and decode capacity must balance in units that reflect the workload. Request rate alone is misleading; prefill load follows uncached input tokens and encoder work, while decode load follows generated tokens and retained contexts.

If prefill outruns decode, completed KV accumulates and consumes transfer buffers or destination memory. If decode outruns prefill, expensive decode devices idle. Add bounded inter-stage queues, destination reservations before prefill commitment, and autoscaling signals based on predicted phase work and SLO risk.

Fault domains and recovery

A model-parallel group generally fails as a unit when one peer disappears. Drain new admissions, fail or retry affected requests according to commit state, and replace the group. Replicas provide capacity redundancy; model-parallel ranks do not.

Replaying a prefill can reconstruct KV if the input and model version are available, but it increases first-token latency and may violate deletion or privacy policy. Replicating KV improves recovery but doubles state traffic and memory. Choose by SLO, prompt cost, and failure frequency.

For streaming responses, retries after emitted tokens are delicate. The service can resume only if it can reproduce exact state and avoid duplicate bytes, otherwise it should terminate with a clear partial-response error. Exactly-once token streaming across arbitrary failures is an application protocol, not an automatic property of the model engine.

Principal Interview Review

- 1 Why can increasing tensor-parallel degree reduce fleet goodput?
- 2 How would you choose between replication, tensor parallelism, and pipeline parallelism for serving?
- 3 Derive the break-even terms for prefill-decode disaggregation.
- 4 Design a correct and retry-safe KV handoff protocol.
- 5 How should routing balance queue delay against state locality?
- 6 What changes in autoscaling and failure handling after disaggregation?

Answer Key

1. Tensor-parallel degree

More shards reduce weights and compute per device but add per-layer collective latency and synchronize on the slowest peer. Small decode messages may use links poorly. A larger group consumes more GPUs per replica, so the fleet runs fewer independent batches and may queue longer. Compare SLO-constrained request goodput for the whole fleet, including collective skew and failure blast radius, rather than single-request kernel speed.

2. Parallelism choice

Start with the smallest group that fits weights, KV reserve, runtime workspace, and safety margin. Prefer replicas for aggregate independent traffic. Add tensor parallelism when the model does not fit or its reduced local compute offsets collectives at target batch. Consider pipeline parallelism when layer partitioning fits the topology and enough compatible microbatches exist to control bubbles. Model queueing, replica count, context, prefill/decode separately, and topology rather than copying the training layout.

3. Disaggregation break-even

Benefits are phase-specific batching and hardware, independent scaling, better parallelism choices, and removal of prefill-decode interference. Costs are KV transfer bytes over sustainable bandwidth, fixed handoff and synchronization, extra queueing, destination reservation, network contention, layout conversion, and operational failure modes. Evaluate goodput under both first-token and token-cadence SLOs across the joint prompt/output distribution. The design wins only when the benefit exceeds all handoff terms at fleet scale.

4. KV handoff

Give the transfer an idempotent identity and reserve compatible destination blocks first. The source transfers only committed state with model, adapter, token range, position, dtype, and layout metadata. Verify integrity and visibility before making decode schedulable, then atomically transfer ownership or release source references. Retries discover completed chunks, and failures clean up destination partials without destroying the source's recoverable copy. Token commit and client streaming have separate idempotency rules.

5. Locality-aware routing

Predict time to useful progress on each eligible destination: queue plus missing prefill or transfer plus execution, adjusted for SLO, tenant, health, and memory pressure. Prefer a cache or adapter owner only while its saved work exceeds extra queue delay. Replicate very hot immutable state, use bounded stickiness for sessions, and retain a recompute fallback. Validate the predictor against actual completion time and prevent one popular prefix from hot-spotting a replica.

6. Scaling and failure

Scale prefill from uncached input/encoder work and first-token risk; scale decode from generated-token demand, context distribution, KV occupancy, and cadence risk. Balance the bounded handoff queue and reserve destination memory before producing more state. Failure recovery now spans two pools and a transfer protocol: retry idempotently, reconstruct or replicate KV by policy, avoid duplicate streamed tokens, and keep capacity reserve for loss of a group or network path.

CHAPTER 14

14

Quantization, Compression, and Adapter Serving

Quantization, Compression, and Adapter Serving

Compression changes capacity, bandwidth, arithmetic, layouts, calibration, and quality risk at once. Adapter serving adds another layer of dynamic state and batch compatibility. Neither is merely a smaller checkpoint.

The performance model made decode a bytes problem, and the fleet section made memory a replica-count problem. Compression attacks both at the source. The running service quantifies the stakes: 14 GB of weights set the 4.7 ms step floor, and KV at 131 kB per token caps concurrency near 200 conversations per replica.

Precision as a systems choice

Quantization maps values to a lower-precision representation plus scale and possibly zero-point metadata. The useful question is not "How many bits?" but:

- which tensors are quantized;
- which operations consume the format directly;
- the scale granularity and metadata layout;
- accumulation and output precision;
- calibration or optimization data;
- hardware and kernel support;
- quality and performance at production shapes.

Common families include:

Family	Main bandwidth/capacity effect	Common challenge
Weight-only	Smaller model reads and footprint	Unpack/dequant overhead, small-row kernels
Weight-activation	Smaller operands and low-precision matrix path	Activation outliers and dynamic scaling
Floating low precision	Wider dynamic range than integers at similar bits	Scale policy and hardware availability
KV quantization	More context or batch, fewer KV bytes	Attention sensitivity and write/read conversion
Mixed precision	Protect selected layers or channels	More variants, conversions, and analysis

The format name alone does not determine speed. Group size, packing order, scale placement, padding, and fused epilogue support can decide whether the kernel reaches the intended hardware path.

Weight-only quantization

In small-batch decode, weight reads often dominate dense linear layers. Weight-only quantization stores packed low-bit weights, loads scales, reconstructs or directly consumes low-precision values, and multiplies by higher-precision activations.

Smaller weights can:

- reduce step latency when bandwidth-bound;
- make the model fit on fewer devices;
- leave more memory for KV and batch;
- reduce model load and replica startup time.

The path can lose if unpack and scale work is not overlapped, tensor-core instructions do not support the exact format, group metadata is large, layout conversion occurs at runtime, or the compressed model shifts the bottleneck to KV, communication, or launch overhead.

Calibration-aware methods protect important weights or choose scales using representative activations. The serving team still owns format validation because a high-quality checkpoint can be paired with a poor runtime layout.

On the running service, a well-executed 4-bit weight path shrinks streamed bytes from about 14 GB toward 4 GB and the small-batch step floor from 4.7 ms toward 1.5 ms - potentially tripling single-stream token rate. The same checkpoint behind a poor unpack path can sit above the FP16 floor. The format's arithmetic is a promise; the kernel's achieved bandwidth is the delivery.

Weight-activation and floating formats

Quantizing activations can unlock lower-precision matrix instructions and reduce intermediate traffic. Activation distributions are input-dependent and often contain outliers. Strategies include per-token or per-row dynamic scales, offline transformations that move quantization difficulty between activations and weights, selective higher precision, and block-scaled floating formats.

Dynamic scaling adds reductions, metadata, and synchronization. Offline scaling depends on calibration coverage. Per-tensor scales are cheap but sensitive to outliers; per-channel, row, or block scales track range better but increase metadata and may complicate kernels.

Accumulate sensitive reductions in an appropriate wider type. Protect normalization, logits, embeddings, or selected layers when evidence shows they dominate error. "Everything at one dtype" is an implementation convenience, not a quality principle.

KV-cache quantization

KV state grows with live cached tokens, so lower precision can increase concurrency or context and reduce attention bandwidth. Keys affect attention scores; values affect the weighted sum, and their sensitivities may differ by layer, head, token position, and model.

Scale choices include per-tensor, per-head, per-channel, per-token, or grouped blocks. Finer scales improve range tracking but consume metadata and append-time work. Dynamic token scales may require a reduction when KV is written. The attention kernel must dequantize without destroying locality or occupancy.

Evaluate long-context retrieval, position-sensitive tasks, multi-turn conversations, rare tokens, tool-use formats, and output distributions, not only short perplexity. A format that doubles theoretical capacity but slows attention can reduce serving capacity under latency SLOs.

For the running service, FP8 KV doubles capacity toward 900,000 cached tokens - about 400 median conversations per replica - and a 4-bit format doubles it again. Each halving also halves long-context attention traffic, which matters once the batch's cached tokens approach the 107,000-token crossover where KV reads rival weight reads.

Compression shifts the optimum

Suppose uncompressed weights require a four-device tensor-parallel group. A weight format that fits the model plus reserve on two devices may create twice as many replicas, reduce collective degree, and change the best batch. This fleet-level gain can exceed its isolated kernel speedup.

The reverse can occur: a quantized format frees memory, the scheduler admits a larger batch, and longer iterations violate token-cadence SLOs. Re-tune group size, batch, KV budget, graph shapes, and autoscaling after changing precision.

Quantization validation ladder

Use a staged evaluation:

- 1 tensor and layer numerical comparisons;

- 2 teacher-forced token probability or perplexity by domain;
- 3 generation quality, task, safety, and structured-output suites;
- 4 long-context, multilingual, code, rare-slice, and adversarial tests;
- 5 kernel benchmarks across batch, context, and alignment shapes;
- 6 end-to-end replay with scheduling, KV pressure, and target hardware;
- 7 canary with slice metrics, rollback, and version isolation.

Measure both mean quality and regressions concentrated in specific layers, languages, adapters, or long contexts. Use the same sampling seeds or distributional test appropriate to the reproducibility contract.

A SMALLER MODEL CAN BE A SLOWER SERVICE

Checkpoint bytes do not reveal unpack cost, scale traffic, unsupported instructions, layout conversion, collective changes, or the batch and group size the scheduler will choose. Benchmark the deployed artifact and re-plan the fleet.

Multi-adapter serving

Low-rank adapters modify selected linear layers while sharing a base model. Serving many adapters on separate full replicas wastes base weights. A multi-adapter engine keeps the base resident, loads adapter matrices dynamically, and groups adapter work across requests.

For a linear layer with base output $\times W$, a low-rank update has the form:

$\text{output} = x W + \text{scale times } x A B$

Requests in one batch may use different A and B. The engine can execute grouped low-rank operations, segment tokens by adapter, or fuse adapter work into a specialized path. Tiny ranks make launch and indexing overhead important; large or multiple adapters can create meaningful compute and memory pressure.

Adapter batching should not require every sequence to use the same adapter, but completely unstructured mixtures can reduce locality. Track adapter ID and rank as scheduling features and limit variant explosion.

Adapter lifecycle and isolation

An adapter has identity, base-model compatibility, rank and target modules, dtype, tokenizer/template expectations, authorization, and version. Loading should verify a manifest and content digest before publication. Keep reference counts while requests execute; evict only when no in-flight use remains.

An adapter cache trades load latency against device memory and batch capacity. Eviction value depends on request frequency, load cost, bytes, and whether the base worker can serve useful traffic while loading. Preload latency-critical adapters, but cap tenants so one customer cannot fill every worker.

Adapter and prefix identities interact: KV produced with one adapter cannot generally be reused under another. A rollout of an adapter version requires cache namespace separation just like a base-model rollout.

Capacity accounting with adapters

A worker memory budget includes:

base weights + resident adapters + active KV + reusable KV + workspaces + graph buffers + allocator reserve

Adapter count alone is misleading because ranks and targeted layers differ. Account bytes and measured per-token work. For sparse popularity, maintain a warm subset and route by adapter locality. For uniformly hot adapters, replicate them. For a long tail, consider host-resident staging, but include transfer time and queue hot spots.

Principal Interview Review

- 1 Compare weight-only, weight-activation, and KV quantization as serving interventions.
- 2 Why can a four-bit checkpoint be smaller but slower than a higher-precision one?
- 3 Design a quality and performance evaluation for a new quantization recipe.
- 4 How can quantization change the optimal parallelism and batching plan?
- 5 Design multi-adapter batching and residency management.
- 6 What isolation and versioning rules connect adapters, tokenizers, and prefix caches?

Answer Key

1. Quantization targets

Weight-only primarily reduces model footprint and decode weight bandwidth while leaving activation compute higher precision; it pays unpack and scale cost. Weight-activation can use lower-precision matrix paths and reduce operand traffic, but activation outliers, dynamic scales, and accumulation matter. KV quantization targets context capacity and attention bandwidth, adding append/read conversion and long-context quality risk. Choose the tensor whose bytes control the target phase and require a supported kernel and quality contract.

2. Smaller but slower

The runtime may unpack into a wider type, read many group scales, execute scalar conversions, use an unsupported or poorly tiled matrix path, or convert layouts at load or every step. The shape may be compute-bound already, or compression may expose KV, collective, sampling, or launch overhead. Measure achieved bandwidth/compute, instruction path, scale and conversion traffic, kernel time by layer and shape, group size, batch, and end-to-end gputput.

3. Evaluation design

Validate layers numerically, then probability/perplexity by representative domain, task and safety quality, generation behavior, long-context retrieval, code/multilingual/rare slices, and adapters. Benchmark exact deployed kernels across batch, context, alignment, and hardware. Replay production arrivals with KV pressure and measure first-token/cadence SLOs, memory, energy if relevant, and goodput. Canary by quantization version with rollback and cache isolation; do not infer production quality from one aggregate benchmark.

4. Re-planning after quantization

Smaller weights may let the model use fewer tensor-parallel devices, increasing replicas and reducing collectives. Freed memory permits more KV or adapters and a larger batch. Faster matrix work can make attention, communication, or host overhead dominant. Re-sweep group size, replica count, batch and iteration budget, KV allocation, graph variants, and autoscaling on the new artifact; the old optimum is no longer evidence.

5. Multi-adapter serving

Share one base model, tag tokens or sequences by adapter, and execute low-rank updates through grouped or segmented operations that support mixed adapters without padding every request to one rank. Cache adapters by expected saved load time per byte, preload latency-critical ones, route with bounded locality, and keep per-tenant quotas. Version manifests, validate base compatibility, reference-count in-flight use, and expose fallback or rejection when an adapter cannot load within SLO.

6. Isolation and versioning

Adapter identity includes content digest, base-model version, target modules, rank, scale, dtype, and any tokenizer/template requirements. Authenticate access before routing or loading. Prefix-cache keys include the exact adapter and tokenizer/template identity, so state never crosses incompatible versions or tenants. Rollout publishes a new namespace, drains references to the old one, and deletes adapter and cache artifacts under the same retention policy.

CHAPTER 15

15

Production Architecture, Capacity, and Reliability

Production Architecture, Capacity, and Reliability

A fast engine becomes a dependable service only when artifacts, control planes, observability, overload, failure recovery, security, and rollout are designed around its stateful streaming behavior.

Everything so far tuned one replica or one mechanism. Production is where the running service must hold its two SLOs through bursts, failures, and rollouts, with a couple hundred conversations of KV state in flight on every replica. This final section closes the loop the part opened: the same boundaries the lifecycle section timestamped are now the boundaries that ownership, capacity, and recovery are organized around.

Data plane and control plane

The **data plane** handles live requests: gateway, tokenizer, router, scheduler, model workers, KV transfer, sampling, and stream transport. The **control plane** manages model and adapter artifacts, placement, configuration, health, rollout, autoscaling, quotas, and policy.

Keep the hot path independent of a synchronous control-plane lookup. Workers consume versioned snapshots and continue safely during temporary control-plane failure. Conversely, the control plane needs enough data-plane telemetry to stop routing to unhealthy or incompatible workers.

Define ownership at boundaries. The gateway owns authentication and client semantics; the router owns eligible placement; the scheduler owns local admission and execution order; the worker owns model state and token commit; the allocator owns KV lifetime. Ambiguous ownership produces double admission, leaked pages, or duplicate streaming during retries.

Artifact build and compatibility

A deployable model is a bundle, not one weight file. It may include:

- model and tokenizer with immutable digests;
- chat templates and special-token configuration;
- quantization scales and packing layout;
- compiled engines or kernels by hardware capability and shape;
- parallelism and placement metadata;
- adapter compatibility schema;
- sampling and grammar versions;
- quality, performance, and security attestations.

Build artifacts reproducibly, sign or verify them, and publish atomically. A worker should reject a partial or incompatible bundle before serving traffic. Engine caches and CUDA graphs are derived artifacts and must be invalidated when their relevant inputs change.

Startup, warmup, and readiness

Startup includes artifact fetch, deserialization, memory allocation, weight placement, communicator creation, compilation or autotuning, graph capture, cache initialization, and warmup. Readiness should require representative model execution, not only a listening socket.

Warm the important precision, batch, context, adapter, and graph variants without trying to enumerate an unbounded shape space. Keep a fallback path for a cold shape. Record cold-start stages so autoscaling knows whether a new replica will arrive in seconds or minutes.

A rolling restart temporarily removes capacity and cache locality. Drain admissions, let bounded in-flight work finish, migrate or recompute state by policy, and maintain enough reserve that the remaining fleet does not enter overload.

Capacity planning

Start from the workload and SLO, not peak hardware FLOPs. For each request class estimate:

- uncached input and encoder work;
- output tokens and decode context trajectory;
- KV bytes and lifetime;
- prefix and adapter locality;
- phase service curves by batch and topology;
- arrival rate, bursts, and target percentiles.

At a stable high level, Little's Law relates average in-system concurrency N , arrival rate λ , and average time in system T :

$$N = \lambda T$$

It is an accounting identity, not a tail-latency guarantee. Use it to cross-check concurrency and memory, then use replay or a queueing model for variability and percentiles.

For the running service, a median request spends roughly 20 to 30 seconds in the system: 300 output tokens at a batch-shared decode cadence plus queueing. At $T = 25$ s and about 200 concurrent conversations of KV capacity, one replica sustains $\lambda = N / T = 8$ requests per second - before failure headroom, bursts, and long-context outliers, which is exactly why the reserve terms in this section exist.

For a worker, enforce:

weights + adapters + active KV + reusable KV + workspaces + reserve $\leq$ usable memory

Use usable memory after runtime and fragmentation measurements, not device nameplate capacity. For the fleet, include at least one relevant failure or maintenance scenario and the startup delay of replacement capacity.

CAPACITY IS A MEMORY CLAIM AS MUCH AS A COMPUTE CLAIM

On the running replica, admission is bounded by roughly 200 conversations of KV before compute saturates. State autoscaling, drain, and failure reserve in cached tokens and replicas, not in GPU utilization percentages.

Autoscaling signals

GPU utilization is a lagging and ambiguous signal. Combine:

- queued predicted prefill and decode work;
- first-token and token-cadence SLO risk;
- active and reserved KV occupancy;
- arrival forecasts and tenant quotas;
- adapter/cache locality pressure;
- replica warmup time and failure reserve.

Scale early enough to cover startup. Scale down by draining workers and accounting for cache coldness that remaining replicas will experience. Hysteresis and minimum residency avoid oscillation when traffic hovers near a threshold.

In disaggregated fleets, scale stages separately but coordinate the handoff queue and destination memory. In model-parallel groups, add or remove complete compatible groups rather than individual ranks.

Observability

The minimum operational view includes:

- arrival, admission, rejection, cancellation, and completion by class;

- queue delay, first-token, inter-token, and end-to-end distributions;
- input/output tokens, goodput, and batch/iteration shape;
- active, reserved, reusable, fragmented, and offloaded KV bytes;
- prefix hit tokens and actual time saved;
- adapter load, hit, eviction, and mixed-batch behavior;
- speculative proposal, acceptance, committed progress, and cycle cost;
- per-phase device time, collective skew, transfer time, and host gaps;
- errors by stage, model version, worker group, and termination reason.

Control metric cardinality. Put request IDs and exact shapes in sampled traces, not unbounded metric labels. Histograms need buckets that resolve the SLO region; averages are insufficient.

Failure taxonomy

Separate failures by recoverability and commit point:

- **before admission:** reject or route elsewhere;
- **queued:** retry safely if deadline permits;
- **during prefill before output:** retry on a compatible replica, perhaps using cached prefix;
- **after streaming begins:** resume only with exact committed state and transport protocol, otherwise terminate clearly;
- **allocator or OOM pressure:** stop admissions, reclaim, and protect active reservations;
- **rank or link failure:** fail the whole synchronized group and replace it;
- **control-plane isolation:** continue from a safe snapshot for a bounded interval;
- **semantic corruption:** quarantine the artifact and roll back, not merely restart.

Test fault injection at token-commit, KV handoff, cancellation, adapter eviction, and rollout boundaries. Many leaks and duplicates require a rare race, not a steady benchmark.

Graceful degradation

Safe overload actions preserve model semantics: reject new work, queue within a bound, route, shed batch traffic, lower speculative effort, disable best-of, or evict reusable caches. Actions that change model, precision, context, safety filters, or sampling require explicit product policy and response metadata.

Protect already admitted requests according to the published contract, but do not let an unbounded generation monopolize the service. Enforce declared maximum output and context, tenant budgets, stream idle timeouts, and cancellation.

Rollout and rollback

Version model, tokenizer, templates, quantization, engine, sampling, and scheduler configuration independently but deploy them as tested compatibility sets. A rollout sequence can be:

- 1 offline semantic and performance validation;
- 2 shadow execution on representative traffic without user impact;
- 3 canary by tenant or request slice with separate cache namespace;
- 4 gradual traffic increase under automated guardrails;
- 5 drain old state only after rollback risk falls;
- 6 retain a tested rollback artifact and capacity path.

Compare SLOs, goodput, error slices, output distributions, safety, cache behavior, and cost. A scheduler change can alter batching and therefore RNG traces even when the model is unchanged; use the correct equivalence criterion.

Rollback is harder with state. New-version KV and adapters may be incompatible with old workers. Either keep version-sticky requests until completion, reconstruct state from tokens on rollback, or maintain an explicitly compatible format. Never reinterpret state across versions by assumption.

Security and abuse resistance

Enforce authentication, quotas, maximum context/output, media limits, grammar complexity limits, adapter authorization, and bounded streaming buffers before scarce accelerator work is committed. Tokenization and grammar compilation can themselves be denial-of-service targets.

Isolate tenant cache keys and adapter state. Scrub or overwrite memory according to the threat model before reallocation, avoid logging prompts or reversible low-entropy hashes, and propagate deletion across prefix caches, offload, traces, and backups.

Side channels include timing differences from cache hits, model or adapter residency, and co-tenant load. Reduce observable detail, namespace reuse, and offer dedicated capacity where stronger isolation is required.

Cost and energy

Useful cost metrics include accelerator-hours per good request, dollars per million accepted output tokens under an SLO, and energy per completed request. Include idle reserve, failed and rejected work, draft computation, cache transfer, and host/network resources. A higher-throughput configuration can cost more per good request if it misses latency or needs a larger failure reserve.

Optimize total service cost after semantic and reliability constraints. The cheapest kernel is irrelevant if its artifact takes too long to roll out safely or its shape specialization multiplies operational burden.

Production readiness checklist

Area	Required evidence
Semantics	Reference logits/sampling/stop/constraint tests and declared approximation
Performance	Open-loop replay, SLO-goodput curve, cold and warm behavior
Memory	Capacity model, allocator invariants, cancellation and OOM tests
Distribution	Topology plan, collective/transfer profiling, group-failure recovery
Operations	Metrics, traces, alerts, runbooks, load shedding, autoscaling
Rollout	Versioned artifacts, cache isolation, canary guardrails, tested rollback
Security	Auth, quotas, tenant isolation, deletion, abuse limits, audit trail

Principal Interview Review

- 1 Produce a capacity plan for a new LLM endpoint from workload traces and SLOs.
- 2 Why is GPU utilization insufficient for autoscaling?
- 3 Design readiness, draining, and rollout for stateful model workers.
- 4 How should the service respond to failure after some tokens have streamed?
- 5 What observability distinguishes a scheduler regression from a kernel or network regression?
- 6 Design an overload and rollback policy that preserves explicit semantics.

Answer Key

1. Capacity plan

Segment traces by prompt, output, context, model, adapter, feature, locality, and burst. Benchmark phase service curves and memory per live token on candidate group layouts. Replay open-loop arrivals to find the highest rate meeting first-token, cadence, error, fairness, and quality SLOs with KV and failure reserve. Convert this to complete replica/group count, add startup and maintenance headroom, validate one-group loss, and report cost per good request with assumptions and sensitivity ranges.

2. Autoscaling signals

Utilization says a device is busy, not whether useful deadlines are met or how much queued work and state is arriving. It can be high during inefficient kernels or doomed overload, and low while memory is full of paused requests. Scale from predicted queued phase work, SLO risk, KV reservations, arrival forecast, locality, warmup delay, and failure reserve, using utilization as supporting evidence rather than the control objective.

3. Stateful worker lifecycle

Readiness requires verified artifact identity, memory/communicator setup, representative warm execution, and registration with compatible capacity. Draining stops new admissions, preserves or transfers committed state by policy, completes bounded in-flight work, and releases cache ownership before termination. Rollout uses separate version/cache namespaces, shadow and canary stages, automatic semantic/performance guardrails, enough old capacity for rollback, and explicit treatment of in-flight KV compatibility.

4. Failure after streaming

Track the last logically committed and acknowledged token. Resume only if a compatible worker can reconstruct exactly the model, sampling/RNG, constraint, and KV state and the transport can suppress duplicate output. Otherwise terminate with a clear partial-response error and stable request identifier; blindly restarting could duplicate or contradict visible text. Release state idempotently and record the commit point for diagnosis and billing.

5. Observability diagnosis

Scheduler regressions change queue delay, batch composition, phase mixing, iteration gaps, reservation failures, and fairness while per-shape kernels may remain stable. Kernel regressions raise device time or reduce achieved bandwidth/compute for the same shape. Network regressions raise collective or KV-transfer time, skew ranks, and correlate with links or paths. Join per-request spans with scheduler decisions, shape-tagged phase timing, per-rank collectives, host gaps, and version/config identity.

6. Overload and rollback

Bound queues and memory reservations, reject early with retry guidance, protect admitted interactive work, shed batch and optional best-of/speculative effort, and enforce tenant quotas. Any change to model, precision, context, sampling, or safety requires an authorized degradation mode and response metadata. Rollout isolates versions and caches, preserves a warm rollback path, drains incompatible in-flight state or reconstructs from tokens, and triggers on goodput, semantic, quality, error, and memory guardrails rather than utilization alone.

Further Study and Primary References

- [Orca: A Distributed Serving System for Transformer-Based Generative Models](#) - iteration-level scheduling and selective batching.
- [Efficient Memory Management for Large Language Model Serving with PagedAttention](#) - paged KV memory and serving throughput.

- [Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve](#) - chunked prefill and stall-free scheduling.
- [DistServe](#) - phase disaggregation and goodput-oriented placement.
- [Fast Inference from Transformers via Speculative Decoding](#) - exact speculative sampling.
- [SGLang: Efficient Execution of Structured Language Model Programs](#) - radix prefix reuse and structured execution.
- [SmoothQuant](#) and [AWQ](#) - activation-aware quantization methods.
- [Punica: Multi-Tenant LoRA Serving](#) - shared-base multi-adapter serving.
- [TensorRT-LLM Documentation](#) - maintained engine features, KV reuse, quantization, and deployment guidance.
- [MLPerf Inference Documentation](#) - benchmark scenarios and reproducibility requirements.

Final Inference Principle

The unit of optimization is a request completed within its semantic and SLO contract, not a kernel, token, batch, or GPU in isolation.

The serving design is complete only when performance models predict the important shapes, state transitions are safe under cancellation and failure, measurements include queueing and user-visible cadence, and rollout can reverse without corrupting live state. For the running service of this part, that means the 4.7 ms step floor, the 131 kB-per-token KV ledger, and the two latency SLOs from the opening table survive contact with schedulers, caches, fleets, and rollouts - defended end to end, not per component.

PART IV - CUDA AND ACCELERATOR PROGRAMMING

Part IV - CUDA and Accelerator Programming

CUDA optimization is the discipline of translating an algorithm into a schedule over threads, instructions, memory levels, and asynchronous work. The correct starting point is never a favorite tile size or instruction. It is a resource model: what must move, what must be computed, which dependencies are unavoidable, and which hardware resource becomes limiting first.

This part is self-contained. It does not require Part III open beside it. Wherever serving concepts appear - KV cache, paging, grouped-query attention, mixture-of-experts routing - they are restated here at the level a kernel engineer needs. Part III remains the place for fleet scheduling and product SLOs; Part IV owns the GPU schedule.

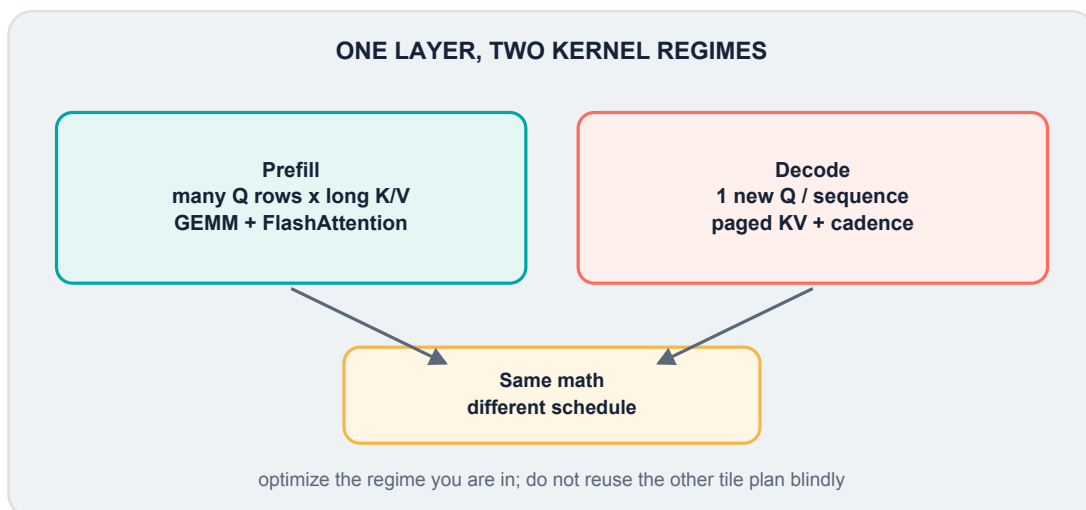
Running example used throughout

Every major section returns to one illustrative workload so estimates stay comparable. Treat the numbers as a teaching machine class, not a product claim.

Symbol	Meaning	Value in this part
D	model hidden width	4096
H	query heads	32
H_kv	key/value heads (GQA)	8
d	head dimension (D / H)	128
Prefill	one prompt	B = 1, S_q = 2048
Decode	concurrent sequences	B = 8, each with context S = 4096
Device	illustrative datacenter GPU	~3 TB/s HBM, high tensor-core peak

Two regimes matter:

- **Prefill / training-like attention:** many query rows attend over keys and values. Arithmetic intensity can be high when tiles reuse well.
- **Decode:** each sequence adds one new query token and rereads a long KV cache. Bytes per useful FLOP rise sharply; launch and memory latency matter as much as peak math.



The same attention equations appear in both regimes. Prefill wants large tiled GEMM reuse. Decode wants efficient paged KV traffic and low per-token overhead.

Prerequisites and reader contract

You need comfort with matrix multiplication, reductions, and reading short CUDA C++ kernels. You do not need prior mastery of a particular GPU generation. Architecture names change; the method does not: define ownership, storage scope, synchronization, bytes, FLOPs, and the measurement that would falsify the claim.

THE CUDA OPTIMIZATION LOOP

Establish a correct reference. Measure the real workload. Build a bytes-and-FLOPs model. Identify one limiting resource. Change the schedule. Recheck correctness. Reprofile. Stop when the remaining gap is below the value of additional complexity.

Map of this part

Section	Role in the story
Execution and memory	Build the resource ledger the rest of the part uses
Host orchestration	Show when kernel speed fails to reach the user
Hierarchical GEMM	Construct the reusable engine behind linear layers
Reductions, scans, histograms	Obtain the collective primitives softmax and routing need
Softmax, norm, top-k	Compose those primitives into transformer epilogues
FlashAttention	Fuse score, softmax, and value reduction without quadratic HBM
LLM inference kernels	Specialize the schedule for decode, paging, and quantization
Profiling and correctness	Close the loop on the running example with evidence

CHAPTER 16

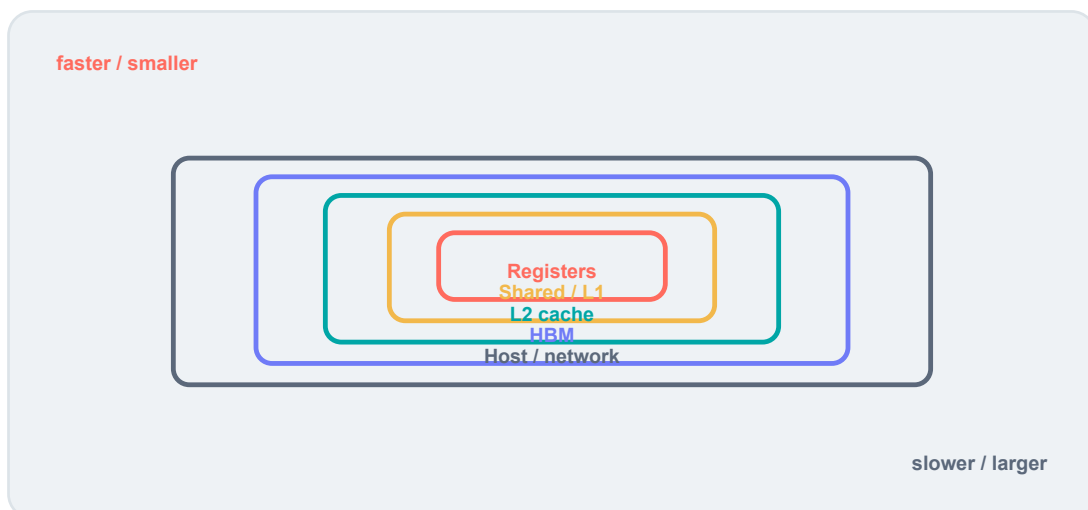
16

GPU Execution, Memory, and Resource Accounting

GPU Execution, Memory, and Resource Accounting

Threads execute in warps, warps are scheduled on streaming multiprocessors, and data moves through a hierarchy whose capacity grows as latency and sharing scope increase. Performance follows from how well the schedule uses those resources.

Before optimizing any kernel in the running example, you need a model of how the GPU executes work and where data can live. The rest of this part repeatedly asks: which level holds the tile, which group synchronizes, and which resource saturates first?



Registers and shared memory provide explicit locality; caches and HBM provide progressively larger scope. Each level has a capacity, bandwidth, allocation unit, and synchronization cost.

The execution hierarchy

A kernel launch creates a grid. The grid contains thread blocks. A block is assigned to one streaming multiprocessor, or SM, and normally remains there until it finishes. Threads in a block can cooperate through shared memory and block-scoped barriers. Threads from unrelated blocks cannot assume concurrent residency or execution order.

Threads are issued in warps, commonly 32 lanes. A warp scheduler selects an eligible warp whose next instruction has ready operands. The GPU hides latency by switching among ready warps and by exploiting independent instructions within each warp.

Newer architectures add optional execution levels such as thread-block clusters and distributed shared-memory access. These features can coordinate nearby blocks, but they do not invalidate the basic reasoning method: define the participating group, storage scope, synchronization contract, and resource cost.

SIMT, divergence, and predication

CUDA uses a single-instruction, multiple-thread execution model. Lanes in a warp may have different registers and addresses while issuing the same instruction. If lanes take different control paths, the warp executes the necessary paths under different masks.

Divergence cost depends on three facts:

- how many lanes are active on each path;
- how much work each path contains;
- whether the compiler converts a short branch into predicated instructions.

Branch divergence is not the same as memory divergence. A uniform branch can still issue scattered loads. A divergent branch can be inexpensive if only a few predicated instructions differ.

CUDA

```
// The boundary condition diverges only in the final block.
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) {
    y[i] = alpha * x[i] + y[i];
}
```

Do not remove a cheap bounds check by launching a separate cleanup kernel unless measurement shows the tail dominates. Extra launches and code variants can cost more than limited edge divergence.

Warp scheduling and latency hiding

A warp becomes ineligible when it waits for data, a dependency, a barrier, or an execution resource. Latency hiding requires other eligible warps or independent instructions. This produces two forms of parallelism:

- **thread-level parallelism:** many resident warps can run while another waits;

- **instruction-level parallelism:** one thread or warp has several independent operations in flight.

Apply this to the running example. A memory-bound decode gather over paged KV may need many resident warps because each warp has dependent long-latency loads. A prefill tensor-core mainloop can run well at lower occupancy if it has an effective asynchronous pipeline and enough independent matrix operations.

Memory spaces and their contracts

Registers are thread-private and compiler allocated. Register pressure can reduce resident blocks; excess live values spill to local memory, which is physically backed by device memory and cached.

Shared memory is block-scoped, explicitly indexed, and suited to tiles, cross-thread exchange, reductions, and staging. Its capacity is reserved per block and therefore affects residency.

L1 and L2 caches provide hardware-managed reuse. L2 is shared across SMs and often explains why a second benchmark pass appears faster. Cache state must be considered when comparing cold and warm behavior.

Global memory or HBM is device-wide. It offers high bandwidth but much higher latency than on-chip storage. Coalescing and reuse determine useful bandwidth.

Constant memory is read-only from device code and can efficiently broadcast when a warp reads the same address. Divergent constant addresses serialize through the cache path.

Pinned host memory supports higher-bandwidth and asynchronous host-device transfers, but it is a scarce host resource. **Unified memory** can simplify address management, but page migration and fault behavior must be understood for performance-critical paths.

Coalescing with a transaction example

For many modern CUDA devices, global accesses by a warp are serviced by the 32-byte sectors needed to cover the requested addresses. Consider 32 lanes loading adjacent float values from a 32-byte-aligned base:

```
address(lane) = base + 4 * lane
```

The warp requests 128 contiguous bytes, so four 32-byte sectors contain all values. Shift the base by one float and the same logical 128 bytes can span five sectors. Access one float every 32 floats and the warp may request one sector per lane while using only 4 bytes from each.

Useful bandwidth is:

```
requested_bytes / transferred_bytes * measured_bus_bandwidth
```

This ratio explains why a kernel can report high device-memory throughput while delivering little useful data.

In the running example, a well-laid-out KV vector of width $d = 128$ in FP16 is 256 bytes per token per head. A warp that loads those values as contiguous pairs can cover them with a small number

of sectors. The same logical read through a badly packed page table, with lanes jumping across pages, can multiply transferred sectors without increasing useful bytes.

EXPLAIN COALESCING WITH ADDRESSES

Say which address each lane requests, which aligned sectors cover those addresses, and how many bytes are useful. "Contiguous is good" is an observation; the transaction count is the explanation.

Alignment and vectorized access

Vector loads such as `float4` reduce instruction count when pointers are aligned and each thread owns adjacent data. They do not automatically fix a bad warp-level access pattern. A warp of threads performing aligned vector loads should still cover compact contiguous sectors.

Use scalar tail handling or a predicated final vector. Never reinterpret an unaligned pointer as a wider vector type solely to obtain fewer instructions. Validate compiler output because source-level vector syntax may be split when alignment is not provable.

Shared-memory banks

Shared memory is divided into banks. A common mapping for four-byte words is conceptually:

`bank = word_address mod bank_count`

If lanes access different addresses in the same bank, the access may be serialized into several transactions. If lanes read the same address, hardware can broadcast.

A transpose tile often adds one padding column:

CUDA

```
__shared__ float tile[32][33];

tile[threadIdx.y][threadIdx.x] = input[row * ld + col];
__syncthreads();

// Column access now advances by 33 words rather than 32.
float value = tile[threadIdx.x][threadIdx.y];
```

Without padding, a column stride equal to the bank count maps many lanes to one bank. Padding rotates successive rows through different banks.

Quantitative occupancy

Theoretical occupancy is active warps divided by the architectural warp limit per SM. Resident blocks are bounded by the minimum of several ceilings:

`blocks_threads = floor(max_threads_per_sm / threads_per_block)`

`blocks_registers = floor(registers_per_sm / registers_per_block)`

```
blocks_shared = floor(shared_bytes_per_sm / shared_bytes_per_block)
resident_blocks = min(blocks_threads, blocks_registers, blocks_shared,
max_blocks_per_sm)
```

Register allocation occurs in architecture-specific granularity, so a small increase in registers per thread can cross a step and remove an entire resident block. Dynamic shared memory has a similar cliff.

Worked example

Suppose an illustrative SM supports 2,048 threads, 64 resident warps, 65,536 registers, and 100 KiB of shared memory. A block has 256 threads, uses 64 registers per thread, and reserves 32 KiB shared memory.

- thread ceiling: $\text{floor}(2048 / 256) = 8$ blocks;
- register ceiling: $\text{floor}(65536 / (256 * 64)) = 4$ blocks;
- shared-memory ceiling: $\text{floor}(100 \text{ KiB} / 32 \text{ KiB}) = 3$ blocks.

The shared-memory limit allows 3 blocks, or 24 warps, which is 37.5 percent theoretical occupancy. That may be enough if the kernel pipelines memory well. Reducing shared memory to admit a fourth block helps only if the added warps hide a real stall and the redesign does not add traffic.

This cliff appears later when FlashAttention or decode kernels grow pipeline stages or accumulator tiles: more on-chip state can raise reuse and still lower achieved bandwidth if residency collapses.

Synchronization and memory ordering

`__syncthreads()` is a block barrier and establishes the shared-memory visibility needed by participating threads. It must be reached by all non-exited threads in the block along a convergent control path.

Warp shuffles exchange register values without shared memory. They require an accurate active mask. Passing a full mask when the final warp has inactive lanes can read undefined values or violate the primitive's participation contract.

Atomics make an update indivisible at a declared scope, but atomicity is not a general barrier. Memory fences order memory visibility but do not force other threads to arrive. Correct concurrent code states both the ordering requirement and the rendezvous requirement.

Resource-led optimization table

Symptom	Likely resource	Evidence	Candidate response
Many long-scoreboard stalls	Dependent memory latency	Scattered loads, low eligible warps	Coalesce, prefetch, increase independent work
High sectors per requested byte	Memory transactions	Poor global-load efficiency	Change layout or lane-to-data mapping
Low residency with spills	Registers	Local loads/stores, allocation report	Shorten live ranges, reduce tile, split kernel
Barrier-heavy timeline	Synchronization	Warp stalls at barriers, imbalance	Repartition work, warp-level exchange
High issue utilization, low tensor utilization	Instruction mix	Scalar work dominates	Reduce address/control overhead, use MMA path

OCCUPANCY IS NOT THE OBJECTIVE

Raising theoretical occupancy while destroying reuse, vector width, or tensor-core issue often slows the kernel. Use occupancy as a residency constraint, then optimize the limiting stall or bandwidth metric on the real shape.

Principal Interview Review

- 1 Explain coalescing using lane addresses and transactions.
- 2 When is low occupancy acceptable?
- 3 Why does padding a shared-memory transpose tile help?
- 4 What is the difference between a barrier, an atomic, and a memory fence?
- 5 A kernel reaches 80 percent of HBM bandwidth. What optimizations remain plausible?

Answer Key

1. Coalescing

Assume each of 32 lanes loads one adjacent four-byte word from an aligned base. The addresses cover 128 contiguous bytes, which can be serviced by four aligned 32-byte sectors on many modern devices. A four-byte base offset can make the range cross a fifth sector. A large stride can require nearly one sector per lane. The goal is to minimize transferred sectors per useful byte, not merely to make each thread's local access look sequential.

2. Low occupancy

Low occupancy is acceptable when existing warps already hide the limiting latency or when instruction-level parallelism and asynchronous pipelines keep execution units busy. Large register tiles, tensor-core fragments, or multi-stage shared-memory pipelines often trade occupancy for reuse. It is unacceptable when the profile shows few eligible warps and exposed memory or dependency stalls that another resident block could cover.

3. Padding a transpose tile

A square tile with row length equal to the shared-memory bank count makes column access advance by a whole bank cycle, so many lanes select the same bank. Adding one element changes the stride to a value relatively prime to the bank count, spreading lanes across banks. The cost is a small shared-memory increase; the benefit is fewer serialized bank transactions.

4. Barrier, atomic, and fence

A barrier makes a group wait until all participants arrive and usually supplies memory visibility for a defined scope. An atomic serializes one read-modify-write operation on a location but does not make unrelated threads rendezvous. A fence orders visibility of memory operations at a scope but does not wait for consumers. Many producer-consumer algorithms require both an ordering mechanism and a separately communicated readiness condition.

5. At 80 percent of HBM bandwidth

First determine whether 80 percent is bus traffic or useful requested bytes. If useful bandwidth is genuinely near the sustainable ceiling, large gains require fewer bytes: fuse passes, improve cache or shared-memory reuse, compress data, change layout, or change the algorithm. Smaller gains may come from reducing uncoalesced sectors, fixing partition camping, overlapping tails, or removing launches. Optimizing arithmetic alone is unlikely to matter unless it overlaps the memory path poorly.

CHAPTER 17

17

Host-Device Orchestration, Streams, and CUDA Graphs

Host-Device Orchestration, Streams, and CUDA Graphs

A fast kernel can sit inside a slow application. Transfers, allocations, launch submission, synchronization, and shape management determine whether kernel speed reaches the user.

The previous section gave a resource model inside one kernel. The running example does not live there. A decode step is a short sequence of kernels, copies, and sampling work submitted from the host or a captured graph. This section is the shell around that sequence: how work is ordered, overlapped, and kept off the global critical path.

The asynchronous execution contract

CUDA calls usually enqueue work into streams. Operations in one stream execute in order. Independent operations in different streams may overlap if dependencies, resources, and hardware engines allow it. "May overlap" is important: concurrency is permitted, not guaranteed.

An event records progress in a stream. Another stream can wait on that event without forcing the host to synchronize the whole device. This builds a dependency graph while preserving unrelated concurrency.

CUDA

```
cudaEvent_t ready;
cudaEventCreateWithFlags(&ready, cudaEventDisableTiming);

cudaMemcpyAsync(d_x, h_x, bytes, cudaMemcpyHostToDevice, copy_stream);
cudaEventRecord(ready, copy_stream);
cudaStreamWaitEvent(compute_stream, ready, 0);
kernel<<<grid, block, 0, compute_stream>>>(d_x, d_y, n);
```

Avoid `cudaDeviceSynchronize()` in a request path unless a full-device boundary is truly required. It converts independent queues into a global critical path and can hide the operation that actually created an error.

Overlapping transfer and compute

Host-device overlap normally requires pinned host memory, asynchronous copy APIs, distinct streams, hardware support, and independent buffers. Double buffering partitions input into chunks:

- 1 copy chunk $i + 1$ to device;
- 2 compute chunk i ;
- 3 copy result $i - 1$ to host.

Chunk size balances transfer efficiency, pipeline fill/drain, and working-set memory. Tiny chunks pay submission overhead; huge chunks leave little overlap.

Pinned memory should be pooled. Pinning and unpinning are heavyweight, and excessive pinned memory harms the operating system by reducing pageable capacity.

Default-stream hazards

Legacy default-stream semantics can synchronize with work in other streams. Per-thread default streams and nonblocking streams provide different behavior. Production code should choose a convention explicitly rather than rely on compilation defaults that vary across components.

Libraries may enqueue work on a caller-provided stream or an internal stream. Document stream ownership and lifetime. A tensor cannot be freed, reused, or modified until all streams that access it have completed their dependencies.

Stream priorities and fairness

Stream priority is a scheduling hint, not a deadline guarantee. A high-priority kernel cannot preempt every already-running block. Long-running persistent kernels and huge blocks can therefore delay latency-sensitive work even when placed in a higher-priority stream.

Design cooperative yielding or partition capacity when strict service isolation matters. Measure interference using the production mixture, not isolated streams.

Allocation on the critical path

Synchronous allocation can introduce global coordination. Stream-ordered allocation with `cudaMallocAsync` and `cudaFreeAsync` ties lifetime to stream progress and lets the runtime reuse a memory pool.

Pool policy includes release threshold, device and process scope, cross-stream dependencies, fragmentation, and warmup. A fast allocator does not excuse unbounded temporary memory. Track high-water mark and reserve behavior.

CUDA Graphs

A CUDA Graph represents a repeated dependency graph of kernels, copies, and supported memory operations. Instantiating the graph performs setup once; launching the executable graph reduces repeated CPU submission overhead.

Graphs are valuable when:

- the sequence of operations repeats;
- kernel work is small enough that launch overhead matters;
- addresses and shapes are stable or can be updated within supported limits;
- the application can manage graph variants and lifetimes.

Graphs are less useful when the workload changes structure every iteration, graph updates are frequent and expensive, or one long kernel dominates latency.

In the decode regime of the running example, each step may launch several short kernels: RoPE or cache append, attention, MLP GEMMs, normalization, logits, and sampling. If each launch costs tens of microseconds and the GPU work is also short at small batch, CPU submission becomes visible. Graph capture amortizes that overhead for stable shape buckets.

CUDA

```
cudaGraph_t graph;
cudaGraphExec_t graph_exec;

cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
stage1<<<g1, b1, 0, stream>>>(a, tmp);
stage2<<<g2, b2, 0, stream>>>(tmp, out);
cudaStreamEndCapture(stream, &graph);

cudaGraphInstantiate(&graph_exec, graph, nullptr, nullptr, 0);
for (int step = 0; step < steps; ++step) {
    cudaGraphLaunch(graph_exec, stream);
}
```

Graph executables and captured resources have ownership constraints. Reusing one executable graph concurrently may be restricted. Graph memory nodes and stream-ordered allocations have GPU-ordered lifetimes that must be respected.

Shape buckets and graph variants

LLM engines often maintain graph variants for batch and token-count buckets. Padding to a captured shape trades extra device work for stable launch overhead and addresses. Too many buckets create compilation, memory, and test burden.

Choose buckets from traffic distribution and kernel sensitivity. Keep a general eager fallback. Track hit rate, padded work, graph memory, and update failures.

For the running example, a decode graph family might cover B in $\{1, 2, 4, 8\}$ with a small set of context-length buckets. Padding eight sequences to the next captured length is acceptable only when the saved launch latency exceeds the wasted KV and GEMM work.

CAPTURE ONLY AFTER THE DEPENDENCY GRAPH IS STABLE

Do not graph-capture a prototype that still allocates, synchronizes globally, or changes topology every step. First make the eager path stream-ordered and leak-free; then capture the steady-state shape.

Multi-GPU copies and peer access

Device-to-device transfers can use peer access over the available interconnect. The relevant path may be PCIe, NVLink, or another fabric, with topology-dependent bandwidth and routing. Batch small transfers, preserve alignment, and overlap communication only when independent compute exists.

For collectives, use a collective library rather than hand-rolled peer copies unless the communication pattern is genuinely unusual. The application-level question is still exposed communication on the critical path, not theoretical link bandwidth.

End-to-end timing

CPU wall-clock timing around an asynchronous launch measures submission, not completion.

CUDA events measure elapsed device time between points in a stream. End-to-end latency should use the product boundary and include queueing, copies, and synchronization.

CUDA

```
cudaEventRecord(start, stream);
kernel<<<grid, block, 0, stream>>>(args);
cudaEventRecord(stop, stream);
cudaEventSynchronize(stop);
cudaEventElapsedTime(&milliseconds, start, stop);
```

Warm up context creation, allocations, JIT compilation, graph upload, and caches according to the claim. Report both cold and warm behavior when users encounter both.

Principal Interview Review

- 1 What conditions are required to overlap a host-device copy with compute?
- 2 Why can `cudaDeviceSynchronize()` destroy concurrency?
- 3 When do CUDA Graphs help, and when do they not?
- 4 How would you choose graph shape buckets for an inference engine?
- 5 Design a double-buffered input pipeline and name its failure modes.

Answer Key

1. Copy-compute overlap

The copy must be asynchronous, host memory should normally be pinned, source and destination buffers must not conflict with compute, the work must be in streams without an ordering dependency, and hardware must support the required concurrent engines. Overlap can still be limited by shared copy engines, memory bandwidth contention, or a chunk too small to amortize submission.

2. Device synchronization

`cudaDeviceSynchronize()` waits for all previously submitted work on the device, including independent streams. It inserts a host-visible global boundary, prevents pipelining across iterations, and can turn recoverable queue slack into latency. Prefer events or stream synchronization scoped to the dependency actually required.

3. CUDA Graphs

Graphs help repeated fine-grained workloads where CPU launch and runtime setup are visible. They do not accelerate the instructions inside a dominant kernel. They lose value when topology, shapes, addresses, or operation sequences change so frequently that capture, update, or variant management exceeds the saved launch cost.

4. Graph buckets

Measure the joint distribution of batch size, token count, and model path. Choose a small set that captures most traffic with tolerable padding, while capping executable-graph memory and compile time. Route unusual shapes to an eager fallback. Optimize hit rate and saved latency against padded FLOPs and variant ownership, not bucket count alone.

5. Double buffering

Allocate two or more host and device buffers. While compute consumes buffer A, copy the next input into B and return the prior output from another buffer. Connect stages with events. Failure modes include overwriting a buffer before its consumer completes, using pageable memory, chunks too small or large, hidden default-stream synchronization, copy and compute contending for memory bandwidth, and missing backpressure when the producer outruns the GPU.

CHAPTER 18

18

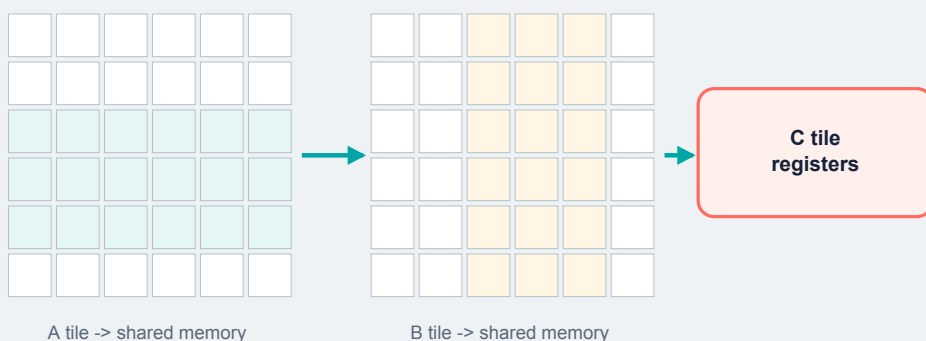
Hierarchical Matrix Multiplication

Hierarchical Matrix Multiplication

GEMM performance comes from hierarchical reuse. Each fetched A and B value should participate in many multiply-accumulates before leaving on-chip storage.

With the machine model and host shell in place, the next building block is the matrix multiply that dominates both prefill and the MLP path. In the running example, a linear layer maps activations of width $D = 4096$ through a weight matrix. Prefill exposes many token rows at once; decode may expose only $B = 8$ rows. The math is the same GEMM. The efficient schedule is not.

TILED GEMM: DATA REUSE BEFORE MORE MATH



A and B tiles are cooperatively staged; warps consume subtiles; threads or tensor-core fragments accumulate C in registers before an epilogue writes it efficiently.

From the equation to the schedule

For $C[M, N] = A[M, K] B[K, N]$, the mathematical work is approximately $2 \cdot M \cdot N \cdot K$ FLOPs. The implementation chooses a loop ordering and a hierarchy of tiles:

- grid tiles partition M and N across thread blocks;
- a block loops over K tiles;
- warps own subtiles within the block tile;
- lanes own scalar accumulators or matrix fragments;
- an epilogue maps accumulator layout to coalesced output stores.

The kernel must balance parallel output tiles against reuse along K .

A correct scalar baseline

CUDA

```
__global__ void gemm_naive(
    const float* A, const float* B, float* C,
    int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= M || col >= N) return;

    float acc = 0.0f;
    for (int k = 0; k < K; ++k) {
        acc += A[row * K + k] * B[k * N + col];
    }
    C[row * N + col] = acc;
}
```

This is a valuable oracle and a poor high-performance schedule. Neighboring threads reread A rows and B columns instead of explicitly sharing them.

Cooperative shared-memory tiling

CUDA

```
template<int TILE>
__global__ void gemm_tiled(
    const float* A, const float* B, float* C,
    int M, int N, int K) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float acc = 0.0f;

    for (int k0 = 0; k0 < K; k0 += TILE) {
        int ak = k0 + threadIdx.x;
        int bk = k0 + threadIdx.y;
        As[threadIdx.y][threadIdx.x] =
            (row < M && ak < K) ? A[row * K + ak] : 0.0f;
```

CUDA (CONTINUED)

```

Bs[threadIdx.y][threadIdx.x] =
    (bk < K && col < N) ? B[bk * N + col] : 0.0f;
__syncthreads();

#pragma unroll
for (int k = 0; k < TILE; ++k) {
    acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
}
__syncthreads();
}

if (row < M && col < N) C[row * N + col] = acc;
}

```

Out-of-range loads become zero, all participating threads reach both barriers, and the output store is guarded. An early return before a later barrier would be incorrect for an edge block.

Arithmetic intensity from tile reuse

Ignoring the final store, a square block tile of width T loads roughly $2 \cdot T^2$ values per K tile and performs roughly $2 \cdot T^3$ FLOPs. Its shared-tile arithmetic intensity is proportional to:

$$2 \cdot T^3 \text{ FLOPs} / (2 \cdot T^2 \text{ values} * \text{bytes_per_value}) = T / \text{bytes_per_value}$$

Larger T raises reuse, but resource consumption grows. The estimate also assumes each input tile is loaded once and does not include cache effects, epilogue traffic, or edge waste.

Running-example intensity sketch

Consider the decode projection that maps $M = B = 8$ rows through $K = N = D = 4096$ in FP16 weights.

- FLOPs are approximately $2 * 8 * 4096 * 4096 = 2.7e8$.
- Weight traffic alone is approximately $4096 * 4096 * 2 = 34 \text{ MB}$ if weights are read once.
- Arithmetic intensity against weights is roughly $2.7e8 / 3.4e7 = 8 \text{ FLOP/byte}$ before activation and output traffic.

On a device whose HBM roof is near 3 TB/s, even perfect weight streaming yields only tens of microseconds of memory time, while tensor-core peak would finish the math much faster if data were free. Decode linear layers are therefore often weight-bandwidth bound at small batch. Prefill with $M = 2048$ raises intensity and can move toward the compute roof for the same weights.

Register tiling and microkernels

One thread can compute an $r_m \times r_n$ output patch. It loads a small vector from A and B , then updates multiple independent accumulators. Register tiling increases reuse from shared memory and exposes instruction-level parallelism.

The cost is $r_m * r_n$ accumulator registers plus address and staging state. Large microtiles can lower occupancy and cause spills. A good microkernel balances accumulator reuse against live-state pressure.

Tensor-core execution

Tensor cores execute matrix multiply-accumulate on architecture-defined fragments. A high-performance tensor-core kernel must:

- use supported input, accumulator, and output types;
- satisfy alignment and layout constraints;
- stage data in a layout that avoids shared-memory conflicts;
- map lanes to matrix fragments correctly;
- pipeline copies and MMA instructions;
- define numerical behavior for reduced precision and accumulation;
- execute an efficient epilogue.

Mixed precision changes the error model. FP16 or BF16 inputs with FP32 accumulation usually reduce accumulation error compared with low-precision accumulation, but input rounding and operation order still differ from an FP32 reference. TF32, FP8, INT8, and block-scaled formats each require a stated quality contract.

Multi-stage pipelines and asynchronous copies

Let stage s hold the tile being consumed and stage $s + 1$ the tile being filled. A software pipeline repeats:

- 1 issue an asynchronous copy for a future A/B tile;
- 2 wait until the current stage is ready;
- 3 execute MMA on the current stage;
- 4 advance the circular stage index.

More stages can cover memory latency but consume more shared memory. At some point the extra stage reduces resident blocks or adds address and synchronization overhead without improving tensor-core utilization.

Modern architectures may provide bulk-copy engines and warp-specialized schedules in which producer warps move data while consumer warps issue MMA. The transferable idea is division of labor with explicit pipeline state, not one architecture-specific mnemonic.

Layouts and the epilogue

The accumulator layout that suits MMA instructions may not produce coalesced global stores. The epilogue remaps fragments, often through shared memory, and applies operations such as:

```
D = activation(alpha * accumulator + beta * C + bias)
```

Fusing bias, scaling, activation, quantization, or residual operations can eliminate a full read and write. An overgrown epilogue can increase registers, inhibit vector stores, or create too many specialized variants.

Split-K, Stream-K, and persistent scheduling

When M and N expose too few output tiles but K is large, split K across blocks. Partial C tiles are combined with atomics or a workspace reduction. The extra reduction is worthwhile when it unlocks otherwise idle SMs.

In the running example's decode GEMM with $M = 8$, ordinary output tiling may leave most of the GPU idle. Split-K or a persistent wave across the weight matrix recovers utilization, at the cost of partial-result traffic.

Stream-K-like scheduling distributes K work more evenly across a fixed set of work units to reduce wave quantization and tail imbalance. Persistent kernels keep blocks resident and pull tiles from a global or hierarchical queue. These strategies improve utilization for awkward shapes but add coordination and deterministic-order questions.

Batched and grouped GEMM

Batched GEMM executes many matrices with compatible shapes and strides. **Grouped GEMM** accepts varying shapes and offsets. Both appear in attention heads, adapters, and mixture-of-experts layers.

Small matrices may be launch- and scheduling-bound. Grouping several problems into one persistent kernel increases available work but requires load balancing. Sorting by shape can improve efficiency while adding preprocessing and reordering cost.

Library, template framework, or custom kernel

Use cuBLAS or another vendor library for standard dense GEMM. Use a template framework such as CUTLASS when you need controlled layouts, data types, schedules, or epilogues without writing every hardware primitive. Write a custom kernel when the operator is unusual enough that the library boundary creates material traffic or launch cost.

The decision is based on end-to-end value, maintainability, architecture coverage, and test burden. Beating a library on one shape is not the same as owning a production GEMM.

WINNING ONE SHAPE IS NOT OWNING GEMM

A custom kernel that beats a library on $M = 8$ decode can lose on prefill, on another dtype, or after an architecture upgrade. Measure the production shape mix and the maintenance boundary before replacing a library path.

Principal Interview Review

- 1 Why does tiling increase arithmetic intensity?
- 2 Fix the out-of-bounds and barrier bug in a naive shared-memory kernel.
- 3 What tradeoff determines tile size and pipeline depth?
- 4 When does split-K help, and what new cost does it create?
- 5 How do tensor cores change implementation and error analysis?
- 6 Why does a high-performance GEMM need an epilogue?

Answer Key

1. Tiling and arithmetic intensity

A tile is loaded once into shared memory and reused by many output accumulators. For a square width- T tile, input traffic grows like T^2 while multiply-accumulate work grows like T^3 , so operations per loaded byte grow with T . The exact gain depends on cache, dtype, edge tiles, and whether the compiler actually preserves the intended reuse.

2. Bounds and barriers

Do not return threads that will be needed at a later `__syncthreads()`. Every thread loads either a valid element or zero into each shared tile, then all threads reach the barrier. After compute, all threads reach the second barrier before the tile is overwritten. Only the final global store is predicated on a valid output coordinate.

3. Tile size and pipeline depth

Larger tiles and more stages increase reuse and overlap. They also consume shared memory, registers, threads, and instruction budget, which can reduce residency or spill. The optimum is the smallest resource footprint that keeps copy and compute pipelines busy for representative shapes. Use the occupancy calculation and achieved-stall profile, then benchmark nearby configurations.

4. Split-K

Split-K helps when the M -by- N tile grid is too small to occupy the device and K is large enough to partition. It creates multiple partial results for each C tile, requiring atomic accumulation or a separate reduction workspace. That adds traffic, synchronization, possible nondeterminism, and initialization cost. It loses when ordinary output parallelism is already sufficient.

5. Tensor cores and error

Implementation changes from scalar FMA loops to architecture-defined matrix fragments, with stricter dtype, alignment, and layout requirements. The kernel must pipeline data into the MMA path and map fragments to an output epilogue. Error analysis must specify input conversion, accumulation dtype, rounding, operation order, saturation or scaling for integer and FP8 paths, and tolerance versus a high-precision reference.

6. The epilogue

MMA accumulator fragments are arranged for compute, not necessarily coalesced output. The epilogue rearranges them for efficient stores and applies α , β , bias, activation, residual, or quantization while the values are still on chip. Without it, output stores may be inefficient and downstream elementwise kernels would reread and rewrite the entire matrix.

CHAPTER 19

19

Reductions, Prefix Scans, and Histograms

Reductions, Prefix Scans, and Histograms

Collective primitives expose the essential GPU issues: participation masks, associative structure, synchronization scope, contention, and the boundary between local and device-wide coordination.

GEMM builds the large linear maps. Softmax, normalization, sampling, and expert routing still need collective operations: maxima, sums, prefix offsets, and bin counts. This section develops those primitives once, with ownership and masks stated carefully, so later kernels can compose them instead of inventing synchronization ad hoc.

Reduction as a tree

A reduction replaces a linear dependency with a tree. Within a warp, shuffle instructions exchange register values. Across warps, one common pattern writes a partial per warp to shared memory and lets the first warp finish.

CUDA

```
__device__ float warp_sum(float x, unsigned mask) {
    int lane = threadIdx.x & 31;
    for (int offset = 16; offset > 0; offset >>= 1) {
        float other = __shfl_down_sync(mask, x, offset);
        int source_lane = lane + offset;
        if (source_lane < 32 && (mask & (1u << source_lane))) x += other;
    }
    return x;
}

__device__ float block_sum(float x) {
    __shared__ float partial[32];
    int lane = threadIdx.x & 31;
    int warp = threadIdx.x >> 5;
    unsigned mask = __activemask();

    x = warp_sum(x, mask);
```

CUDA (CONTINUED)

```

if (lane == 0) partial[warp] = x;
__syncthreads();

int warp_count = (blockDim.x + 31) >> 5;
float value = (threadIdx.x < warp_count) ? partial[lane] : 0.0f;
if (warp == 0) {
    unsigned first_warp_mask = __ballot_sync(0xffffffff, lane < warp_count);
    value = warp_sum(value, first_warp_mask);
}
return value; // valid in lane 0 of warp 0
}

```

The ownership contract matters: only one lane owns the final value. A caller that lets every thread store value is incorrect.

Device-wide reduction

Blocks cannot generally synchronize with arbitrary other blocks in one ordinary launch. Common strategies are:

- first kernel writes one partial per block; second kernel reduces partials;
- blocks atomically accumulate into one or several outputs;
- a persistent/cooperative kernel performs a grid-wide protocol on supported launches;
- use a production primitive such as CUB.

Atomics are attractive when there are few blocks or contention is low. A two-pass reduction has extra launch and workspace cost but scales predictably.

Numerical reduction

Floating-point addition is not associative. A parallel tree changes result order and can vary across launch shape. Accumulating FP16 or BF16 values in FP32 improves stability. Pairwise trees often have better error than one long linear sum. Compensated summation improves accuracy but adds instructions and state.

Determinism may require a fixed partition and reduction order, which can reduce performance. State whether reproducibility means bitwise identity, stable tolerance, or stable model-level metrics.

Prefix scan

An exclusive scan of [a, b, c, d] produces [identity, a, a+b, a+b+c]. An inclusive scan includes the current element. Scan powers stream compaction, radix sort, cumulative offsets, token packing, and sparse indexing.

A work-efficient block scan has an upsweep that builds partial sums and a downsweep that propagates prefixes. Warp-shuffle scans reduce shared-memory traffic within warps; shared memory carries warp totals.

CUDA

```

__device__ int warp_inclusive_scan(int x, unsigned mask) {
    int lane = threadIdx.x & 31;
    for (int offset = 1; offset < 32; offset <= 1) {
        int y = __shfl_up_sync(mask, x, offset);
        if (lane >= offset) x += y;
    }
    return x;
}

```

For a device-wide scan, each block scans a tile, block totals are scanned, and the resulting offsets are added to each tile. A library handles edge cases, recursion, tuning, and temporary storage.

Stream compaction

Compaction keeps elements whose predicate is true:

- 1 compute a zero-or-one flag;
- 2 exclusive-scan flags into output positions;
- 3 scatter selected elements;
- 4 derive output count from the last prefix and flag.

This avoids one atomic per selected item. For very sparse or small workloads, atomics can be simpler and competitive. Compare contention and scan overhead.

Histograms

A naive histogram performs one global atomic per sample. If many samples choose the same bin, contention serializes updates. Common improvements:

- privatize bins per warp or block in shared memory;
- aggregate repeated values within a warp before one atomic;
- use a sort-and-run-length strategy for highly concentrated inputs;
- partition bins or data across blocks, then merge partial histograms.

Shared-memory atomics are not free, and a large bin count may not fit. The input distribution controls the best method. Benchmark uniform, skewed, and adversarial distributions.

These primitives reappear later when mixture-of-experts routing counts tokens per expert and scans those counts into expert-buffer offsets.

Segmented reductions

A segmented reduction combines values within variable-length groups. Segments may be represented by offsets, keys, or boundary flags. Short segments waste blocks if assigned one per block; long segments need multi-block cooperation.

Sort or bucket by size, combine tiny segments per block, and split very long segments. The overhead of preprocessing must be amortized across repeated operations or large workloads.

Gather, scatter, and irregular access

Gather reads `out[i] = input[index[i]]`; scatter writes to indexed destinations. Address locality, duplicate indices, and output conflicts dominate. Sort or reorder indices to improve locality when semantics allow. For scatter with duplicates, define whether updates are last-writer, atomic sum, max, or invalid.

Caching helps repeated gather indices, but random accesses can remain latency-bound. Vectorizing random accesses does not make them coalesced.

Use production primitives deliberately

CUB provides warp-, block-, and device-wide reductions, scans, histograms, and selection. Use it unless a fused operator, unusual data type, fixed small shape, or special semantics justify custom code.

The interview value of implementing a primitive is understanding its invariants. The production decision still favors a maintained library when the boundary fits.

STATE OWNERSHIP BEFORE WRITING THE LOOP

Before coding a reduction or scan, name which lane owns the result, which mask participates, what identity fills inactive lanes, and which barrier makes partials visible. Most collective bugs are ownership bugs dressed as arithmetic bugs.

Principal Interview Review

- 1 Write and explain a warp-plus-block sum reduction.
- 2 When would you choose atomics over a two-pass device reduction?
- 3 Explain a device-wide exclusive scan.
- 4 Design stream compaction and derive the output count.
- 5 How would you optimize a histogram with severe hot-bin contention?
- 6 What does deterministic reduction cost?

Answer Key

1. Warp-plus-block reduction

Each warp reduces its lanes with shuffles under the correct active mask. Lane zero writes a warp partial to shared memory. A block barrier ensures all partials are visible. The first warp loads those partials, filling inactive entries with the identity, and reduces again. Lane zero of the first warp owns the block result. The answer must state mask construction, identity values, barrier placement, and result ownership.

2. Atomics versus two passes

Use atomics when the number of blocks is modest, contention is low, the hardware supports the type efficiently, or the output has several independent bins. Use a two-pass reduction when many blocks target one location, deterministic order matters, or atomic serialization dominates. The two-pass method costs workspace, another launch, and another read/write of partials.

3. Device-wide scan

Each block scans a tile and writes its total. Recursively scan block totals to produce a prefix offset for each tile. Launch a uniform-add phase that adds the tile offset to every local prefix. Production single-pass variants exist, but this three-stage decomposition is easy to reason about and test. The operator must be associative and the identity must be correct.

4. Stream compaction

Map the predicate to flags. Exclusive-scan the flags; each true element writes to `output[prefix[i]]`. The final count is `prefix[n-1] + flag[n-1]` for nonempty input. Handle `n = 0`, avoid duplicate writers, and choose stable or unstable semantics explicitly.

5. Hot-bin histogram

Reduce the number of contending atomics. Privatize bins per warp or block, aggregate equal values inside a warp, then merge partial histograms. If the bin range is large or the distribution is extremely concentrated, sort or partition keys and reduce runs. Measure shared-memory footprint, merge traffic, and skewed cases; a uniform-only benchmark hides the problem.

6. Deterministic reduction

Determinism fixes partitioning and operation order, restricts atomic races, and may require extra passes or workspace. This can reduce load balancing, eliminate faster nondeterministic atomics, and lower parallelism. The cost is justified only when bitwise reproducibility is a real requirement; many applications need a numerical tolerance rather than identical bits.

CHAPTER 20

20

Softmax, Normalization, Top-K, and Sampling

Softmax, Normalization, Top-K, and Sampling

These kernels are small relative to GEMM in FLOPs and large in system importance. They are often bandwidth- or launch-bound, numerically sensitive, and excellent fusion candidates.

The collectives from the previous section become transformer epilogues here. Softmax is two reductions plus a normalize pass. LayerNorm and RMSNorm are statistics plus an affine map. Top-k and sampling sit at the end of a decode step. In the running example, these kernels rarely dominate FLOPs, but they dominate launch count, numerical edge cases, and fusion decisions around attention and logits.

Stable softmax

For a row x , softmax is:

$$p_i = \exp(x_i - m) / \sum_j \exp(x_j - m), \text{ where } m = \max_j x_j$$

Subtracting the row maximum prevents overflow. A basic row kernel has three logical phases:

- 1 reduce to the row maximum;
- 2 reduce the shifted exponentials to the denominator;
- 3 normalize and write outputs.

CUDA

```
float local_max = -INFINITY;
for (int i = threadIdx.x; i < cols; i += blockDim.x) {
    local_max = max(local_max, x[row * cols + i]);
}
float row_max = block_max(local_max);

float local_sum = 0.0f;
```

CUDA (CONTINUED)

```

for (int i = threadIdx.x; i < cols; i += blockDim.x) {
    local_sum += expf(x[row * cols + i] - row_max);
}
float row_sum = block_sum(local_sum);

for (int i = threadIdx.x; i < cols; i += blockDim.x) {
    y[row * cols + i] = expf(x[row * cols + i] - row_max) / row_sum;
}

```

The code may recompute exponentials in the final loop. Keeping every value in registers can be faster for narrow rows but creates spills for wide rows. Recompute spends special-function arithmetic to avoid HBM or local-memory traffic.

For attention in the running example, a softmax row length equals the attended context. Prefill rows can be length 2048; decode rows can be length 4096. Materializing those scores in HBM is the problem FlashAttention later removes. The reduction structure remains the same.

Online softmax derivation

Suppose a processed prefix has maximum m_a and exponential sum $l_a = \sum \exp(x - m_a)$. A new tile has maximum m_b and local sum $l_b = \sum \exp(x - m_b)$. The combined maximum is:

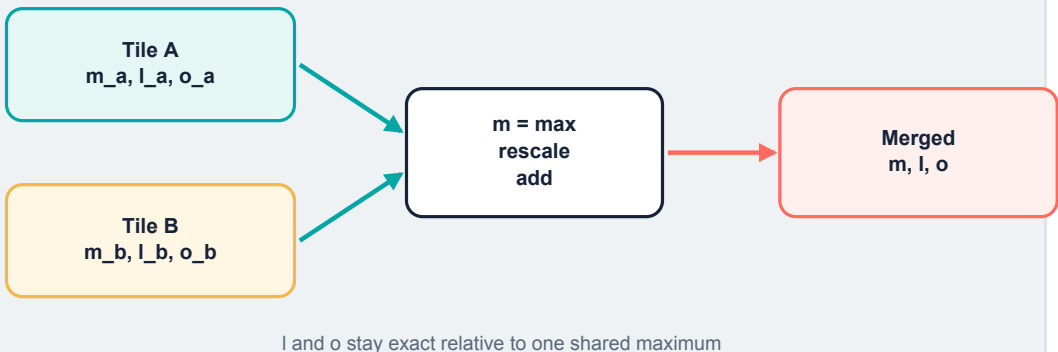
$$m = \max(m_a, m_b)$$

Rescale each partial into the new reference frame:

$$l = \exp(m_a - m) l_a + \exp(m_b - m) l_b$$

This state (m, l) is associative up to floating-point order. Adding an output accumulator produces the recurrence used in tiled attention.

ONLINE SOFTMAX: MERGE TILE STATES WITHOUT FULL SCORES



Two tiles keep compact state instead of the full score vector. Merging rescales both sides into one maximum, then adds denominators and weighted outputs.

Masking and edge cases

A masked element contributes negative infinity before the maximum. A fully masked row is a policy decision: produce zeros, NaNs, or a declared fallback. Silently evaluating $\exp(-\text{inf} - -\text{inf})$ produces invalid arithmetic.

Causal masks can avoid loading disallowed tiles. Arbitrary masks may add branches or extra memory traffic. Packed ragged sequences require per-row lengths and careful prevention of cross-sequence reads.

Softmax backward

Given output p and upstream gradient g , the gradient with respect to logits is:

$$dx_i = p_i * (g_i - \sum_j g_j p_j)$$

This needs one dot-product reduction followed by an elementwise pass. Fusing the reduction and output when the row fits on chip avoids an intermediate. Accumulate the dot product in higher precision for low-precision inputs.

LayerNorm

LayerNorm over a row computes:

$$\text{mean} = \sum x_i / N$$

$$\text{variance} = \sum (x_i - \text{mean})^2 / N$$

$$y_i = \text{gamma}_i * (x_i - \text{mean}) / \sqrt{\text{variance} + \text{epsilon}} + \text{beta}_i$$

A one-pass Welford state (count, mean, M2) is numerically stable and mergeable across lanes. A two-pass mean-then-variance kernel can be competitive when rows are moderate and input rereads hit cache.

In the running example, normalization rows have length $D = 4096$. That fits a block-strided reduction comfortably. The dominant cost is usually reading and writing the activation tensor, which is why residual-plus-norm fusion matters.

LayerNorm backward reduces both $\sum(dy \text{ times } \text{gamma})$ and $\sum(dy \text{ times } \text{gamma times } x_{\text{hat}})$ for each row, then applies the closed-form gradient. Gradients for gamma and beta reduce across rows and may require a second dimension of parallelism.

RMSNorm

RMSNorm omits mean subtraction:

$$\text{inv_rms} = 1 / \sqrt{\text{mean}(x_i^2) + \text{epsilon}}$$

$$y_i = \text{gamma}_i x_i \text{ inv_rms}$$

It needs one reduction and is simpler to fuse. Accumulate squares in FP32 when inputs are FP16 or BF16. Extremely large values can still overflow before accumulation if conversion or scaling is wrong.

Residual and normalization fusion

A common transformer sequence is:

```
residual = x + branch
```

```
normalized = RMSNorm(residual)
```

Fusing residual addition, optional bias, normalization, and perhaps quantization can eliminate one or more full-tensor round trips. The kernel must still expose the updated residual if a later branch needs it.

Fusion tradeoffs include:

- saved HBM bytes and launches;
- extra live values and register pressure;
- more output tensors;
- dropout or RNG semantics;
- backward-save requirements;
- variant count across dtype and hidden size.

For one residual stream of shape `[tokens, D]` in FP16, a separate residual add performs two tensor reads and one intermediate write. RMSNorm must then read that intermediate to compute statistics, read it again to apply the scale unless the row remains on chip, and write the output. The unfused lower bound is therefore five full activation-tensor transfers and the common two-pass case is six, before counting gamma. Fusion removes at least the intermediate write and reread. At prefill with `tokens = 2048` and `D = 4096`, each FP16 tensor is 16 MiB, so eliminating two full transfers saves at least 32 MiB per fused site. At decode with `tokens = 8`, the absolute byte saving is small, but avoiding a launch can still matter.

Top-k selection

Full sorting is unnecessary for small `k`. A common hierarchy is:

- 1 each thread scans a strided subset and keeps local candidates;
- 2 lanes merge candidates within a warp;
- 3 warps merge through shared memory;
- 4 the block emits `k` values and indices.

For very small `k`, register insertion networks or compare-exchange sequences work well. For larger `k`, radix selection, heap-like structures, or partial sorting may win. The best method depends on vocabulary size, `k`, dtype, and batch.

Distributed vocabulary top-k

If logits are sharded across P ranks, each rank computes its local top-k with global token IDs. The global top-k must lie in the union of $P * k$ candidates, so ranks can all-gather candidates and select again.

Communication is small relative to gathering the full vocabulary. For top-p, local top-k may not include enough mass; use a larger candidate bound, histogram/bucket thresholding, or an exact distributed algorithm with additional rounds.

Sampling pipeline

A sampling kernel may apply temperature, repetition or presence penalties, forbidden-token masks, top-k, top-p, normalization, and RNG. Fusing the pipeline reduces launches but complicates exact semantics and variant coverage.

Counter-based RNG maps request, sequence position, and sample index to deterministic random bits without mutable per-thread generator state. Reordering requests must not change a user's random stream unless the product contract permits it.

FUSION THAT CHANGES SAMPLING SEMANTICS

A fused logits path that reorders penalties, truncates top-p differently, or reindexes RNG under batching can pass unit tests on toy shapes and still alter user-visible text. Preserve the declared probability contract, then fuse.

Principal Interview Review

- 1 Why is stable softmax logically a maximum reduction and a sum reduction?
- 2 Derive the online-softmax merge rule.
- 3 When is recomputing exp cheaper than storing it?
- 4 Compare LayerNorm and RMSNorm kernel structure.
- 5 How would you implement distributed vocabulary top-k?
- 6 What can go wrong when fusing sampling operations?

Answer Key

1. Two reductions

The maximum establishes a safe numerical reference so shifted exponentials cannot overflow. The denominator is then a sum over those shifted exponentials. The second reduction depends on the first result, so they are logically sequential even if each reduction is parallel. Omitting the max risks overflow; using a local max without global row combination gives the wrong distribution.

2. Online merge

For partial states (m_a, l_a) and (m_b, l_b) , choose $m = \max(m_a, m_b)$. Because l_a was measured relative to m_a , convert it by multiplying with $\exp(m_a - m)$; do the analogous conversion for l_b ; then add. This maintains $l = \sum \exp(x - m)$ for the union. The same rescaling applies to a weighted output accumulator.

3. Recompute versus store

Recompute wins when storing exponentials would spill registers or create a global/local-memory round trip whose latency and bandwidth cost exceeds another exp. It is especially attractive on compute-rich GPUs for moderate rows. Storing wins when values already fit in registers, exp throughput is the bottleneck, or the values are needed by several later operations.

4. LayerNorm versus RMSNorm

LayerNorm needs mean and variance, usually Welford or two reductions, then scale and bias. RMSNorm needs only mean square, inverse RMS, and scale. Both are often bandwidth-bound and benefit from residual fusion. LayerNorm backward has two rowwise statistics; RMSNorm backward is simpler. Both should accumulate statistics in higher precision for low-precision inputs.

5. Distributed top-k

Each rank selects local top-k pairs $(score, global_token_id)$. All-gather the $P * k$ pairs and perform a second top-k. This is exact because an item outside a rank's local top-k has at least k better items on that rank and therefore cannot enter the global top-k. Define tie-breaking and NaN handling consistently.

6. Fused sampling risks

Fusion can change operation order, numerical rounding, tie behavior, RNG indexing, and top-p truncation semantics. It can also create high register pressure and too many variants. Validate outputs statistically and, where exact reproducibility is promised, bitwise against the reference for every policy combination. Preserve request-specific RNG under batching and reordering.

CHAPTER 21

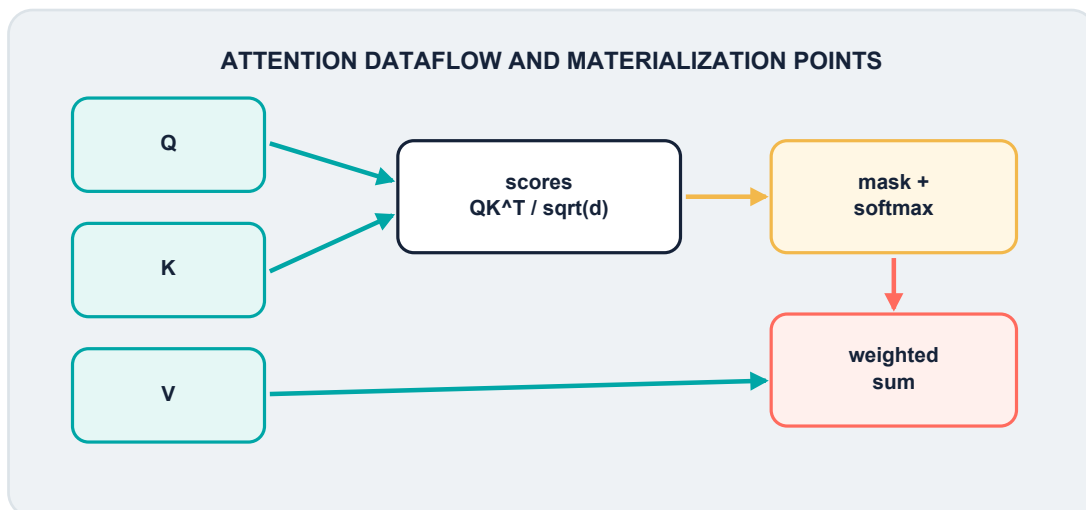
21

FlashAttention and IO-Aware Exact Attention

FlashAttention and IO-Aware Exact Attention

FlashAttention is not an approximate attention mechanism. It is an exact tiled schedule that avoids materializing the quadratic score and probability matrices in HBM.

Online softmax made the merge rule available. FlashAttention is that rule applied to attention tiles so the running example's prefill path never writes S or P to HBM. The algorithm stays exact; the schedule changes.



Attention combines QK scores, masking and softmax, then a weighted V reduction. The performance question is which intermediates must cross HBM.

The naive schedule

For one head, standard attention is:

$$S = Q K^T * \text{scale}$$

$$P = \text{softmax}(S + \text{mask})$$

$$O = P V$$

A naive implementation writes S to HBM, reads it for softmax, writes P , reads P for the V product, and writes O . S and P each contain sequence^2 elements.

Even if the GEMMs are efficient, moving quadratic intermediates can dominate memory and prevent long contexts from fitting.

Running-example HBM contrast

For one head during prefill with $S_{\text{q}} = 2048$ and FP16 scores:

- S or P alone is $2048^2 * 2 = 8$ MiB per head;
- across $H = 32$ heads, one complete score or probability tensor is 256 MiB;
- merely storing both tensors consumes 512 MiB, and writing then rereading each produces at least 1 GiB of quadratic HBM traffic before counting Q , K , V , or output traffic;
- FlashAttention keeps per-row state (m, l) and an output tile of size $S_{\text{q}} * d$, so activation traffic scales with sequence times head dimension rather than sequence squared.

The arithmetic may increase because tiles are rescaled and scores are recomputed in backward. The wall-clock win comes from staying under the HBM roof.

Tiled forward algorithm

Partition Q into row tiles and K/V into column tiles. For each Q tile:

- 1 load Q and initialize per-row maximum m , sum l , and output accumulator o ;
- 2 load a K/V tile;
- 3 compute local scores with tensor-core or SIMT matrix multiply;
- 4 apply scale, causal or explicit mask, and local row maxima;
- 5 rescale old state and add local exponentials;
- 6 update the output accumulator with local probabilities times V ;
- 7 continue across K/V tiles;
- 8 normalize and store O plus compact row statistics.

Q can remain on chip while K/V tiles stream through, or the work can be repartitioned based on sequence length and hardware resources.

Deriving the output recurrence

Suppose existing state uses reference max m_a , denominator l_a , and unnormalized output o_a = $\sum \exp(s_i - m_a) v_i$. A new tile has state (m_b, l_b, o_b) .

Set:

$$m = \max(m_a, m_b)$$

$$\alpha = \exp(m_a - m)$$

$$\beta = \exp(m_b - m)$$

Then:

$$l = \alpha \ l_a + \beta \ l_b$$

$$o = \alpha \ o_a + \beta \ o_b$$

The normalized output is o / l . Every term is now expressed relative to the same maximum.

Causal and ragged workloads

For causal attention, a Q tile does not need future K/V tiles. Diagonal tiles require element-level masking; tiles entirely above the causal diagonal can be skipped. This creates different work per Q tile, so static assignment can leave tail imbalance.

Ragged batches add per-sequence lengths and offsets. Pack sequences to avoid padding, but ensure no tile crosses sequence identity. Sort or bucket by length only if reordering cost and output restoration are controlled.

Work partitioning

The original IO schedule is only the first step. Efficient implementations reduce non-matmul scalar work, increase parallelism across sequence tiles, and assign warp roles to minimize shared-memory exchange.

At short sequence or small batch, there may be too few heads and Q tiles to occupy the GPU. Splitting one head across more blocks increases parallelism but requires combining partial softmax/output states. The merge uses the same max-and-sum recurrence.

Backward equations

Let upstream gradient be dO . The conceptual backward is:

$$dV = P^T dO$$

$$dP = dO \ V^T$$

$$dS = P * (dP - \text{rowsum}(dP * P))$$

$$dQ = dS \ K * \text{scale}$$

$$dK = dS^T \ Q * \text{scale}$$

The kernel recomputes score tiles and P from Q, K, and saved row statistics rather than reading a stored quadratic P. This adds arithmetic and saves quadratic storage and HBM traffic.

Backward partitioning is difficult because K and V receive contributions from many Q tiles. Strategies assign ownership to avoid atomics, use separate kernels, or accumulate partials with controlled reductions. The fastest plan depends on head dimension, sequence, and batch.

What must be saved

At minimum, backward needs Q, K, V or access to recompute them, output O or equivalent state for some formulations, per-row log-sum-exp or max/sum statistics, masks or lengths, scale, and dropout RNG information if dropout is used.

Saving less increases recomputation. Saving more increases activation memory. The chosen boundary belongs in the model's activation-checkpointing plan.

Dropout and reproducibility

Attention dropout must regenerate the same mask in backward. Counter-based RNG can derive mask bits from batch, head, row, column, and a seed/offset. Tiling and work partition changes must not silently change the logical random mask when reproducibility is promised.

Training versus decode attention

FlashAttention-style training and prefill process many queries and exploit matrix-multiply reuse. Decode has one new query per sequence and reads a long KV cache. It is usually memory-bandwidth and latency sensitive, with a very different optimal schedule.

Calling every fused attention kernel "FlashAttention" hides this distinction. State whether the workload is training, prefill, or decode, and model the corresponding shapes.

For the running example's decode step, each of $B = 8$ sequences contributes one query row of length $d = 128$ per head and must read $S = 4096$ cached keys and values. With GQA, $H = 32$ query heads share $H_{kv} = 8$ KV heads, so each KV stream is reused by four query heads. The kernel problem is no longer "avoid S^2 materialization"; it is "stream paged KV efficiently and keep online softmax state compact."

NAME THE ATTENTION REGIME BEFORE NAMING THE KERNEL

If the workload is prefill or training, argue IO-aware tiling and tensor-core occupancy. If it is decode, argue KV layout, paging, vectorized loads, and launch overhead. Reusing the wrong regime's plan is a common false optimization.

Fusion boundaries

Useful attention fusion may include Q/K scaling, RoPE, masking, bias, softmax, dropout, or output transformations. Fusion loses when it inflates live state, reduces occupancy, complicates recomputation, or creates a variant explosion.

Principal Interview Review

- 1 Why can FlashAttention perform more arithmetic yet run faster?
- 2 Derive the running max, sum, and output update.
- 3 What must be saved for backward?
- 4 How does causal masking affect work partitioning?
- 5 When does attention fusion reduce performance?
- 6 Why is a decode-attention kernel different from a training kernel?

Answer Key

1. More arithmetic, less time

FlashAttention recomputes or performs extra scalar work but removes reads and writes of quadratic score/probability matrices. Modern accelerators have far more arithmetic throughput than HBM bandwidth relative to this workload, so spending additional FLOPs to reduce HBM traffic lowers wall time and activation memory.

2. Running state

Merge old and new tile maxima into $m = \max(m_a, m_b)$. Rescale the old denominator and output by $\exp(m_a - m)$ and the new tile by $\exp(m_b - m)$. Add the rescaled sums and weighted outputs. This preserves the exact softmax numerator and denominator relative to one stable maximum.

3. Backward state

Save or make recomputable Q, K, and V; save output or the formulation's equivalent; save per-row log-sum-exp or max/sum statistics; and preserve mask, lengths, scale, and dropout RNG mapping. Full attention probabilities need not be saved because score tiles can be recomputed.

4. Causal work partitioning

Q tiles near the beginning attend to fewer K/V tiles than later Q tiles, and tiles above the causal diagonal are skipped. This creates triangular work and tail imbalance. Assign tiles dynamically, process longer rows first, or split long rows across blocks and merge partial states. Diagonal tiles still need fine-grained masking.

5. Fusion losses

Fusion can increase registers and shared memory, reduce resident blocks, prevent use of optimized GEMM/Tensor Core paths, force incompatible shapes into one schedule, or multiply compile variants. It is beneficial only when saved launches and intermediate bytes exceed these costs for real workloads.

6. Decode versus training

Training/prefill has many query rows and large tiles, enabling compute reuse and tensor-core efficiency. Decode has one new query per active sequence, streams a long and often paged KV cache, and is commonly bandwidth- and latency-bound. Its schedule emphasizes KV layout, vectorized loads, batch mapping, and low launch overhead rather than quadratic-score avoidance.

CHAPTER 22

22

CUDA Kernels for LLM Inference

CUDA Kernels for LLM Inference

LLM serving kernels operate on dynamic batches, ragged state, low precision, and strict token cadence. Layout and scheduler contracts are part of kernel design.

Prefill attention and GEMM covered the compute-rich path. This section specializes the schedule for decode and related serving kernels. The serving vocabulary is restated here so the section stands alone: what KV cache is, why it is paged, how GQA shares keys and values, and which contracts a kernel must honor.

Serving contracts restated for kernel work

Autoregressive decode keeps, for every layer and every active sequence, the keys and values of all tokens generated or prefills so far. That working set is the **KV cache**. Its lifetime follows requests, not training batches. Capacity is measured in tokens times layers times KV heads times head dimension times dtype bytes, plus allocator fragmentation.

A **paged KV cache** stores tokens in fixed-size physical pages and maintains a block table from logical token blocks to physical pages. Kernels must translate (`sequence`, `position`) into a page and offset, then load K/V vectors. The page size trades fragmentation against contiguous access and metadata overhead.

Grouped-query attention (GQA) and **multi-query attention (MQA)** reduce KV heads relative to query heads. In the running example, $H = 32$ and $H_{kv} = 8$, so four query heads read one KV head. The kernel must map those query heads to reuse loaded K/V without silently mixing sequences or heads.

Token cadence is the product constraint that decode steps are short and frequent. A kernel that wins a microbenchmark but adds host synchronization, allocator traffic, or an extra full-tensor pass can miss inter-token latency even when its FLOP rate looks healthy.

Embedding lookup and output projection

Embedding lookup is a gather. Adjacent output dimensions for one token are contiguous; token IDs across requests are irregular. Assign lanes across the embedding dimension so one token's vector loads coalesce. Repeated token IDs may benefit from cache, but do not rely on it for the worst case.

The output projection is a GEMM against the vocabulary matrix. For small decode batch it can be bandwidth-bound. Tensor parallelism shards vocabulary or hidden dimension, after which top-k and sampling need distributed coordination.

RoPE

Rotary position embedding rotates pairs of query and key components. For pair (x_0, x_1) and angle θ :

$$y_0 = x_0 \cos(\theta) - x_1 \sin(\theta)$$

$$y_1 = x_0 \sin(\theta) + x_1 \cos(\theta)$$

The kernel can fuse RoPE into Q/K projection output handling or KV-cache write. Precomputed sine/cosine tables trade memory reads for transcendental arithmetic. Generate angles on the fly when table bandwidth or very long positions make it worthwhile.

Layout must define whether paired dimensions are adjacent or split into halves. A mismatch can produce numerically plausible but incorrect results.

CUDA

```
float2 pair = load_pair(q, token, head, pair_id);
float c = cos_table[position * pairs + pair_id];
float s = sin_table[position * pairs + pair_id];
float2 rotated = make_float2(
    pair.x * c - pair.y * s,
    pair.x * s + pair.y * c);
store_pair(q_out, token, head, pair_id, rotated);
```

KV-cache append

Each decode step writes one K and V vector per layer and sequence. The write is small and frequent. Fuse layout conversion, RoPE for K, and quantization when doing so reduces full-vector traffic without blocking the main attention path.

The address depends on logical sequence, token position, layer, KV head, page mapping, and dtype. Keep that indexing contract centralized. A one-off mismatch between append and read kernels corrupts attention silently.

For the running example, one decode step appends, per layer, $B * H_{kv}$ vectors of length d . In FP16 that is $8 * 8 * 128 * 2 = 16$ KiB of K plus the same for V - tiny compared with reading the full context, but correctness-critical because every later attention step depends on the written layout.

Paged KV attention

A block table maps logical token blocks to physical pages. Decode attention traverses pages, loads K/V vectors, computes query-key scores, performs online softmax, and accumulates values. This is the same (m, l, o) recurrence derived earlier; only the K/V addressing and parallelism differ from FlashAttention prefill.

Key design decisions:

- tokens per page;
- K/V head and dimension layout inside a page;
- vector width and alignment;
- how sequences map to blocks or warps;
- how GQA query heads share one KV head;
- how page-table lookups are cached or prefetched;
- where quantization scales reside;
- how partial results merge for long context.

Smaller pages reduce fragmentation and copy cost but increase page-table entries and discontinuities. Larger pages improve contiguous access but waste more tail capacity.

Bandwidth sketch for the running example

One decode attention step, for all heads, must read roughly:

$$B * H_{kv} * S * d * \text{bytes_per_element} * 2 \text{ (K and V)}$$

With $B = 8$, $H_{kv} = 8$, $S = 4096$, $d = 128$, FP16:

$$8 * 8 * 4096 * 128 * 2 * 2 = 1,073,741,824 \text{ bytes} = 1 \text{ GiB}$$

At an illustrative 3 TB/s HBM ceiling, the memory roof is a fraction of a millisecond before scoring arithmetic, softmax, and writes. Useful bandwidth falls if page jumps destroy coalescing. This is why layout and paging dominate decode attention more than peak tensor-core throughput.

Split-K decode attention

One sequence with long context may not expose enough parallel work if one block owns the entire KV range. Partition context across blocks. Each block emits partial (m, l, o) state; a second phase merges states with the online-softmax recurrence.

This is analogous to split-K GEMM: more parallelism in exchange for a partial-result reduction. It helps long contexts and small batches, then loses when batch already supplies enough blocks.

GQA and MQA mapping

With grouped-query attention, several query heads read one KV head. Map those query heads close enough to reuse K/V cache lines or shared tiles. Excessive sharing can increase live output accumulators. The kernel chooses between reloading K/V for simpler ownership and retaining K/V while processing multiple query heads.

KV-cache quantization

Quantized K/V reduces capacity and bandwidth. The kernel reads packed values plus scales, dequantizes into compute registers, and accumulates in higher precision. Scale granularity may be per tensor, head, channel, page, or token group.

Finer scales improve fidelity and increase metadata traffic. Dynamic token scales add append-time reduction. K and V can have different sensitivity and therefore different formats.

If the running example quantizes KV to 4 bits with modest scale metadata, theoretical KV read volume drops by roughly 4x before considering dequant instructions and scale loads. The optimization wins only when the unpack path still feeds the score pipeline faster than the FP16 baseline.

Weight-only quantized GEMM

For low-batch decode, weight bandwidth dominates many linear layers. A weight-only kernel loads packed weights, dequantizes them, and multiplies by higher-precision activations. The schedule must overlap unpack/dequant with MMA and load scales efficiently.

Compression is useful only if the hardware path processes packed data efficiently. A format can reduce stored bytes and still run slower because unpacking, irregular scales, or unsupported tensor-core instructions dominate.

MoE routing and grouped GEMM

Mixture-of-experts layers activate a subset of experts per token. Routing computes top experts, counts tokens per expert, scans counts into offsets, scatters tokens into expert-contiguous buffers, executes grouped GEMM, then scatters outputs back with routing weights.

The pipeline combines top-k, histogram, scan, gather/scatter, and GEMM - the primitives from earlier sections. Fusing every phase is rarely ideal. The important interfaces are compact routing metadata and layouts that let grouped GEMM consume contiguous expert batches.

Hot experts create imbalance. Capacity limits, token dropping, or expert replication change semantics and belong to the model contract, not only the kernel.

Sampling and token cadence

The decode iteration often ends with logits processing and sampling. At small batch, several tiny kernels can add visible launch latency. Graph capture and carefully scoped fusion help. Preserve request cancellation, dynamic sampling policies, and RNG mapping.

DECODE KERNELS INHERIT THE CACHE CONTRACT

Append, attention, and quantization kernels share one address and dtype contract for KV pages. Document that contract once. Most silent attention corruptions are layout mismatches, not softmax algebra mistakes.

Principal Interview Review

- 1 How would you fuse RoPE with KV-cache append?
- 2 Explain the page-size tradeoff in paged KV attention.
- 3 When does split-K decode attention help?
- 4 How should a GQA kernel exploit K/V sharing?
- 5 Why can KV quantization increase capacity but reduce speed?
- 6 Walk through an MoE routing kernel pipeline.

Answer Key

1. RoPE plus append

The Q/K projection produces K in a temporary layout. Instead of writing K, rereading it for RoPE, then writing the cache, assign lanes across K pairs, apply the rotation in registers, convert to the cache layout, optionally quantize, and store directly to the physical page. Preserve a separate Q output path, correct position indexing, and the model's pair layout. Fusion loses if it delays a library GEMM epilogue or inflates registers excessively.

2. Page size

Small pages reduce internal fragmentation and make branch/copy operations cheaper, but increase block-table size, address translation, and discontinuous loads. Large pages improve contiguous vector access and reduce metadata but waste more memory in partially filled tails and make copy-on-write heavier. Choose from context distribution, kernel transaction efficiency, allocator pressure, and sharing behavior.

3. Split-K decode

It helps when batch and head count provide too few blocks while context is long enough to partition. Each block scans a KV range and emits partial softmax/output state; a second phase merges. It costs partial-state memory, another synchronization or launch, and extra arithmetic. It loses at high batch or short context.

4. GQA reuse

Map query heads that share one KV head into the same block or nearby warps so a loaded K/V tile serves several queries. Balance that reuse against more output accumulators and registers. Another design processes fewer query heads per block and reloads K/V, trading bandwidth for occupancy. Benchmark by head dimension, group size, context, and batch.

5. Quantized KV slower

Capacity grows because stored state is smaller, but latency can worsen if scale loads, address calculations, unpacking, type conversion, or uncoalesced packed layout exceed the bandwidth saved. The kernel may also lose vector width or tensor-core eligibility. Measure total bytes and instruction throughput on the exact decode shapes.

6. MoE pipeline

Compute router logits and top experts; form token-expert pairs; count pairs per expert; exclusive-scan counts into expert offsets; scatter token rows into expert-contiguous buffers; execute grouped GEMMs; multiply by routing weights; scatter-add outputs back to token order. Handle capacity, duplicate expert choices, load imbalance, quantization, and multi-GPU all-to-all explicitly.

CHAPTER 23

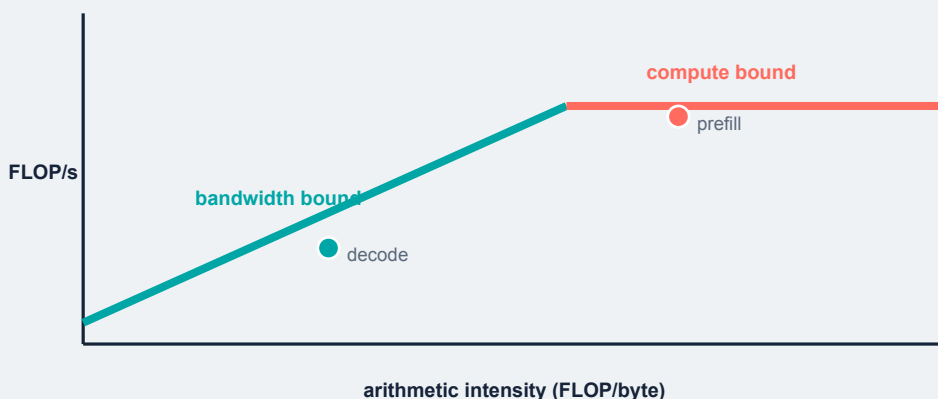
23

Kernel Engineering, Profiling, and Correctness

Kernel Engineering, Profiling, and Correctness

Kernel work is complete only when the optimization is reproducible across representative shapes, numerically valid, integrated into the application, and understandable from evidence.

The preceding sections built a schedule for the running example's layer and decode step. This final section is how you defend that schedule: benchmarks that match the claim, profiler hypotheses tied to resources, and correctness layers that catch the shapes serving actually produces.



A roofline places achieved work against arithmetic intensity. Hierarchical rooflines can reveal whether HBM, L2, L1, or compute is the active ceiling.

Build a trustworthy benchmark

Specify:

- device, architecture, clocks or power mode, toolkit, driver, and compiler flags;
- input shapes and their production frequency;
- dtype, layout, alignment, sparsity, and value distribution;
- warmup, iteration count, synchronization boundary, and cache policy;
- reference implementation and numerical tolerance;
- whether allocation, transfer, launch, and framework dispatch are included.

For this part's running example, a minimum matrix includes prefill [B=1, S=2048] attention and GEMM shapes, decode [B=8, S=4096] paged attention, and the corresponding linear layers at D=4096. Random inputs can hide value-dependent behavior such as histogram contention, sparsity, overflow, or early exit. Include adversarial and production-derived distributions.

Cold, warm, and steady-state claims

Cold latency may include CUDA context initialization, module load, JIT compilation, graph instantiation, allocation, and cache miss. Warm latency may assume all of these are complete. Both are legitimate if labeled.

For interactive systems, report p50 and tail distributions over the real request mix. Median kernel time alone does not predict queueing or graph-cache miss.

Roofline analysis

Arithmetic intensity is useful operations divided by bytes moved across a chosen boundary. The simple roofline bound is:

```
attainable_work_rate <= min(compute_peak, bandwidth *
arithmetic_intensity)
```

Choose the correct work units and peak. Tensor-core FLOPs, scalar FP32 FLOPs, integer operations, and special functions have different ceilings. Choose the correct bytes: requested HBM bytes, measured HBM traffic, or cache-level traffic.

A hierarchical roofline adds L1 and L2 ceilings. A kernel may sit below the HBM roof because it is actually limited by L1 bandwidth, shared-memory conflicts, instruction issue, or dependency latency.

Place the running example on that roof deliberately: decode linear layers and paged attention belong near the bandwidth side; large prefill GEMMs and well-tiled FlashAttention can approach the compute side. If a decode kernel claims compute-bound behavior, demand the intensity calculation.

From profiler sections to a hypothesis

A disciplined Nsight Compute investigation asks:

- 1 Is the kernel compute-, bandwidth-, latency-, or synchronization-limited?

- 2 Is achieved occupancy different from theoretical occupancy?
- 3 Are there enough eligible warps per scheduler?
- 4 Which stall reasons are large, and which instructions create them?
- 5 Are global and shared transactions efficient?
- 6 Are tensor-core, scalar, load/store, and special-function pipes balanced?
- 7 Is the tail caused by grid size or per-block imbalance?

Do not optimize a counter in isolation. A higher cache hit rate can hurt if obtaining it adds instructions or reduces occupancy. A lower stall percentage can mean the denominator changed.

Read generated code

Compiler reports show registers, shared memory, and spills. SASS or disassembly confirms vectorization, tensor-core instructions, local-memory traffic, and unexpected conversions. Source code expresses intent; generated instructions determine the schedule.

Inspect generated code after large performance changes, unexpected register cliffs, or architecture/compiler upgrades. Avoid hand-tuning assembly before proving the compiler path is the bottleneck.

Correctness layers

- 1 **Semantic reference:** compare with a simple CPU or high-level implementation.
- 2 **Shape coverage:** zero, singleton, odd, prime, aligned, maximum, ragged, and empty slices.
- 3 **Value coverage:** zeros, infinities where valid, NaNs, extreme magnitude, repeated keys, adversarial masks.
- 4 **Concurrency:** run with multiple streams, cancellation, reused buffers, and sanitizer tools.
- 5 **Numerical analysis:** absolute/relative error, ULP where useful, distributional error, and model-level quality.
- 6 **Gradient checks:** finite difference on tiny cases plus reference backward.
- 7 **Determinism:** define and test the promised level.

Serving kernels add cases that dense unit tests miss: partially filled final pages, GQA head mapping, sequence packing boundaries, and graph replay with reused device addresses.

Tolerances

Use a combined test such as:

```
abs(actual - reference) <= atol + rtol * abs(reference)
```

Choose tolerances from dtype, operation count, accumulation, and downstream sensitivity. One global tolerance can hide large error near zero or reject harmless scale-proportional error.

For softmax and probabilities, check normalization and KL or output behavior in addition to elementwise error. For top-k, ties require a declared ordering. For quantization, test task quality and long-context behavior.

Race and memory checking

Run compute-sanitizer modes appropriate to memory access, race, initialization, and synchronization. Debug variants may reduce optimization or alter timing, so sanitizer success complements rather than replaces stress tests.

Add canaries around buffers and test noncontiguous strides. Many kernels are correct for dense tensors and silently wrong when framework views carry a different stride.

Autotuning without overfitting

Tune over a declared shape distribution and cache the result by device, toolkit, dtype, layout, and semantic options. Separate a training set of shapes from a validation set to catch over-specialization.

Constrain the search space with resource models. Reject configurations that exceed register/shared limits or produce poor transaction geometry before benchmarking. Retain a safe general fallback.

Integration and regression

A faster kernel can make the application slower through conversion, layout changes, synchronization, graph misses, or loss of fusion elsewhere. Benchmark the operator boundary and the end-to-end model.

Maintain performance thresholds by shape family, not one absolute number. Hardware load and clocks vary; use robust baselines, confidence bands, and dedicated performance workers where possible.

Code review checklist

Area	Review questions
Ownership	Which block, warp, and lane owns each output and partial?
Memory	Are addresses in bounds, aligned, coalesced, and correctly scoped?
Synchronization	Does every participant reach barriers? Are masks accurate?
Resources	What limits resident blocks? Are there spills or bank conflicts?
Numerics	What rounds, accumulates, rescales, or becomes nondeterministic?
Shapes	Which cases use specialized paths and which use fallback?
Integration	Which streams, graphs, allocations, and layouts are assumed?
Evidence	Which measurement proves the optimization helps end to end?

CLOSE ON THE CLAIM YOU OPENED WITH

If the chapter's running example was decode at $B=8$, $S=4096$, the acceptance evidence must include that shape, not only a large square GEMM where every kernel looks good. Optimize the workload you promised to serve.

Principal Interview Review

- 1 Design a benchmark that distinguishes launch-bound from bandwidth-bound behavior.
- 2 A profiler reports low occupancy. What do you do next?
- 3 How do you interpret a high long-scoreboard stall percentage?
- 4 What belongs in a correctness suite for a fused attention kernel?
- 5 How do you choose numerical tolerances?
- 6 When should an autotuner create a specialized kernel variant?

Answer Key

1. Launch-bound versus bandwidth-bound

Sweep problem size and iteration count. Measure device event time for the kernel, an empty or minimal launch baseline, achieved memory bytes, and end-to-end submission. A launch-bound kernel has a relatively flat time floor and improves materially under graph capture or fusion. A bandwidth-bound kernel scales with bytes and approaches sustainable memory throughput; graph capture changes little once the workload is large.

2. Low occupancy

Calculate the resource limiter: threads, registers, shared memory, or block limit. Then inspect achieved eligible warps and stall reasons. If execution units are busy and latency is hidden, do nothing. If exposed stalls correlate with too few warps, test a smaller tile, shorter live ranges, fewer stages, or launch bounds, watching for lost reuse and spills. Optimize performance, not the occupancy percentage.

3. Long-scoreboard stalls

They commonly indicate waiting on long-latency memory dependencies, but the counter alone is not a diagnosis. Locate the load instructions, inspect coalescing and cache behavior, measure eligible warps, and check dependency chains. Possible fixes are improved locality, prefetch, independent loads, more resident warps, or a different data layout. A correctly HBM-bound kernel may legitimately retain these stalls.

4. Fused-attention correctness

Compare forward and backward with a high-precision reference across causal/noncausal, ragged, boundary, GQA/MQA, masked, and extreme-logit cases. Test fully masked-row policy, dropout mask reproducibility, noncontiguous layouts, gradients, multiple streams, and sanitizer results. Verify no cross-sequence access and test tolerance by dtype and head dimension.

5. Numerical tolerances

Start from input and accumulation dtypes, reduction length, operation conditioning, and expected rounding order. Use combined absolute and relative error, add invariant checks such as softmax sums, and examine worst-case plus distributional error. Tighten or loosen only with a documented numerical model and downstream quality evidence, not to make a failing test pass.

6. Specialization

Create a variant when a frequent shape or semantic mode has a materially different optimal schedule and the end-to-end gain exceeds compile, binary, memory, dispatch, and test costs. Bucket nearby shapes when padding is cheaper than another variant. Require a fallback and monitor traffic hit rate so dead variants can be removed.

Further Study and Primary References

- [CUDA C++ Programming Guide](#) - execution, memory, synchronization, streams, graphs, and architecture features.
- [CUDA C++ Best Practices Guide](#) - transfer, coalescing, bandwidth, occupancy, and optimization methodology.
- [Nsight Compute Documentation](#) - profiler sections, metrics, rooflines, and analysis workflow.
- [Efficient GEMM in CUDA](#) - hierarchical GEMM mapping, mainloop, and epilogue design.
- [CUB Documentation](#) - maintained warp-, block-, and device-wide collectives.
- [FlashAttention paper](#) - IO-aware exact attention and tiling analysis.
- [FlashAttention-2 paper](#) - improved work partitioning and parallelism.

Final CUDA Principle

A kernel is a proof that an algorithm, a data layout, and a hardware schedule agree.

The proof has three parts: correctness for every supported shape, a resource model that predicts the bottleneck, and measurements that show the optimization survives integration. For this part, that means the same running example you opened with - prefill tiles that avoid quadratic HBM, and decode steps that honor the KV contract - still holds after profiling and fusion. Missing any one produces a benchmark artifact rather than a production kernel.

PART V - DISTRIBUTED ML SYSTEMS

Part V - Distributed ML Systems



Distributed design begins when one accelerator, process, memory domain, or failure domain cannot satisfy the objective. Every parallelism axis exchanges one scarce resource for another: memory for communication, latency for throughput, replication for availability, or implementation simplicity for scale.

This part builds from communication cost to complete training and inference plans. It covers collectives, topology, data and fully sharded parallelism, tensor and context parallelism, pipelines, sparse experts, distributed inference, checkpoint recovery, cluster scheduling, and diagnosis. Every chapter ends with principal-level questions and complete answer keys.

WRITE THE PHYSICAL PLAN, NOT ONLY THE DEGREES

A configuration such as TP=8, PP=4, DP=16 is incomplete until every process group is mapped to devices, links, nodes, failure domains, tensors, and schedules. Logical parallelism becomes performance only through physical placement.

CHAPTER 24

24

Communication Models, Collectives, and Topology

Communication Models, Collectives, and Topology

Distributed performance is governed by the bytes, synchronization steps, link paths, and overlap opportunities on the critical path. FLOPs alone cannot explain a job that waits on communication.

The alpha-beta model

For a message of n bytes, a first-order point-to-point time model is:

$$T(n) = \alpha + n / \beta$$

α is fixed latency, including launch, protocol, and synchronization. β is sustainable bandwidth for the path and message regime. The model separates two common failures:

- many tiny operations pay latency repeatedly;
- large operations saturate bandwidth or a shared bottleneck.

Real systems add contention, topology hops, protocol thresholds, device synchronization, reduction compute, and software overhead. The alpha-beta model is still useful because it predicts whether to fuse messages, reduce bytes, change the algorithm, or overlap work.

Measure effective values by message size, source/destination placement, concurrency, and direction. Nameplate link bandwidth is not application bandwidth. A path may traverse a GPU fabric, PCIe switch, host bridge, NIC, top-of-rack switch, and oversubscribed spine.

Collective semantics

The core collectives have different ownership contracts:

Collective	Input ownership	Output ownership	Typical ML use
Broadcast	One rank owns full input	Every rank receives full tensor	Parameters, metadata
All-reduce	Every rank owns full contribution	Every rank receives reduced tensor	Replicated gradients
Reduce-scatter	Every rank owns full contribution	Each rank receives one reduced shard	Sharded gradients
All-gather	Each rank owns one shard	Every rank receives the concatenation	Sharded parameters/activations
All-to-all	Each rank owns destination partitions	Each rank receives its incoming partitions	Expert token dispatch
Send/receive	One source and destination per operation	Destination receives message	Pipeline activations

A collective is both data movement and a participation contract. Every rank in the communicator must issue compatible operations in compatible order. Shape, dtype, count, root, and sequence disagreements can hang or corrupt the job.

Ring all-reduce

For p ranks and a tensor of N bytes per rank, a ring all-reduce is commonly decomposed into a reduce-scatter and an all-gather. Each rank sends and receives approximately:

$$2 (p - 1) N / p \text{ bytes}$$

There are approximately $2 (p - 1)$ serialized steps. A simplified time is:

$$T_{\text{ring}} \approx 2 (p - 1) \alpha + 2 (p - 1) N / (p \beta)$$

The ring approaches two tensor volumes per rank for large p , making it bandwidth-efficient for large messages. Its latency term grows with rank count, so a tree or other algorithm can be better for small messages. Implementations choose protocols and algorithms by topology and size; do not assume one ring for every collective.

Trees and hierarchical collectives

A balanced tree reduces the number of latency steps to order $\log p$, but link utilization and total bytes depend on the tree and topology. It often helps small or medium messages where startup dominates.

Hierarchical collectives first reduce or gather within a high-bandwidth domain, then communicate representatives across slower links, then distribute locally. Common domains are accelerator fabric within a node, NIC rails across nodes, or islands inside a rack.

The hierarchy must match contention. If every local rank sends independently through one NIC, aggregate injection bandwidth, not GPU links, limits the operation. If ranks are mapped inconsistently across nodes, the logical ring can cross slow links more often than expected.

Bandwidth accounting

Distinguish:

- **algorithm bandwidth:** useful tensor bytes divided by elapsed time;
- **bus bandwidth:** an adjusted measure reflecting bytes carried by the collective algorithm;
- **link bandwidth:** measured bytes on a physical edge;
- **injection bandwidth:** aggregate traffic a node or NIC can enter into the network;
- **bisection bandwidth:** capacity between large partitions of the cluster.

These numbers answer different questions. A collective can report respectable per-rank bandwidth while oversubscribing a shared uplink and slowing other groups. Compare the definition used by the benchmark with the application traffic pattern.

Topology and process groups

Build a topology inventory before selecting degrees:

- GPUs per high-bandwidth fabric domain;
- PCIe root and CPU socket affinity;
- NIC count, rail mapping, and GPU-to-NIC locality;
- intra-rack and inter-rack oversubscription;
- shared storage and control-plane paths;
- failure boundaries for host, switch, rack, and zone.

Map frequent, latency-sensitive collectives to the fastest domain. Tensor-parallel groups commonly stay within a node or fabric island. Data-parallel groups can span nodes because their larger gradient collectives have more overlap opportunity. Expert and context groups require separate analysis because they move activations and may be sensitive to token imbalance.

Process-group construction is part of correctness. Generate groups from a deterministic rank coordinate system, print membership, and test that every rank derives the same groups. Overlapping groups are legal, but concurrent collectives need a globally consistent launch order or independent synchronization.

Communication and computation overlap

Overlap removes communication from the critical path only when:

- a dependency-free compute interval exists;
- communication and compute use resources that can progress concurrently;
- network traffic does not starve the compute kernel's memory path;
- buffers remain alive and are not reused early;

- the launch order avoids communicator serialization.

Measure **exposed communication**, not total communication duration. If a 4 ms all-reduce overlaps 3 ms of backward compute, only roughly 1 ms is exposed, assuming no slowdown from contention. Timeline comparison must also check whether overlapped compute became slower.

Chunking creates earlier overlap but increases latency and launch overhead. Large buckets improve bandwidth but start later. The optimum follows layer order, bandwidth curve, compute gaps, and communicator scheduling.

GPUDirect, RDMA, and registration

Direct device networking avoids staging every transfer through host memory. It still depends on correct PCIe/NIC topology, memory registration, queue resources, and transport configuration. Registration caches and persistent buffers reduce setup cost; unbounded registration can exhaust resources.

Host control remains relevant. CPU oversubscription, interrupt placement, NUMA mismatch, container limits, or a blocked progress thread can lower collective performance even when GPU and network hardware are healthy.

Collective correctness and deadlocks

Common causes of hangs include:

- one rank takes a different branch and skips a collective;
- communicators issue overlapping collectives in inconsistent order;
- tensor sizes differ because of ragged inputs or a local error;
- a prior CUDA fault prevents a rank from reaching the collective;
- one rank is blocked in input, checkpoint, or host synchronization;
- a failed link or process leaves peers waiting.

Use sequence numbers, communicator IDs, tensor metadata, and call stacks in diagnostics. Timeouts limit damage but do not identify the first divergence. Capture the last successfully completed and first pending collective on every rank.

A SLOW COLLECTIVE MAY BE AN INNOCENT BARRIER

Because ranks synchronize through collectives, the operation often waits for a straggler created earlier by data loading, a kernel, page fault, or another communicator. Measure arrival skew separately from transport duration.

Principal Interview Review

- 1 Derive ring all-reduce bytes and explain when a tree can be faster.
- 2 Compare all-reduce with reduce-scatter plus all-gather by ownership and use.

- 3 How would you map tensor- and data-parallel groups onto a multi-node topology?
- 4 What evidence proves communication-compute overlap is useful?
- 5 Diagnose a collective whose duration is high only on some iterations.
- 6 Why can a bandwidth benchmark disagree with application communication performance?

Answer Key

1. Ring and tree

A ring reduce-scatter sends $(p-1)/p$ tensor bytes per rank, and the all-gather sends the same again, for $2(p-1)N/p$. It uses about $2(p-1)$ ordered steps, so startup grows linearly with ranks. A tree uses order $\log p$ levels and can win for small messages or many ranks where latency dominates. The ring is attractive for large messages because it keeps links busy with near-minimal per-rank bandwidth. Measure the implementation and topology because hybrid algorithms are common.

2. Ownership contracts

All-reduce begins with a full-size contribution on each rank and returns the full reduced tensor to each rank. Reduce-scatter produces only one reduced shard per rank; a following all-gather reconstructs a full tensor when needed. Sharded optimizers exploit the interval between these operations to keep gradients or parameters partitioned, while ordinary data parallelism needs the complete gradient on every replica.

3. Group mapping

Keep frequent latency-sensitive tensor-parallel collectives within the fastest GPU fabric and align ranks with PCIe/NIC locality. Form data-parallel groups from corresponding tensor/pipeline coordinates across replicas, preferably spreading them over independent rails without creating oversubscribed hot spots. Print group membership and physical paths, reserve cross-node bandwidth for large overlap-friendly reductions, and validate placement under one-node failure and scheduler fragmentation.

4. Useful overlap

Compare timelines and step time with overlap enabled and disabled at the same workload. Show that communication starts earlier, the dependency-free compute interval covers it, exposed communication decreases, and neither compute kernels nor other collectives slow from contention. Verify buffer lifetime and numerical equivalence. Total communication duration can increase while step time falls; the relevant metric is the critical path.

5. Intermittent collective latency

Separate rank arrival time from transport completion. Correlate slow iterations with input stalls, long kernels, memory pressure, checkpoint activity, network counters, message size, routing, and other communicator traffic. Identify the earliest late rank and its preceding event. If arrivals are aligned but transport is slow, inspect link errors, shared-uplink contention, protocol thresholds, and topology. Compare per-rank traces rather than only the maximum collective span.

6. Benchmark disagreement

Microbenchmarks often use aligned contiguous buffers, one communicator, steady message sizes, dedicated links, warm registrations, and no competing compute. Applications use buckets, nonuniform timing, multiple groups, topology-specific paths, concurrent storage or KV traffic, and may expose only part of the communication. Ensure the benchmark uses the same ranks, sizes, directions, concurrency, and metric definition before treating its bandwidth as a bound.

CHAPTER 25

25

Data Parallelism, ZeRO, and Fully Sharded Training

Data Parallelism, ZeRO, and Fully Sharded Training

Data parallelism scales independent examples. Sharded variants reduce replicated model state, but replace simple gradient synchronization with a schedule of parameter gathers, gradient partitions, prefetches, and memory-lifetime decisions.

Replicated data parallelism

Each data-parallel rank owns a complete model and optimizer replica and processes different local examples. Backward produces local gradients, and an all-reduce makes the reduced gradient identical before every rank performs the same optimizer update.

For local microbatch b , gradient-accumulation steps g , and data-parallel degree d , the global batch is $B_{\text{global}} = b * g * d$. This assumes b is per rank and every rank contributes the same number of examples. Ragged or filtered batches require explicit weighting so the reduced gradient represents the intended global objective.

Scaling d increases global batch unless b or g changes. That can alter optimization, learning-rate schedule, and sample order. System scaling and training semantics must be planned together.

Gradient reduction and buckets

Frameworks register backward hooks and place parameters into buckets. When all gradients for a bucket are ready, its all-reduce can overlap with backward computation for earlier layers.

Small buckets start early but pay more launch and latency. Large buckets use bandwidth well but may not begin until late in backward. Parameter registration order should roughly match gradient-ready order; unused or conditional parameters complicate readiness.

Gradient accumulation can suppress synchronization for intermediate microbatches and reduce once at the accumulation boundary. This saves communication but holds gradients longer and

changes the overlap opportunity. Ensure the final microbatch triggers synchronization on every rank, including early termination paths.

Memory accounting

For P parameters, per-rank training memory includes some combination of:

- model parameters, possibly low precision;
- gradients, possibly accumulated in higher precision;
- optimizer moments and master parameters;
- activations and saved tensors;
- temporary workspaces, communication buckets, and allocator reserve.

For Adam-like training with low-precision parameters, optimizer states and master weights can exceed model parameter bytes several times over. State the exact dtype and optimizer representation rather than quoting a universal bytes-per-parameter number.

Activation memory follows microbatch, sequence length, hidden size, layer count, checkpoint policy, and parallel partitioning. Sharding model state does not automatically solve activation OOM.

ZeRO stages

ZeRO-style sharding progressively removes data-parallel redundancy:

Stage	Sharded across data ranks	Main new communication/lifetime issue
Stage 1	Optimizer states	Parameter updates need shard ownership and synchronization
Stage 2	Optimizer states and gradients	Gradients naturally arrive through reduce-scatter
Stage 3	Optimizer states, gradients, and parameters	Parameters must be gathered around computation

Exact implementations differ. The principle is that each rank owns a partition of persistent state while temporarily materializing what its local compute needs.

Sharding reduces memory approximately with data-parallel degree for shardable objects, but replicas still carry activations, buffers, some metadata, padding, and peak gathered parameters. Peak memory, not steady shard size, decides whether the model fits.

Fully sharded execution

A fully sharded module generally performs:

- 1 prefetch or all-gather parameter shards for a module group;
- 2 execute forward with materialized parameters;

- 3 reshard or retain parameters according to policy;
- 4 gather again as backward needs the module;
- 5 reduce-scatter gradients to owners;
- 6 update local optimizer shards.

Wrapping granularity controls this schedule. Very small units issue many latency-bound gathers. Very large units raise peak memory and delay prefetch. Flattening compatible parameters reduces metadata and improves collective efficiency but complicates per-parameter tooling and checkpoints.

Backward prefetch can overlap the next parameter gather with current gradient computation. Forward prefetch is useful when execution order is static and CPU issue cannot stay ahead. Too much prefetch creates multiple live gathered units and causes OOM.

Reshard-after-forward decisions

Resharding immediately after forward frees parameter memory but requires another gather in backward. Retaining parameters saves communication but increases peak live state, particularly with many layers or microbatches.

Choose by available memory, module size, recomputation, pipeline schedule, and bandwidth. Hybrid sharding can fully shard within a node and replicate across nodes, reducing cross-node parameter gathers while retaining some memory savings.

Activation checkpointing and offload

Activation checkpointing saves selected boundary tensors and recomputes internal forward operations during backward. It trades compute for activation memory and can change overlap timing. Selective checkpointing targets expensive saved tensors while avoiding recomputation of IO-heavy or nondeterministic work.

CPU or storage offload expands capacity but introduces transfer and page-fault risk. It is viable when transfers overlap and the interconnect sustains the required bytes per step. An offload plan that fits but extends step time beyond the training budget is not a solution.

Checkpointed regions must preserve random-number behavior for dropout or other stochastic operations. Distributed recomputation also needs the same collective order as the original forward.

Mixed precision and global numerical state

Ranks must agree on loss-scaling overflow, skipped optimizer steps, gradient clipping norm, and scheduler progress. A local overflow decision that only one rank follows will desynchronize parameters and future collectives.

Global norm clipping usually reduces partial squared norms across the appropriate parameter ownership group. Avoid double-counting replicated tensors or omitting shards. Optimizer-state sharding changes who owns the update but not the mathematical parameter set.

Uneven inputs and join semantics

If one data rank exhausts input early while others continue, ordinary synchronized training hangs or changes the effective gradient. Prefer globally constructed batches with equal step counts. When uneven-input support is deliberate, participating ranks must shadow collectives or adjust normalization, and additional model collectives may require stricter termination handling.

Input failures are distributed failures. A corrupt sample on one rank, data-service timeout, or divergent filtering decision can leave peers waiting in gradient reduction. Log sample/shard identity and fail the group coherently.

SHARDING IS A SCHEDULE, NOT A MEMORY FLAG

The memory saving appears only if gathers, resharding, prefetch, backward, optimizer ownership, and checkpoints agree on tensor lifetime. A poorly wrapped fully sharded model can both use more peak memory and communicate more.

Principal Interview Review

- 1 Compare replicated DDP, ZeRO stage 2, and fully sharded training.
- 2 How do bucket size and parameter order affect gradient overlap?
- 3 Design a wrapping and prefetch policy for a deep transformer under a hard memory limit.
- 4 When should parameters be retained after forward rather than resharded?
- 5 What distributed state must agree when mixed-precision overflow occurs?
- 6 Why can activation checkpointing make a communication plan slower or incorrect?

Answer Key

1. DDP and sharded variants

DDP replicates parameters, gradients, and optimizer state and all-reduces gradients; it is simple and compute-granular but memory-heavy. ZeRO stage 2 shards optimizer and gradients, often using reduce-scatter, while parameters remain replicated. Fully sharded training also shards parameters, requiring gathers around module execution and careful reshard/prefetch timing. Compare peak bytes, collective frequency and size, overlap, checkpoint format, and local compute efficiency, not only persistent memory.

2. Buckets and order

Smaller buckets become ready earlier and overlap more but pay fixed latency and may compete with compute. Larger buckets reach bandwidth efficiency but start later and can leave a reduction tail after backward. If bucket parameter order differs from gradient-ready order, one late gradient blocks an otherwise ready bucket. Profile ready times, exposed reduction tail, and network contention, then tune size and order for the real model and accumulation schedule.

3. Wrapping and prefetch

Start with transformer-block-sized units, measure each unit's gathered bytes and compute window, and choose the largest unit that preserves peak reserve. Prefetch one dependency ahead when its gather can hide under current compute; cap outstanding gathers to a byte budget. Flatten compatible parameters, special-case unusually large embeddings or heads, reshard after forward when memory is tight, and validate with peak snapshots across the full forward/backward and accumulation schedule.

4. Retain versus reshard

Retain when memory headroom exists and the avoided backward all-gather is expensive or difficult to overlap. Reshard when gathered parameters would accumulate to the peak limit, especially across many layers, microbatches, or pipeline stages. Hybrid policies retain small or soon-reused units and reshard large ones. Measure step time and peak live bytes; persistent shard size alone misses the decision.

5. Mixed-precision agreement

All ranks must reach the same overflow decision, loss-scale update, optimizer-step skip, global gradient norm, clipping coefficient, step counter, learning-rate schedule, and RNG/data progression policy. Reduce overflow flags and partial norms over the group that collectively owns the parameters. If one rank skips while another updates, parameters diverge and subsequent collective order or checkpoint state becomes invalid.

6. Checkpointing interactions

Recomputation adds compute where parameter gathers or gradient reductions were expected to overlap, so the critical path can change. It may repeat collectives, random operations, or mutable side effects unless the region is designed for recomputation. Preserve RNG state, use the same process-group order, avoid external side effects, and profile the new timeline. Memory saved is useful only if recompute and additional communication still meet throughput.

CHAPTER 26

26

Tensor, Sequence, and Context Parallelism

Tensor, Sequence, and Context Parallelism

Intra-layer parallelism partitions the tensor dimensions of one model replica. It makes large layers or long activations fit, but puts communication inside every transformer block and can shrink local kernels below efficient shapes.

Column-parallel linear layers

For a linear operation $Y = X A$, column parallelism partitions output columns of A across p ranks. Each rank computes a shard $Y_i = X A_i$. The input X is replicated across the tensor-parallel group, while the output feature dimension is sharded.

This maps naturally to Q, K, V , or the first MLP projection when attention heads or intermediate features divide evenly. A following operation that can consume the sharded features avoids an immediate all-gather.

Bias, activation, and gating should be applied locally when their partition matches. If the API materializes full Y unnecessarily, an all-gather erases the benefit and adds memory.

Row-parallel linear layers

Row parallelism partitions the input rows of A and the corresponding feature dimension of X . Rank i computes a partial output $X_i A_i$; partials must be summed across ranks. The result is usually produced by all-reduce, or reduce-scatter when the next region can consume a sharded output.

A common transformer plan pairs a column-parallel expansion with a row-parallel contraction. The intermediate activation remains sharded, and only the final partial outputs require reduction. The same principle applies to attention projections: choose adjacent layouts so communication occurs at a small number of deliberate boundaries.

Tensor layouts are types

Treat each tensor as carrying a distribution type such as:

- replicated;
- sharded on hidden, head, vocabulary, sequence, or batch dimension;
- partial, meaning local values require reduction;
- uneven or padded shard with a validity mask.

Operators transform these types. A compiler or framework can insert collectives, but a principal design should state them. Many accidental all-gathers come from an interface that forgets a tensor is already sharded.

Residual connections require compatible layouts. Dropout masks, bias, and normalization statistics must have a defined logical mapping so changing tensor-parallel degree does not silently change model semantics beyond the reproducibility contract.

Vocabulary-parallel embedding and loss

Large vocabularies can shard embedding rows or output logits. For embedding lookup, each rank identifies token IDs it owns, produces local vectors for those IDs, and reduces or routes results.

For vocabulary-parallel cross-entropy, compute a distributed log-sum-exp:

- 1 reduce the maximum logit across vocabulary shards;
- 2 compute local shifted exponentials and reduce their sums;
- 3 locate the target token's owning shard and reduce its logit contribution;
- 4 form loss and local gradient without gathering all logits.

Numerical stability and padding vocabulary entries matter. Sampling from sharded logits similarly needs exact global top candidates or distributed normalization, depending on the decoding policy.

Local GEMM efficiency

Increasing tensor-parallel degree divides matrix dimensions. Communication grows as a fraction of step time, and local GEMMs may lose tile efficiency or tensor-core alignment. The useful degree is bounded by:

- hidden, head, and intermediate dimensions;
- batch-token dimension at the target workload;
- link latency and bandwidth;
- memory required for one shard;
- collective overlap opportunity.

Strong scaling eventually stops when smaller local work no longer amortizes collectives. Quantization or faster kernels can move that crossover to a lower degree because compute shrinks while communication may not.

Sequence parallelism

Within a tensor-parallel region, operations such as layer normalization and dropout do not need the full hidden dimension to be replicated across ranks for every token. Sequence parallelism can partition activations along sequence for these regions, reducing replicated activation memory.

Transitions often use reduce-scatter to produce sequence shards and all-gather before operations that require another layout. It complements tensor parallelism; it does not by itself partition attention's all-token dependency.

Random operations need a logical mask policy. If dropout is intended to be invariant to partition degree, map randomness by global token and feature coordinate, not local rank index. If exact cross-degree identity is not promised, at least preserve distribution and rank consistency.

Context parallelism

Context parallelism partitions the entire sequence and its activations across a group. Tokenwise linear and normalization operations run locally. Attention is different: local queries require keys and values from the full logical context.

Implementations may:

- all-gather K/V for attention, paying memory;
- circulate K/V blocks in a ring while updating online attention state;
- exchange queries and accumulate partial results;
- use hierarchical variants aligned with topology.

Ring-style attention keeps only a K/V block at a time and overlaps transfer with attention compute. It needs an online softmax merge so partial score ranges combine exactly. Backward must route gradient contributions to the owners of Q, K, and V.

Causal and ragged context

Causal attention has triangular work. A naive contiguous sequence partition gives early query blocks little valid K/V work and late blocks much more, creating imbalance. Zigzag or interleaved partitions can distribute early and late positions across ranks. The mapping must preserve output order and mask semantics.

Packed ragged sequences add boundaries and different lengths. Do not let an attention block cross sequence identity. Cost models should use valid query-key pairs, not padded maximum length alone. Load balance may require bucketing, packing, or a nonuniform block assignment.

Choosing TP versus CP

Both can reduce activation memory, but they affect different work. Raising TP shrinks hidden-dimension GEMMs and adds collectives throughout the block. Raising CP shrinks sequence-local activations and attention queries while duplicating weights across the CP group and communicating attention context.

For very long sequence and already efficient hidden-dimension kernels, CP may preserve better local GEMM shapes than another TP step. For huge hidden layers that do not fit, TP remains necessary. Compose them only after measuring the communication paths and divisibility constraints.

Topology and communicator concurrency

TP collectives are frequent and latency-sensitive, so keep TP groups on the fastest local fabric. CP transfers can be large and may span a broader domain if attention compute hides them. When TP and CP overlap, their communicator traffic can contend for the same links.

Schedule group operations in a consistent order across ranks and reserve streams/buffers deliberately. A theoretically independent CP ring and TP all-reduce can deadlock or serialize if communicator creation or launch order differs.

FOLLOW THE TENSOR LAYOUTS

For every transformer sublayer, label input, intermediate, and output as replicated, sharded, or partial. The required collective and its bytes then become visible, and redundant gathers are hard to hide.

Principal Interview Review

- 1 Derive the collectives in paired column- and row-parallel linear layers.
- 2 How would you implement vocabulary-parallel cross-entropy without gathering logits?
- 3 Why can a larger tensor-parallel degree make both compute and communication efficiency worse?
- 4 Compare sequence parallelism with context parallelism.
- 5 Design causal context partitioning that avoids severe load imbalance.
- 6 How do you choose TP and CP degrees for a long-context transformer?

Answer Key

1. Paired linear layers

Column sharding replicates input X , partitions output columns of A , and produces a sharded feature activation with no reduction. A compatible activation and next linear consume that shard. Row sharding of the contraction computes partial full outputs from each input-feature shard, then all-reduces them, or reduce-scatters if the next region accepts a shard. The efficient pair avoids gathering the expanded intermediate and communicates only at the contraction boundary.

2. Vocabulary-parallel loss

Reduce the maximum across vocabulary shards for stable logits, subtract it locally, reduce the local exponential sums into a global denominator, and reduce the target logit from the one owning rank. Each rank can then compute loss and its local gradient shard. Mask padded vocabulary entries and use consistent reduction precision. This avoids materializing batch-by-full-vocabulary logits on every rank.

3. Excess TP

Each extra shard reduces local matrix dimensions and may break tile alignment, lower arithmetic intensity, and expose launch overhead. Collectives remain inside every layer; smaller messages can become latency-bound and a higher rank count adds steps or contention. The group also spans slower links once it exceeds a fabric domain. Profile local GEMM efficiency, collective exposure, and memory; stop strong scaling when step time or cost per token worsens.

4. Sequence versus context

Sequence parallelism typically shards tokenwise activation regions associated with tensor parallelism, reducing duplicated normalization/dropout activations and using layout transitions around tensor-parallel blocks. Context parallelism partitions full network inputs and activations by sequence. Attention then communicates K/V or Q across context ranks because tokens interact. CP targets long-context activation/attention scaling; SP is a lighter complement to TP.

5. Causal balance

Estimate valid query-key work per sequence block. Assign each rank a mixture of early and late query blocks, such as a zigzag mapping, so triangular work sums are similar. Circulate K/V blocks and skip masked interactions, preserve global position and output order, and handle ragged sequence boundaries explicitly. Validate both per-rank attention FLOPs and communication, because equal tokens do not imply equal causal work.

6. TP and CP selection

Use enough TP to fit large hidden layers while keeping local GEMMs efficient and groups within fast links. Use CP when sequence-driven activation memory or attention work remains the constraint and another TP step would make hidden-dimension kernels too small. Sweep feasible factor pairs with actual batch/sequence, measure peak memory, TP and CP exposed communication, attention balance, and end-to-end tokens per second, then map groups to noncontending topology paths.

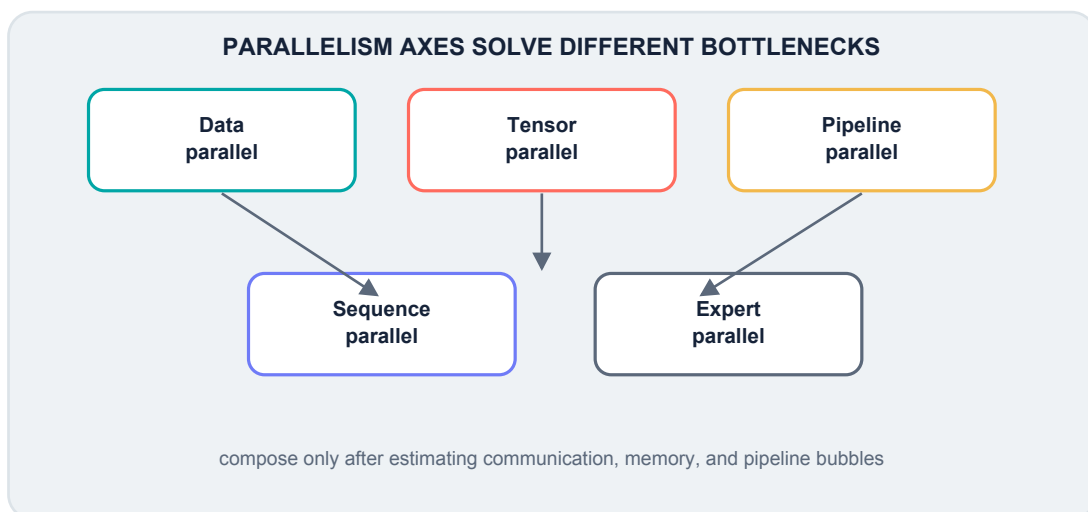
CHAPTER 27

27

Pipeline Parallelism and Hybrid Plans

Pipeline Parallelism and Hybrid Plans

Pipeline parallelism partitions model depth and turns a batch into a schedule of activation and gradient messages. Throughput depends on filling stages, balancing their service times, and controlling the number of in-flight microbatches.



A hybrid plan partitions batch, hidden dimensions, sequence, layers, and experts. Each axis creates a different process group and communication boundary.

Pipeline fundamentals

Partition a model into s ordered stages. A microbatch executes forward through the stages and backward in reverse, with activations and activation gradients sent between neighbors.

For a forward-only pipeline with m equal-time microbatches and balanced stages, a simple bubble fraction is:

$$\text{bubble} = (s - 1) / (m + s - 1)$$

Equivalently, useful stage slots are m out of $m+s-1$. For training schedules, exact utilization depends on forward/backward times, memory policy, and schedule. The approximation still shows why more microbatches amortize fill and drain.

More microbatches are not free. They change local matrix shapes, increase scheduler operations and in-flight activation state, and may force a smaller per-microbatch batch size. The global-batch and optimizer semantics must remain deliberate.

GPipe-style schedule

A flush schedule runs forward for all microbatches, then backward for all. It is simple and has one model-weight version per batch, but retains activations for many microbatches and has a fill/drain bubble.

Activation checkpointing can reduce memory by storing stage inputs and recomputing internal activations. The recompute cost and communication schedule must be included in stage balance.

Flush boundaries simplify checkpointing and failure recovery because no microbatches from the next weight version are in flight.

One-forward-one-backward

After warmup, a 1F1B schedule alternates forward and backward work. It reduces peak live activations because backward frees a microbatch's saved state earlier. It still has warmup and cooldown bubbles.

The number of in-flight forward microbatches differs by stage. Earlier stages may hold more activation state during warmup. Memory planning needs a per-stage schedule simulation rather than multiplying one activation estimate by the same count everywhere.

Schedulers must define send/receive ordering. Blocking operations issued inconsistently across adjacent stages can deadlock. Use explicit tags or sequence IDs, bounded buffers, and a schedule generated identically on every rank.

Interleaved and virtual stages

A device can own multiple nonadjacent model chunks. Interleaving gives it useful work while another chunk waits, reducing bubble and improving balance. It also increases the frequency of activation transfers, complicates ordering, and can enlarge live state.

Virtual-stage count should follow measured imbalance and communication. Too many chunks turn large efficient stage work into small launches and more messages. The right goal is critical-path reduction, not the maximum number of virtual stages.

Stage partitioning

Equal layer count rarely means equal time. Embeddings, output heads, long-context attention, MoE layers, and checkpointed segments have different compute and memory. Partition using:

- measured forward, backward, and recompute time;
- parameter and optimizer bytes;
- peak activation state under the chosen schedule;
- activation-transfer bytes between candidate cuts;
- topology and colocated communicator traffic.

The slowest stage sets steady-state cadence. A 10 percent imbalance can waste roughly that fraction even after bubbles are small. Dynamic sequence length may change which stage is slow, so evaluate workload buckets rather than one synthetic shape.

Activation communication

Boundary activation bytes scale with microbatch, sequence, hidden dimension, and dtype. Tensor parallelism changes ownership: pipeline sends may be per-TP rank, gathered, or scattered depending on adjacent layouts. Preserve consistent rank pairing between stages so traffic uses intended links.

Compression can reduce bytes but adds conversion and numerical risk. It is attractive only when boundary transfer is exposed and the compressed representation is part of the training-quality contract.

Hybrid parallel coordinates

Represent each rank by coordinates such as:

(data, pipeline, tensor, context, expert)

Not every axis is always independent; expert groups may fold with data or tensor dimensions. Write the exact group construction. A common dense-model relation is:

`world_size = DP times PP times TP times CP`

For each coordinate, list:

- tensors owned persistently;
- activations and parameters temporarily materialized;
- communicators and their physical paths;
- microbatch schedule and random-number mapping;
- checkpoint shard identity.

A 64-GPU design example

Suppose eight nodes contain eight tightly connected GPUs each. A model needs more memory than one node but has large hidden layers and moderate sequence length. One candidate is:

- TP=8 within each node;
- PP=2 across pairs of nodes;
- DP=4 across four pipeline replicas;
- CP=1 for the initial sequence target.

TP traffic stays local. Pipeline boundaries cross nodes once per microbatch in each direction. Data-parallel reductions connect corresponding TP/PP coordinates across replicas and should be mapped across network rails.

This is a hypothesis, not a default. Compare with TP=4, PP=4, DP=4 or a sharded-data plan. The first may have better local TP bandwidth but larger pipeline stages; the second changes pipeline bubbles and per-rank GEMM shapes. Measure peak memory and step time under the same global batch.

Selecting the global batch and microbatches

For local microbatch size b , accumulation microbatches m , and data degree d :

$$B_{\text{global}} = b \cdot m \cdot d$$

Increasing m reduces pipeline bubble but raises global batch unless b or d falls. b may already be one sequence, leaving optimization changes or sequence packing as the remaining levers. Do not silently change training hyperparameters to make a systems schedule look efficient.

Plan evaluation

For every candidate plan, calculate and then measure:

- persistent and peak memory per rank and stage;
- local matrix dimensions and achieved compute;
- bytes, steps, and exposed time for every communicator;
- pipeline bubble, imbalance, and recomputation;
- data input and checkpoint bandwidth;
- failure blast radius and spare-capacity requirement.

Reject plans that fit only with zero allocator reserve or depend on perfect overlap. Include shape changes across curriculum, sequence-length ramp, or expert routing.

CHOOSE THE SIMPLEST PLAN INSIDE THE PERFORMANCE ENVELOPE

Every additional axis multiplies process groups, state mappings, checkpoints, tests, and failure modes. Add a parallelism dimension only when it solves a measured memory or throughput constraint that a simpler plan cannot.

Principal Interview Review

- 1 Derive pipeline bubble and explain why more microbatches can still hurt throughput.
- 2 Compare flush and 1F1B schedules by memory, semantics, and recovery.
- 3 How would you partition heterogeneous transformer layers into stages?
- 4 Design rank coordinates and process groups for a 3D or 4D plan.
- 5 Evaluate the 64-GPU example against an alternative plan.
- 6 Why is the fastest isolated parallel plan sometimes wrong for the full training run?

Answer Key

1. Bubble and microbatches

With s equal forward-only stages and m microbatches, useful slots are m across $m+s-1$, so bubble fraction is $(s-1)/(m+s-1)$. Training schedules add backward and schedule-specific terms. Raising m lowers fill/drain fraction but may shrink local batch below efficient GEMM shapes, increase messages and scheduler overhead, increase live activations, or alter global batch. Sweep complete step time and memory, not the formula alone.

2. Flush versus 1F1B

Flush runs all forwards then all backwards, retains more activations, and has clear weight-version and checkpoint boundaries. 1F1B alternates after warmup, releases activations earlier, and reduces peak memory, but has stage-dependent live state and more intricate ordering. Both can preserve synchronous update semantics when weights do not update until the batch completes. Recovery is simpler at flush points; mid-schedule restart needs microbatch commit state or a full-step replay.

3. Stage partitioning

Profile forward, backward, recompute, parameter bytes, and boundary activations by layer and workload bucket. Solve for balanced maximum stage time subject to per-stage peak memory and legal cut points. Account for embeddings, loss, MoE, and long-context attention separately, then map adjacent stages to links and simulate the actual schedule. Validate with per-stage idle time because equal estimated FLOPs may hide memory or communication differences.

4. Rank coordinates

Assign deterministic (dp, pp, tp, cp) coordinates whose product equals world size for the dense case. TP groups vary tp while other coordinates are fixed; PP groups vary pp; CP groups vary cp; DP groups vary dp. Map TP to local fabric, PP to low-contention neighbor paths, and DP across replicas/rails. Print memberships and include expert folding explicitly rather than assuming another independent product dimension.

5. Sixty-four GPUs

For TP8-PP2-DP4, verify that TP8 local GEMMs remain efficient, two-node stage memory fits, pipeline activation traffic is acceptable, and DP groups use independent rails. Compare TP4-PP4-DP4: it may improve local GEMM size and lower TP communication but doubles pipeline depth and boundary count. Measure bubble at the same global batch, stage imbalance, peak bytes, collective exposure, and failure recovery. The winner follows the actual model and topology.

6. Full-run objective

An isolated steady-state step excludes startup, data stalls, checkpoint pauses, sequence curriculum, evaluation, failures, and scheduler fragmentation. A plan may require a changed global batch, have fragile zero-reserve memory, or make checkpoints and resharded restore expensive. Choose time-to-valid-model and recovery-adjusted cost under the full run, while preserving optimization semantics and quality.

CHAPTER 28

28

Mixture-of-Experts and Sparse Communication

Mixture-of-Experts and Sparse Communication

Mixture-of-experts models increase parameter capacity while activating only a subset per token. The systems cost moves from dense compute to routing, variable all-to-all traffic, expert memory, and load imbalance.

The MoE layer

For each token representation, a router scores experts and selects top k . The layer then:

- 1 computes expert assignments and weights;
- 2 counts tokens per destination expert;
- 3 packs tokens into expert-contiguous buffers;
- 4 dispatches tokens to expert owners, often with all-to-all;
- 5 executes expert MLPs as grouped or batched GEMMs;
- 6 sends outputs back to original token owners;
- 7 combines selected expert outputs by routing weights.

Only selected experts compute for a token, but all expert parameters must reside somewhere. Total parameter count drives memory and checkpoint size; activated expert count drives most FLOPs; routing distribution drives communication and local GEMM shapes.

Capacity and overflow

If T tokens are routed across E experts with top-1 routing, ideal load is T/E . A capacity factor c might allocate approximately:

$$\text{capacity_per_expert} = \text{ceil}(c \cdot T / E)$$

Top-k routing increases total assignments. Real load is not uniform, so some experts can overflow while others idle. Possible policies include dropping overflow tokens, routing to a backup expert, increasing capacity and padding, or dynamically scheduling variable counts.

Dropping or rerouting changes model semantics and training. Padding preserves shape but wastes compute. Dynamic shapes improve useful work but complicate kernels, graphs, and collectives. State the policy in the model contract and report overflow by layer and data slice.

Load-balancing objectives

Auxiliary losses encourage balanced token counts and router probability mass. They improve system utilization but can oppose expert specialization. Router z-losses, noise, or regularization may stabilize training. The exact objectives belong to the training recipe, not an invisible runtime fix.

Balance has several levels:

- token count per expert;
- weighted compute per expert when ranks or experts differ;
- bytes per destination rank;
- maximum expert time on the critical path;
- balance over a step, accumulation window, and data domain.

Equal counts do not guarantee equal time if sequence packing, expert dimensions, quantization, or hardware differ.

Expert parallelism

An expert-parallel group partitions experts across ranks. Non-expert transformer layers may use data or tensor parallelism, while MoE layers enter expert-dispatch groups. Token ownership and expert ownership are different coordinates.

If a rank owns multiple experts, it sorts received tokens by local expert and executes a grouped GEMM. Very small expert batches underuse the accelerator. Combining tokens from multiple sequences or microbatches improves efficiency but may increase latency or saved activation memory.

Expert tensor parallelism can shard a single large expert. It lowers expert parameter bytes per rank but adds collectives inside expert computation, on top of dispatch traffic. Use it only when experts do not fit or local expert GEMMs are large enough.

All-to-all cost

All-to-all traffic is personalized: each source sends a different partition to every destination. Total token bytes may appear balanced while one destination or network link is hot. Fixed-size padded all-to-all is predictable but sends empty capacity; variable-size exchange requires counts and offsets and can create launch variation.

Measure:

- dispatch and combine bytes;
- per-source/destination matrix, not only total bytes;
- packing/unpacking time;
- arrival skew before the collective;
- maximum rank receive count;
- expert GEMM occupancy and tail;
- overlap with attention or other layers.

Hierarchical dispatch can first exchange within a node, aggregate by remote destination, then traverse the network. It reduces small cross-node messages but adds a local pack stage.

Token packing and ordering

Packing requires a histogram of assignments, prefix offsets, and scatter. Preserve original token identity so outputs return to the right sequence and position. For top-k, one token has multiple branch identities and combine weights.

Stable ordering may be needed for reproducibility or exact checkpoint replay. An unstable parallel sort can change floating accumulation order. Define whether results need bitwise identity or only distributional/numerical equivalence.

Padding, duplicate expert choices, masked tokens, and zero-token experts are edge cases. Test them directly; they often expose mismatched counts that hang all-to-all.

Expert replication

Replicating a hot expert increases memory but splits its tokens across copies. A placement policy needs:

- expert popularity and variance by layer/domain;
- parameter bytes and load/eviction cost;
- routing distance and topology;
- consistency of replicated weights and optimizer state;
- deterministic mapping or acceptable stochasticity.

In training, replicas of one expert require gradient synchronization and consistent optimizer updates, creating another group. In inference, immutable weights simplify replication, but router-to-replica assignment affects cache locality and batching.

Dynamic expert migration during a step is rarely free. A slower-timescale placement controller can use recent routing histograms, move at safe boundaries, and keep a fallback copy until publication completes.

Expert, data, and tensor group geometry

Expert parallelism does not always multiply independently with data parallelism. Non-expert weights may be replicated across an expert group while expert weights are partitioned, so gradient groups differ by parameter class. Some systems fold expert and data axes to use the same devices differently in dense and sparse layers.

Draw group membership for:

- dense parameters and their gradient reductions;
- each expert's replicas or shards;
- router parameters;
- token dispatch and return;
- pipeline/context boundaries.

A single global "data parallel degree" is insufficient for an MoE checkpoint or optimizer.

MoE inference

Decode may provide only one token per active sequence. Per-expert batches become tiny and routing skew varies every iteration. The all-to-all latency can dominate even when experts are computationally cheap.

Serving strategies include larger continuous batches, expert replication, locality-aware request grouping, wide expert-parallel domains for capacity, and small local dispatch groups for latency. These goals conflict. Quantized expert weights reduce memory and load bandwidth but can make communication a larger fraction.

For strict token cadence, cap the work admitted to a group and protect against a rare hot-expert iteration. Report per-token tail latency by routed expert, not only average output tokens per second.

MoE failure and observability

Failure of one expert owner can invalidate the entire model replica unless another consistent copy exists. Recovery must restore expert placement, router version, optimizer ownership, and process groups together.

Operational dashboards should show expert load histograms, overflow, entropy, dispatch matrix, per-expert compute, routed-token quality slices, and replica placement. A routing collapse may first appear as network imbalance, then training instability.

SPARSE FLOPS DO NOT IMPLY CHEAP EXECUTION

An MoE model can activate few parameters yet wait on two all-to-alls, token packing, a hot expert, and tiny grouped GEMMs. Capacity parameters still consume memory and checkpoint bandwidth even when inactive.

Principal Interview Review

- 1 Trace one token through distributed top-k expert routing and back.
- 2 Compare padding, dropping, backup routing, and dynamic capacity on overflow.
- 3 Why can balanced expert counts still produce an imbalanced step?
- 4 Design expert parallel groups and gradient groups for a hybrid MoE model.
- 5 When should a hot expert be replicated, and how is consistency maintained?
- 6 Why is expert-parallel decode often harder than expert-parallel training?

Answer Key

1. Token path

The router produces expert IDs and weights. Counts and prefix offsets pack a token copy for each selected expert with original token identity. All-to-all sends each packed copy to the expert owner, which groups by local expert and runs the MLP. A return all-to-all restores source ownership; outputs are scattered to original positions and combined by routing weights. Counts, masks, top-k branches, and ordering must match at both transfers.

2. Overflow policies

Padding to fixed capacity gives predictable shapes but wastes compute and bandwidth. Dropping caps cost but changes model output and gradient. Backup routing preserves more tokens but changes specialization and can move the hot spot. Dynamic variable counts do useful work but create irregular collectives and kernels. Choose in the model/training contract, measure quality and tail performance, and expose overflow per layer and slice rather than hiding it in aggregate throughput.

3. Hidden imbalance

Experts may have equal counts but different token shapes, hardware, quantization, or branch multiplicity. Packing work, source-destination network paths, and arrival times can differ. One rank may own several simultaneously hot experts. Padding to maximum count also makes every rank pay for the worst destination. Profile per-expert compute, per-rank receive bytes, dispatch matrix, and arrival skew; count balance is only one indicator.

4. Group design

Give ranks explicit data, tensor, pipeline, context, and expert coordinates. Define separate gradient groups for dense parameters, router parameters, and each expert's replicas or shards. Define expert dispatch groups between token owners and expert owners, plus expert-TP groups if used. Map frequent dispatch within fast topology where possible and print ownership in checkpoints. Do not assume EP simply multiplies DP; parameter classes can have different replication geometry.

5. Expert replication

Replicate when a persistent hot expert sets the critical path and saved queue/dispatch time exceeds parameter memory, load, and synchronization cost. Place copies near token sources and split assignments using load-aware deterministic mapping. During training, reduce gradients across copies and apply identical optimizer updates; publish placement only at a safe boundary. During inference, immutable replicas simplify consistency, but version and quantization identity must match.

6. Decode difficulty

Training microbatches provide many tokens, creating larger expert GEMMs and amortizing all-to-all. Decode supplies roughly one token per sequence, so per-expert groups are tiny, skew changes each step, and two personalized collectives sit on token latency. Batching helps but conflicts with cadence and KV capacity. Expert replication and topology-aware grouping may be needed even when training used pure partitioning.

CHAPTER 29

29

Distributed Inference and Stateful Placement

Distributed Inference and Stateful Placement

Distributed inference repeats communication for every generated token while carrying request-specific KV state. A layout that scales training throughput can fail interactive latency because decode has small local work and a strict serial cadence.

Replication first

If a model and its working-state reserve fit on one device or node, independent replicas usually scale serving capacity most simply. Replicas run separate batches, isolate failures, and avoid per-layer cross-replica synchronization.

Model parallelism is needed when the model or KV does not fit, or when one request's latency benefits from multiple devices enough to offset communication. Use the smallest model-parallel group that satisfies fit and SLO; spend remaining devices on replicas.

The fastest single request plan can lower fleet goodput if it consumes more devices per replica and creates longer queues. Evaluate the full arrival process and failure reserve.

Tensor-parallel decode

Tensor parallelism shards weights and compute but adds reductions or gathers in many layers. Decode local matrix height is approximately active batch size, so high TP degree produces small per-rank GEMMs and latency-bound collectives.

A simplified step decomposition is:

$$T_{\text{step}} \approx T_{\text{local_compute}} + T_{\text{exposed_collectives}} + T_{\text{KV}} + T_{\text{host}}$$

Quantization reduces local weight traffic and compute, which can make collectives a larger fraction. Batching raises local GEMM efficiency and amortizes communication, but lengthens each iteration and consumes more KV. Choose TP jointly with the scheduler's latency budget.

Collective fusion across operations is constrained by dependencies and residual paths. CUDA graph capture or persistent scheduling can reduce host gaps, but the distributed launch order and communicator buffers must remain stable.

Pipeline inference

Pipeline stages reduce per-device weight memory. For one request, first-token and token latency include traversal across all stages. With many requests, stages can process different microbatches and improve throughput.

Continuous batching creates dynamic microbatches and output lengths. A stage may receive a different active set each iteration, so sequence IDs and KV ownership must be explicit. Pipeline bubbles arise from too few microbatches, stage imbalance, and pauses for long prefills.

Chunked prefill can create more uniform stage work. Separate prefill and decode pipelines can specialize schedules but introduce KV handoff, as Part III discusses. This chapter focuses on how the chosen parallel groups carry that state.

KV ownership under model parallelism

KV layout follows attention partitioning:

- head-sharded TP stores only local KV heads when compatible;
- replicated KV simplifies access but multiplies capacity cost;
- context or sequence partitioning stores token ranges on different ranks;
- pipeline stages store KV only for their layers;
- expert parallelism normally does not own attention KV directly.

Grouped-query and multi-query attention may have fewer KV heads than TP ranks. Replicating a KV head across a subgroup can avoid empty shards but increases bytes. Alternatively, ranks can share or exchange K/V, adding latency. The best mapping depends on head count, context, and fabric.

Block tables and prefix references must use the same distributed ownership. A request is not admitted until every required rank can reserve its shard; otherwise one rank can OOM after peers have advanced.

Context-parallel inference

Long-context prefill can partition queries or sequence blocks across ranks and use context-parallel attention. Decode has one new query per sequence but a large historical KV. Sharding the history reduces local KV memory and read bandwidth, then ranks must combine partial attention states.

For exact attention, each shard computes a local maximum, exponential sum, and weighted value. These statistics can be merged with global reductions or an online recurrence. Communication volume may be smaller than gathering all KV, but collective latency appears on every layer and token.

Use CP for inference when context capacity or attention time is the constraint and the reduction cost fits the token SLO. It is not automatically beneficial merely because training used CP.

KV transfer and remote memory

Migration or prefill-decode disaggregation transfers state between model-parallel groups. A complete transfer plan specifies:

- logical token/layer/head ranges;
- source and destination rank mapping;
- packed page layout, dtype, and scales;
- direct versus staged network path;
- chunk order and overlap;
- integrity, idempotency, and ownership transfer.

If source TP and destination TP degrees differ, the transfer includes resharding. Avoid collecting the entire KV on one coordinator. Each source sends the intersections of its logical shards with destination ownership.

Remote KV memory is useful for inactive sessions or pooled capacity when transfer/recompute cost is below holding scarce device memory. Fetching remote pages on every token makes the network part of the attention critical path and requires a much stronger availability and tail-latency contract.

Replica routing with distributed state

Route on estimated completion, not request count. Inputs include queue delay, model/adaptor residency, prefix locality, KV capacity on every rank, transfer or recompute time, topology health, and deadline.

State affinity should be bounded. Prefer the owner while saved work exceeds its extra queue delay; otherwise migrate, recompute, or start cold. A tensor-parallel group's capacity is the minimum admissible shard across its ranks, not the sum of free bytes.

Hot prefixes can be replicated across complete model groups. Partial replication is useful only if every consumer has a defined way to access the missing shards without creating a worse network dependency.

Expert-parallel serving

An MoE inference group adds token dispatch to attention and tensor communication. Small decode batches make expert GEMMs inefficient. Replicate popular experts, co-batch compatible tokens, or keep EP within fast links where memory permits.

The router and scheduler can cooperate: predicted expert affinity may improve placement, but future routes are input-dependent and exposing or storing them can raise complexity. Always keep a capacity-safe fallback for unexpected hot experts.

Streaming failure semantics

A failure of any rank generally invalidates the model-parallel replica. A replacement group can resume only with compatible weights, KV shards, sampling state, constraint state, and the exact emitted-token commit point.

Replication supplies spare groups, not automatic state replication. Recomputing KV from prompt and accepted output is often the simplest recovery before the first token; after streaming begins it may exceed latency and must avoid duplicate visible bytes.

Use request IDs and token sequence numbers. If exact resume cannot be proven, terminate the partial stream clearly rather than emitting a divergent continuation. Part III defines the product contract; the distributed layer must preserve it across all ranks.

SLO-constrained scaling

Report:

- first-token and inter-token latency by group layout;
- input and output goodput per accelerator;
- collective and KV-transfer tail time;
- queueing under complete replica count;
- memory headroom on the most constrained rank;
- performance after one group or network path fails.

Strong-scaling efficiency is secondary to good requests per fleet cost. A plan that halves kernel time but doubles devices and queueing is not an inference win.

INFERENCE PARALLELISM SPENDS A DEVICE ON EITHER ONE REQUEST OR ANOTHER REPLICA

Every added rank must justify the requests it prevents the fleet from serving independently. Fit, latency, queueing, and failure reserve decide together.

Principal Interview Review

- 1 Why is tensor parallelism more latency-sensitive in decode than in training?

- 2 Design distributed KV ownership for TP, PP, and GQA.
- 3 When can context-parallel decode outperform replicated or head-sharded KV?
- 4 How do you reshard KV during migration without a central bottleneck?
- 5 Design state-aware routing to a model-parallel group.
- 6 Specify safe recovery after one rank fails during streaming generation.

Answer Key

1. Decode sensitivity

Decode has a small token dimension and repeats a serial step for every output token. TP shrinks already-small local GEMMs while collectives remain inside many layers, so latency and launch overhead are exposed. Training has larger microbatches and long backward compute that better amortize or overlap communication. Quantization can worsen the ratio by accelerating compute without reducing collective latency.

2. KV ownership

PP stages own KV only for their layers. Within a stage, TP ranks own compatible attention heads when KV-head count permits. If GQA has fewer KV heads than TP ranks, either form KV-sharing subgroups, replicate selected heads, or lower attention TP; document the added bytes or exchange. Block tables and reservations are sharded identically, and admission succeeds only when every stage/rank reserves its required pages.

3. Context-parallel decode

CP helps when long-history KV memory or read time dominates and no rank can hold or scan the history efficiently. Each rank reads a shard and reduces compact online-softmax statistics instead of gathering KV. It loses when collective latency per layer/token exceeds saved KV time, especially at short context or small models. Sweep context, batch, CP degree, and topology under the inter-token SLO.

4. KV resharding

Describe KV in logical layer, token, head, and feature ranges independent of source layout. Each source rank intersects its owned ranges with destination ranges and sends chunks directly to the corresponding destination ranks. Pre-reserve destination pages, include version/dtype/scale metadata and checksums, and commit ownership after all chunks are visible. Avoid assembling full KV on a coordinator; distribute metadata planning separately from data movement.

5. Group routing

Treat a model-parallel group as one service unit whose admissible memory is constrained by its tightest rank. Estimate queue plus cache miss/recompute or transfer plus execution, adjusted for adapter, topology health, and SLO. Prefer state locality only while it reduces completion time, replicate hot immutable prefixes to whole groups, and retain cold-start fallback. Monitor prediction error and rank-level imbalance.

6. Rank failure during streaming

Abort and quarantine the whole synchronized group. Identify the last token committed by the model and acknowledged by the transport. Resume on a compatible spare only if prompt, accepted tokens, distributed KV or deterministic recompute, RNG, adapter, constraints, and model version reconstruct exactly, with duplicate suppression by token sequence. Otherwise end the stream with a clear partial-response failure and release all sharded state idempotently.

CHAPTER 30

30

Distributed Checkpoints, Recovery, and Elasticity

Distributed Checkpoints, Recovery, and Elasticity

A checkpoint is a distributed commit of model meaning. It must bind logical tensors, optimizer and scheduler state, data position, random streams, and topology-independent ownership into one recoverable publication.

What must be recoverable

A training restart commonly needs:

- model parameters and nonparameter buffers;
- optimizer moments, master weights, and step counters;
- learning-rate and loss-scale scheduler state;
- data-loader epoch, shard, sample, and packing position;
- random-number states for data, dropout, and model operations;
- parallelism metadata and tensor layout;
- training recipe, code/artifact version, and global progress;
- any curriculum, mixture weights, or dynamic routing state.

Saving only weights supports inference or a warm start, not an exact training continuation. Define checkpoint classes such as recovery, milestone, export, and evaluation artifact. They have different contents, retention, and compatibility promises.

Logical versus physical state

A logical tensor has a global name, shape, dtype, and semantic role. A physical checkpoint contains shards written by ranks under one parallel plan. The manifest maps logical ranges to immutable objects with checksums and ownership metadata.

This separation enables resharded restore. A checkpoint saved with DP8-TP8-PP4 may load into a different compatible plan by computing intersections between stored logical ranges and destination ranges. Avoid formats that encode only rank-local filenames without global offsets.

Optimizer state follows parameter identity, not current rank. Flattening, padding, expert placement, or pipeline cuts can change physical layout. Stable logical IDs and versioned transformation rules are essential.

Atomic publication

One safe protocol is:

- 1 coordinator allocates a unique checkpoint ID and immutable object prefix;
- 2 every rank writes assigned shards and reports size/checksum;
- 3 metadata validation confirms complete logical coverage without illegal overlap;
- 4 a manifest is written after all required objects are durable;
- 5 a small atomic pointer or catalog entry publishes the manifest;
- 6 retention removes unreachable older objects only after a grace period.

Readers see either the previous complete pointer or the new complete pointer. A partially written checkpoint is never named as latest. Object stores may provide atomic creation of one manifest object even when directory rename is unavailable.

Manifest publication must be idempotent. A retry with the same checkpoint ID either discovers the same complete contents or writes a new immutable attempt; it must not mix shards from different steps.

Checkpoint bandwidth model

For total checkpoint bytes C , aggregate sustainable storage bandwidth B , and fixed coordination overhead L , a lower bound is:

$$T_{\text{checkpoint}} \geq C / B + L$$

The slowest writer, metadata service, shared network, or storage partition may determine B . Per-rank bandwidth multiplied by ranks can exceed the storage system's aggregate limit.

Checkpoint traffic also contends with training collectives and input reads. A burst of synchronized writers can collapse the shared filesystem. Stagger writes, aggregate locally, use topology-aware writers, or provision a separate path when necessary.

Measure time paused, time until durable publication, device-to-host staging bytes, pending checkpoint memory, and impact on concurrent training steps.

Synchronous and asynchronous save

A synchronous save pauses training until required state is durable. It is simple and bounds the version of tensors being written, but the pause can be large.

An asynchronous save first snapshots or stages a consistent state, then writes while training continues. The snapshot must remain immutable. Copying to host frees device state sooner but consumes host memory and link bandwidth. Copy-on-write or double buffering can reduce pause but may temporarily double large state.

Backpressure is mandatory. If storage is slower than checkpoint production, do not queue unbounded snapshots. Skip a nonessential interval, block at a safe point, or lower frequency while preserving the recovery objective.

Incremental and local checkpoints

Incremental checkpoints store only changed chunks relative to a base. They save bytes when change is sparse or compression/deduplication is effective, but optimizer tensors often change everywhere. Long dependency chains slow restore and complicate retention; periodically compact into a full base.

Local node or rack checkpoints provide fast recovery from process failure but disappear with larger failure domains. Remote durable checkpoints cover rack or site loss at higher cost. A tiered policy can write frequent local recovery points and less frequent durable milestones.

Parity or erasure coding can tolerate lost shards without full replication, but encoding consumes compute/network and recovery reads many fragments. Use it when object loss risk and storage economics justify the complexity.

Restore and resharding

A restore planner should:

- 1 validate manifest, recipe, and requested topology compatibility;
- 2 allocate destination tensors in their new layouts;
- 3 assign read ranges across ranks to balance storage and network;
- 4 stream stored chunks directly into destination shards where possible;
- 5 transform dtype, padding, or layout only under a versioned rule;
- 6 reconstruct optimizer, RNG, data, and scheduler state;
- 7 run cross-rank validation before the next optimizer step.

Avoid loading a full tensor onto one rank and broadcasting it. Parallel reads and range intersection scale better. Test restore into the actual disaster topology, not only the same healthy cluster.

Validate checksums, finite values, representative tensor hashes, global step, optimizer ownership, and a deterministic short replay when feasible.

Failure interval and checkpoint frequency

Checkpointing too often wastes time; too rarely loses expensive work. If a synchronous checkpoint costs C_{time} and the mean time between relevant failures is M , a common first-order heuristic places the useful interval on the order of:

$\text{sqrt}(2 \cdot C_{\text{time}} / M)$

This assumes independent failures and roughly constant checkpoint cost. Large jobs have a lower job-level mean time between failures than one device. Include restart time, queue reacquisition, local versus durable failures, and asynchronous overlap before applying the heuristic.

Optimize expected time to a valid model, not only average step throughput. A plan with slightly slower steps but much faster checkpoint and recovery can finish sooner.

Coordinated failure handling

Synchronous model-parallel groups usually fail as a unit. When one rank faults:

- stop launching new collectives;
- surface the first known error, not a cascade of timeouts;
- capture rank and communicator state;
- terminate or fence all group members;
- release cluster resources and stale rendezvous state;
- restart from a verified checkpoint or a documented in-memory recovery point.

Trying to keep healthy ranks alive indefinitely can make recovery slower and state ambiguous. Use bounded timeouts and a control-plane lease so an old group cannot rejoin after replacement.

Elasticity

Data-parallel replicas are the easiest axis to change because model ownership within each replica remains complete. Even then, changing DP changes global batch or accumulation unless adjusted, and it changes sample assignment and random streams.

Changing TP, PP, CP, or EP requires resharding model and optimizer state, rebuilding communicators, and possibly recompiling engines. Treat this as a coordinated restart unless the framework explicitly supports a safe online transition.

Elastic recovery is not free capacity. Keep spare nodes or accept scheduler wait, and include rendezvous, artifact load, cache warmup, and checkpoint read in recovery time.

Recovery drills

Automate tests for:

- rank process crash during compute and collective;
- node loss during checkpoint write;
- corrupt or missing shard;
- restore with different DP or model-parallel degree;
- stale latest pointer and duplicate publication retry;

- storage throttling and unavailable metadata service;
- data-loader and RNG continuation correctness.

A checkpoint that has never been restored is an untested backup. Track restore success and duration as release metrics.

OPTIMIZE RECOVERY-ADJUSTED THROUGHPUT

The useful rate is training progress that survives expected failures. Include checkpoint pause, storage contention, lost work, restart, resharding, and queue delay when comparing parallel plans.

Principal Interview Review

- 1 Design atomic publication for a multi-terabyte sharded checkpoint.
- 2 What distinguishes a recovery checkpoint from a model export?
- 3 How can asynchronous checkpointing corrupt state or cause OOM?
- 4 Design topology-independent resharded restore.
- 5 How would you choose checkpoint frequency for a large cluster job?
- 6 What does safe elasticity require beyond restarting with fewer data-parallel ranks?

Answer Key

1. Atomic checkpoint

Write all shards under a unique immutable attempt ID, each with logical ranges, size, and checksum. Validate complete tensor coverage and required global state, then write one immutable manifest and atomically publish a pointer/catalog record. Readers ignore unreferenced partial attempts. Publication and retries are idempotent, and garbage collection waits past a grace period so it cannot delete shards still referenced by readers or the previous checkpoint.

2. Recovery versus export

A recovery checkpoint includes optimizer, scheduler, loss scale, global step, data position, RNG, parallel-layout metadata, and recipe identity needed to continue training. An export may contain only inference weights, tokenizer/configuration, and serving metadata in a consolidated or quantized form. A milestone may retain additional audit lineage. Name the class so operators do not attempt exact continuation from an artifact that lacks state.

3. Async risks

If training mutates tensors while writers read them, one checkpoint contains mixed steps. Staging solves consistency but consumes device/host memory and bandwidth; multiple pending stages can OOM. Copy completion and object durability are different boundaries. Use an immutable snapshot, explicit stream synchronization, bounded outstanding saves, checksummed shards, and publish only after durability. Apply backpressure when storage falls behind.

4. Resharded restore

Describe tensors globally by stable identity, shape, dtype, and logical ranges. The destination allocates its new shards and a planner intersects them with stored ranges, assigning parallel reads directly to consumers. Versioned transformations handle flattening, padding, dtype, pipeline cuts, and experts. Rebuild optimizer ownership, RNG, data, and schedulers, then validate hashes and a short replay. Never require one rank to materialize every full tensor.

5. Frequency

Estimate synchronous or exposed checkpoint cost, job-level failure interval, restart/queue time, and value of lost work. The square-root heuristic gives a starting interval near $\sqrt{2 \cdot C_time \cdot M}$ under simple assumptions. Simulate failure domains and tiered local/durable saves, then optimize expected time to valid completion under storage limits. Revisit as cluster size, checkpoint bytes, or failure rate changes.

6. Elasticity

Changing DP requires consistent new rank membership, adjustment of local batch or accumulation to preserve the intended global batch, sample reassignment, RNG policy, optimizer ownership, and scheduler progress. Changing model-parallel axes additionally needs resharding, communicator rebuild, engine compatibility, and new placement. Fence the old group, publish one membership epoch, and resume only after all ranks validate the same checkpoint and plan.

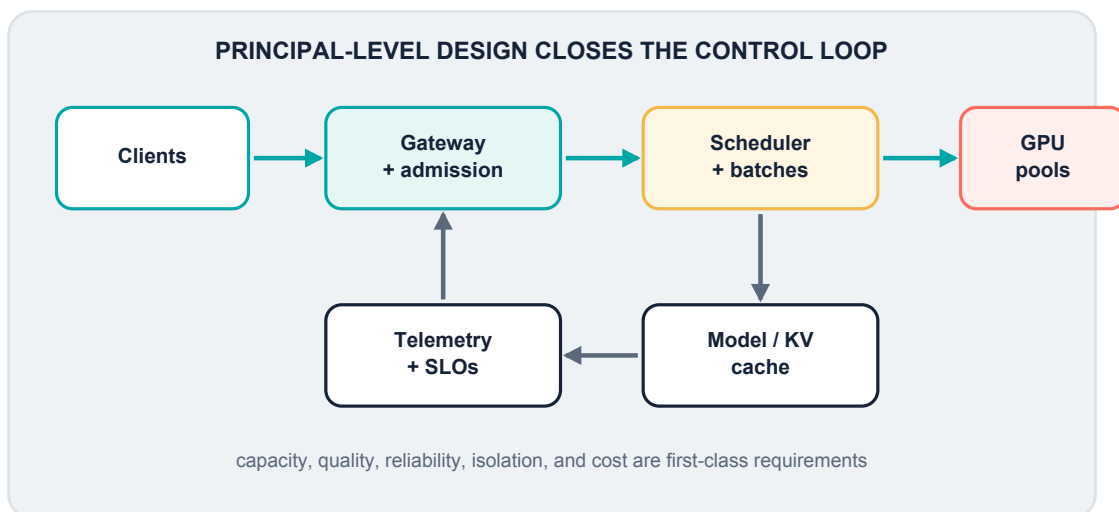
CHAPTER 31

31

Cluster Scheduling, Observability, and Distributed Diagnosis

Cluster Scheduling, Observability, and Distributed Diagnosis

A distributed job is a coordinated tenant of accelerators, network, storage, CPUs, and control-plane services. Production performance depends on placement, reproducibility, and the ability to identify the first rank or resource that diverges.



The distributed system has a control loop: placement and policy shape execution; telemetry, failures, and checkpoints feed the next scheduling decision.

Job specification and rendezvous

A reproducible job specification binds:

- code and container or environment digest;
- model architecture, data, tokenizer, and training recipe versions;
- world size and every parallel degree;
- rank-to-device and group mapping policy;
- network, storage, CPU, and memory requirements;
- checkpoint input/output and recovery policy;
- retry, timeout, priority, and preemption semantics;
- observability and security configuration.

Rendezvous assigns one membership epoch and rank mapping. Every process must agree before communicator creation. Reusing stale addresses or allowing two epochs to write the same checkpoint namespace can corrupt a run.

The control plane should fence old workers with leases or epochs. A restarted rank is not allowed to join an existing synchronous group casually; the whole group transitions through a supported recovery protocol.

Gang scheduling and placement

Synchronous jobs need all required ranks, so gang scheduling avoids holding partial allocations that cannot progress. Large rigid gangs can wait behind fragmented free GPUs even when aggregate capacity is sufficient.

Topology-aware placement chooses nodes and rails that satisfy the communication plan. Pack latency-sensitive TP groups tightly; place DP groups to exploit network bandwidth; avoid sharing oversubscribed links with another communication-heavy job when the scheduler can see that demand.

Placement objectives include:

- expected step time and variance;
- queue delay and fragmentation;
- failure-domain spread or concentration;
- locality to data/checkpoints;
- power, cooling, and maintenance constraints;
- preemption and replacement availability.

Packing one job into a small failure domain improves communication but a single failure loses more of its ranks. Spreading replicas improves fault isolation but may cross slower links. Make the trade explicit.

Admission and quotas

Reserve accelerators, host memory, network injection, storage bandwidth, and checkpoint capacity. GPU count alone permits too many jobs to saturate a shared filesystem or fabric.

Tenant quotas can be expressed in accelerator time, priority-weighted queue share, or reserved pools. Backfilling short jobs improves utilization if it cannot delay reserved starts. Preemption needs a checkpoint or restart-cost model; killing a job just before its next durable point wastes more than its nominal GPU time.

Use predicted checkpoint and startup duration in scheduling. A large job that needs thirty minutes to restore should not be repeatedly preempted for brief capacity gaps.

Step-time decomposition

Measure each step as a distributed critical path:

```
input -> forward -> backward -> optimizer -> checkpoint/eval boundaries
```

Within it, separate compute kernels, exposed collectives, rank arrival skew, pipeline idle, host gaps, memory stalls, and storage. The maximum rank determines synchronous step time, but the earliest divergence often occurs before the maximum event.

Report tokens or samples per second, model FLOP utilization with a declared FLOP convention, scaling efficiency, and time-to-valid-model. FLOP utilization can look low because of sparse experts, recomputation, padding, or a mismatched numerator; define what counts.

Rank-synchronous observability

Useful telemetry includes:

- global step, microbatch, pipeline slot, and membership epoch;
- rank coordinates and host/device/link identity;
- collective sequence number, group, type, bytes, start, and completion;
- kernel and data-loader spans;
- allocator and activation peak memory;
- expert load and context/pipeline balance;
- checkpoint stage and storage throughput;
- hardware errors, throttling, and network counters.

Align clocks or use logical sequence IDs. Wall-clock traces from unsynchronized hosts can invert causality. Aggregate percentiles without losing per-rank maxima and identities.

Control cardinality: ranks and steps belong in traces or bounded debug logs, while durable metrics aggregate by job, stage, node class, and group. During an incident, promote sampled detail for the affected job.

Straggler diagnosis

A slow step can originate from:

- data or CPU preprocessing on one rank;
- a long kernel due to shape, thermal throttling, or memory fault;
- pipeline stage imbalance;
- expert routing skew;
- collective transport or link errors;
- another job contending for NIC or storage;
- host scheduling and progress-thread delay.

Find the earliest event whose start or completion diverges. A late collective start indicates upstream skew; aligned starts with a late completion indicate communication or one rank failing to progress. Compare the slow rank with its physical neighbors and logical peers.

Do not begin by averaging. The mean rank is not on the critical path.

Hang and desynchronization debugging

For each communicator, capture a bounded flight recorder of collective sequence, operation, tensor metadata, stream, completion state, and call stack. On timeout, gather snapshots from all reachable ranks.

Look for:

- a sequence present on some ranks but absent on another;
- the same sequence with mismatched operation or count;
- ranks blocked before the communicator;
- a prior device error;
- cyclic waits across multiple communicators;
- stale membership or duplicate rank.

Reproduce with synchronization checks and smaller scale if timing permits, but remember that added barriers can hide races. Preserve the original flight recorder first.

Network and hardware isolation

Run layered tests:

- 1 single-device kernel and memory health;
- 2 pairwise device links within a node;
- 3 local collectives;

- 4 GPU-to-NIC and host affinity;
- 5 pairwise inter-node paths by rail;
- 6 collective benchmarks on the exact group;
- 7 application replay with competing traffic.

Quarantine nodes or links that show repeated correctable errors, bandwidth cliffs, or timeouts. A retry that lands on the same bad path is not recovery.

Firmware, drivers, runtime, collective library, and topology discovery form a compatibility set. Canary infrastructure changes on representative distributed jobs, not only single-GPU tests.

Reproducibility and semantic identity

Record model, tokenizer, data snapshot, packing, optimizer, precision, kernels, collective library, topology, parallel plan, checkpoint manifest, and membership history. Two jobs with the same source commit can differ because container, data order, or group layout changed.

Bitwise reproducibility across rank counts is often impractical due to reduction order and random mapping. Define the promised level: exact same-plan replay, numerical tolerance, or statistical training equivalence. Deterministic modes can cost performance and still require deterministic data and kernels.

End-to-end design method

A principal distributed design proceeds in this order:

- 1 define training/inference objective, SLO, batch semantics, and failure tolerance;
- 2 build per-rank memory and per-step FLOP/byte models;
- 3 choose the simplest parallel axes that make the workload fit;
- 4 map process groups to physical topology;
- 5 simulate schedules and collective critical paths;
- 6 plan checkpoints, recovery, and capacity reserve;
- 7 benchmark components, then full-step and failure scenarios;
- 8 instrument the invariants and automate regression gates.

Change one axis at a time when possible. Autotuning can search degrees, but constraints and cost models should prune invalid plans before expensive runs.

Production readiness checklist

Area	Required evidence
Fit	Peak per-rank memory with allocator and failure reserve
Compute	Local shapes, achieved kernels, recomputation cost
Communication	Bytes, groups, topology paths, overlap, tail ranks
Schedule	Pipeline/accumulation semantics, bubble, imbalance
Correctness	Cross-rank invariants, numerical tests, deterministic contract
Recovery	Atomic checkpoints, resharded restore, fault drills
Operations	Placement, quotas, alerts, flight recorder, runbooks
Economics	Queue plus run time, recovery-adjusted cost, useful output

Principal Interview Review

- 1 Design topology-aware scheduling for several competing large training jobs.
- 2 How do you distinguish a network slowdown from rank arrival skew?
- 3 Design a flight recorder for distributed collective hangs.
- 4 What should be included in distributed job semantic identity?
- 5 Compare strong-scaling efficiency with time-to-valid-model as objectives.
- 6 Walk through the first hour of diagnosing a 5 percent intermittent step-time regression.

Answer Key

1. Topology-aware scheduling

Collect each job's process-group graph, bandwidth/latency demand, gang size, duration, checkpoint/startup cost, and failure policy. Pack TP and other latency-sensitive groups into fast fabric domains, place cross-node groups across sufficient rails, and account for shared storage/NIC contention. Use gang admission, reservations, backfill that cannot delay starts, and preemption based on recoverable work. Balance communication locality against failure-domain and fragmentation cost.

2. Network versus arrival skew

Record per-rank collective enqueue/start and completion using logical sequence IDs. If one rank starts late and peers wait, inspect that rank's preceding input, kernel, pipeline, expert, or host work. If starts align but completion stretches, inspect link/NIC counters, route, contention, transport retries, and progress. Compare the same group/message size across healthy nodes and a topology-matched microbenchmark. The collective span alone cannot separate the cases.

3. Flight recorder

Maintain a bounded per-process-group ring buffer containing membership epoch, communicator ID, sequence number, operation, tensor count/dtype, stream, enqueue/completion timestamps, and call stack. On timeout or signal, persist local buffers and aggregate reachable ranks. The analyzer finds missing sequences, mismatched operations/counts, earliest uncompleted calls, prior device errors, and cycles across communicators. Keep overhead bounded and redact data values.

4. Semantic identity

Bind code/container, model architecture, tokenizer, data snapshot and order, optimizer/scheduler, precision/loss scaling, random seeds and policy, kernels and collective runtime, complete parallel layout/topology, membership epochs, and checkpoint manifest. Add training recipe and evaluation version. A source commit or weight checksum alone cannot explain user-visible or optimization behavior.

5. Scaling versus valid model

Strong-scaling efficiency measures step-speed gain as devices increase at fixed work. It can reward a fragile plan with high checkpoint cost, changed batch semantics, long queue/startup, or poor recovery. Time-to-valid-model includes queueing, data, evaluation, checkpoint pause, lost work, failures, and quality convergence. Use component scaling to diagnose efficiency, but choose plans by recovery-adjusted cost and time to the accepted artifact.

6. Regression diagnosis

Confirm version, workload, topology, and measurement changes; compare the same step and shape buckets. Split the regression into input, kernels, exposed collectives, arrival skew, pipeline idle, optimizer, and checkpoint interference. Identify affected ranks/nodes/rails and earliest divergence from synchronized traces. Check hardware throttling/errors and competing jobs, run topology-matched collectives, bisect software/configuration, and mitigate by quarantining or rollback while preserving evidence.

Further Study and Primary References

- [NCCL User Guide](#) - collective semantics, communicators, topology, and runtime behavior.
- [PyTorch Distributed Documentation](#) - process groups, collectives, and debugging controls.
- [DistributedDataParallel](#) and [FullyShardedDataParallel](#) - replicated and sharded execution contracts.
- [ZeRO](#) - optimizer, gradient, and parameter-state sharding.
- [Megatron-LM](#) and [Efficient Large-Scale Language Model Training](#) - tensor, pipeline, and composed parallelism.
- [Megatron Core Parallelism Guide](#) and [Context Parallelism](#) - maintained parallel layouts and long-context execution.
- [GPipe](#) - microbatch pipeline parallelism.
- [GShard](#) and [Switch Transformers](#) - sparse expert routing and scaling.

- [PyTorch Distributed Checkpoint](#) - parallel save/load and resharded restore.
- [PyTorch Flight Recorder](#) - collective hang and desynchronization diagnosis.

Final Distributed Systems Principle

A parallel plan is correct only when tensor ownership, communication order, physical placement, numerical semantics, and recovery state agree.

Scale is not the number of accelerators allocated. It is the amount of valid progress preserved per unit time and cost after communication, imbalance, queueing, checkpointing, and failure are included.

PART VI - CODING AND SYSTEM DESIGN

Part VI - Coding and System Design



Interview coding at senior levels tests more than syntax. The candidate must choose a data structure, state invariants, analyze complexity, handle streams and concurrency, and show how the local algorithm fits a production system.

CHAPTER 32

32

Streaming Algorithms and Top-K

Streaming Algorithms and Top-K

A stream cannot be revisited cheaply. The design must decide which information to retain, which approximation to accept, and how error is communicated.

Exact top-k

For a finite stream with known scores, maintain a min-heap of size k . Each item above the current minimum replaces the root. Time is $O(n \log k)$ and memory $O(k)$.

PYTHON

```
def top_k(stream, k):
    heap = []
    for item in stream:
        scored = (item.score, item.id, item)
        if len(heap) < k:
            heappush(heap, scored)
        elif scored > heap[0]:
            heapreplace(heap, scored)
    return [x.item for x in sorted(heap, reverse=True)]
```

The stable `id` breaks score ties. Production code defines updates, deletions, late data, serialization, and merge across partitions.

For distributed top-k, each shard emits local top-k; the coordinator takes top-k of their union. This is exact because an item outside a shard's local top-k cannot be in the global top-k. With per-key groups, the same argument applies per key but state can explode.

Sliding windows

Window semantics may be count-based, processing-time, or event-time. Event-time windows require watermarks and a lateness policy. A monotonic deque maintains min or max over a count window in amortized $O(1)$ time: remove expired indices from the front and dominated values from the back.

PYTHON

```
def sliding_max(values, window):
    q = deque() # indices; values decrease from front to back
    for i, value in enumerate(values):
        while q and q[0] <= i - window:
            q.popleft()
        while q and values[q[-1]] <= value:
            q.pop()
        q.append(i)
        if i + 1 >= window:
            yield values[q[0]]
```

Online mean and variance

Welford's update avoids the cancellation of $E[x^2] - E[x]^2$:

```
n <- n + 1
```

```
delta = x - mean
```

```
mean <- mean + delta / n
```

```
M2 <- M2 + delta * (x - mean)
```

Variance is $M2 / (n - 1)$ for a sample. Two Welford states can be merged, which makes the method useful in distributed streams.

Reservoir sampling

To sample k items uniformly from a stream of unknown length, fill the reservoir with the first k . For item i using one-based indexing, choose a random integer in $[1, i]$; replace a reservoir slot if the result is at most k .

The proof is inductive: after processing i items, every item is retained with probability k/i . Weighted reservoirs require different keys or priority sampling.

Heavy hitters and Count-Min Sketch

A Count-Min Sketch maintains several hash rows. To update key x by count c , increment one counter per row. Estimate by the minimum of those counters. Collisions only overestimate for nonnegative updates.

With width proportional to $1/\epsilon$ and depth proportional to $\log(1/\delta)$, error is bounded by ϵ times total count with probability at least $1 - \delta$. Hash functions and adversarial robustness matter in exposed systems.

Combine the sketch with a candidate heap for heavy hitters. The sketch estimates frequency, while the heap retains keys worth reporting. Merging is elementwise addition if dimensions and hashes match.

HyperLogLog and Bloom filters

HyperLogLog estimates cardinality from the position of the first set bit in hashed values across many registers. Its relative error is roughly proportional to $1 / \sqrt{\text{registers}}$. Bias correction and small-range behavior matter in implementation.

A Bloom filter answers set membership with false positives and no false negatives under append-only use. With m bits, n elements, and k hashes, false-positive rate is approximately:

$$(1 - \exp(-k n / m))^k$$

Counting Bloom filters support deletions with counters but use more memory and can underflow if updates are inconsistent.

STATE THE APPROXIMATION CONTRACT

For every sketch, say whether error is one-sided, probabilistic, mergeable, and sensitive to adversarial keys. That is the difference between naming a structure and designing with it.

Continuous log-stream design

If asked to process billions of events, clarify keys, window, ordering, lateness, update rate, query rate, and error tolerance. Partition by stable key, pre-aggregate locally, checkpoint state, and use watermarks for event time. Hot keys may need salting plus a second-stage merge.

Backpressure is part of correctness. Dropping events can bias top-k or averages. If approximation under overload is allowed, make sampling or shedding explicit and account for it in the estimator.

Principal Interview Review

- 1 Implement exact top-k and merge distributed results.
- 2 Explain a monotonic deque and its amortized complexity.
- 3 Prove reservoir sampling is uniform.
- 4 Compare Count-Min Sketch, HyperLogLog, and Bloom filters.
- 5 Design a late-data policy for event-time heavy hitters.

CHAPTER 33

33

ML Algorithms in Production Code

ML Algorithms in Production Code

Production ML code turns mathematical assumptions into shapes, invariants, numerical policies, and measurable failure behavior.

Vectorized logistic regression

For examples X , labels y , and weights w , logits are $z = Xw$. Binary cross entropy should use a stable logits form rather than computing $\log(\text{sigmoid}(z))$ directly.

PYTHON

```
def logistic_loss_and_grad(X, y, w, l2=0.0):
    z = X @ w
    # softplus(z) - y*z is stable binary cross entropy from logits.
    loss = mean(softplus(z) - y * z) + 0.5 * l2 * dot(w, w)
    p = sigmoid(z)
    grad = (X.T @ (p - y)) / X.shape[0] + l2 * w
    return loss, grad
```

Discuss sparse features, class weighting, calibration, distributed reduction, feature normalization, leakage, and stopping. Complexity is $O(\text{nnz}(X))$ for sparse matvec plus the update.

K-means

Lloyd's algorithm alternates nearest-centroid assignment and centroid recomputation. Complexity is $O(n \cdot k \cdot d)$ per iteration. Initialization strongly affects the local optimum; k-means++ spreads initial centers.

For large data, mini-batch k-means updates centroids from samples. Distributed implementations compute local counts and sums, then reduce. Empty clusters require a policy: reinitialize to a far point, split a large cluster, or retain the old centroid.

The Euclidean objective assumes roughly spherical clusters and comparable feature scales. If those assumptions fail, changing the optimizer will not fix the model.

Decision trees

A tree searches splits that reduce impurity. For classification, Gini or entropy is common; for regression, squared error. Efficient continuous-feature search sorts values or uses histograms. Missing values and categorical features require explicit routing.

Depth, minimum leaf size, feature subsampling, and pruning trade fit against variance. Distributed histogram methods reduce candidate statistics instead of shipping examples. Production code also handles monotonic constraints, stable serialization, and feature schema evolution.

Attention from primitives

PYTHON

```
def attention(q, k, v, mask=None):
    scale = 1.0 / sqrt(q.shape[-1])
    scores = q @ swapaxes(k, -1, -2) * scale
    if mask is not None:
        scores = where(mask, scores, -inf)
    probs = softmax(scores, axis=-1)
    return probs @ v
```

Follow-ups include causal masks, padding, batched heads, mixed precision, stable softmax, dropout, KV caching, GQA, and memory-efficient attention. The concise code is a semantic reference, not a performant long-context implementation.

Beam search

Beam search keeps the best partial sequences by cumulative log probability. Length normalization prevents systematic preference for short outputs. Finished beams must remain candidates without being expanded. Per-step top-k can be taken over beam * vocabulary scores.

Beam search is not sampling. It searches high-probability sequences under the model and may reduce diversity. Diverse beam variants, constraints, or stochastic beams modify the objective.

PYTHON

```
for step in range(max_steps):
    logits, state = model.step(beams.tokens, beams.state)
    scores = beams.scores[:, None] + log_softmax(logits)
    candidates = top_k(scores.reshape(-1), beam_width)
    beams = gather_parent_state_and_append(candidates, state)
    if beams.all_finished():
        break
return rank_with_length_penalty(beams)
```

Nearest-neighbor search

Exact search computes all distances. Approximate methods reduce query work through graphs, inverted files, product quantization, or trees. The design must specify recall target, latency, update rate, filtering, memory, and metric.

Embedding normalization makes cosine similarity equivalent to inner product ranking. Product quantization compresses vectors and uses lookup tables for approximate distance, trading recall for capacity and bandwidth. Metadata filters can destroy index efficiency if applied after retrieval; integrate filters or route to partitions.

COMPLEXITY WITHOUT SHAPES IS INCOMPLETE

$O(nkd)$ does not reveal whether the implementation is a dense GEMM, sparse reduction, or cache-unfriendly loop. State shapes, memory layout, and the operation the hardware will actually execute.

Principal Interview Review

- 1 Derive stable logistic loss from logits.
- 2 How do you distribute k-means updates?
- 3 Implement beam search state gathering correctly.
- 4 Explain why the reference attention code is memory-heavy.
- 5 Choose an ANN index for high update rate and metadata filtering.

CHAPTER 34

34

A Principal System Design Method

A Principal System Design Method

The best system design answer is a sequence of decisions tied to requirements. A diagram is evidence of that reasoning, not a substitute for it.

Step 1: define the contract

Clarify users, operations, scale, latency, consistency, availability, durability, privacy, compliance, cost, and evolution. For ML systems, add model quality, freshness, feedback, evaluation, and failure containment.

Convert vague statements into working numbers. If the interviewer withholds them, declare reasonable assumptions and make the design parameterized. Distinguish hard requirements from preferences.

Step 2: estimate

Estimate storage, throughput, state, bandwidth, and hotspots. Use powers of ten. Identify peak-to-average and read/write ratio. For model systems, estimate training tokens, model bytes, KV state, feature volume, embedding count, or evaluation cost.

The purpose is to choose architecture, not impress with arithmetic. Recompute when an assumption changes.

Step 3: draw the baseline

Show clients, API, durable state, compute workers, queues or streams, indexes or caches, and observability. Draw trust and failure boundaries. Assign ownership of each state transition.

Start with the simplest design that meets the contract. Premature multi-region, five cache tiers, or exotic consensus consumes time without proving judgment.

Step 4: walk the critical path

Trace one write and one read. For training, trace one example from acquisition to a checkpoint and evaluation. For inference, trace prompt admission to streamed token. State where retries, ordering, idempotency, and version checks occur.

Step 5: stress the design

Apply load, skew, failure, change, and abuse:

- burst traffic and hot keys;
- long-tail request sizes;
- partial network partition;
- corrupted or late data;
- model or schema rollout;
- dependency slowdown;
- malicious tenant;
- cost or capacity shortfall.

Step 6: close the loop

Define metrics, alerts, experiments, rollback, and capacity planning. State unresolved risks and the next experiment. This is often where a Staff answer becomes Principal: the design becomes operable and evolvable.

NAME WHAT YOU WOULD NOT BUILD YET

Restraint is a design skill. Defer components whose complexity is not justified by the stated scale or risk, and identify the trigger that would cause you to add them.

Tradeoff matrix

Decision	Option A	Option B	Flip condition
Sync vs async	Immediate consistency	Isolation and smoothing	User needs result in request path
Push vs pull	Low update latency	Consumer-controlled load	Fanout or offline consumers dominate
Replicate vs shard	Availability, simple reads	Capacity, write scaling	One node or replica no longer fits
Exact vs approximate	Strong semantics	Lower cost or latency	Bounded error is product-acceptable
Shared vs isolated	High utilization	Failure and security isolation	Noisy neighbor or compliance risk dominates

Communication during the interview

Signpost transitions: "I will first establish the workload, then estimate the dominant state, then draw a baseline and stress it." Keep a visible list of requirements and risks. When interrupted, answer the question and return to the structure.

If new information invalidates the design, revise it. Defending an obsolete choice signals rigidity, not leadership.

Principal Interview Review

- 1 Run the method on a feature store or evaluation service.
- 2 Which estimates change a vector database architecture?
- 3 How do you show failure domains on a whiteboard?
- 4 What makes an approximation acceptable?
- 5 Name a component you would defer in a new LLM platform and the trigger to add it.

CHAPTER 35

35

RAG, Vector Search, and Evaluation Pipelines

RAG, Vector Search, and Evaluation Pipelines

Retrieval-augmented generation is a data and evaluation system wrapped around a model. Retrieval quality, access control, freshness, and citation behavior matter as much as the generator.

Ingestion

Connectors fetch documents under source-specific permissions. Normalize without destroying structure. Chunk using semantic and layout boundaries while retaining document, section, time, and ACL metadata. Generate embeddings under a versioned model and write both canonical content and index entries.

Updates need identity and deletion. A replaced document should tombstone or supersede old chunks. Permission changes must propagate to caches and indexes. Store content hashes to avoid re-embedding unchanged chunks.

Retrieval

Hybrid retrieval combines lexical and dense signals. Lexical search handles rare names and exact identifiers; dense search handles semantic paraphrase. A reranker spends more compute on a small candidate set.

Filters should be enforced before content leaves the retrieval trust boundary. Post-filtering an approximate index can reduce recall; use filter-aware partitions, bitsets, or over-retrieval with a measured bound.

Query rewriting and decomposition can improve recall but also drift intent. Preserve original query, log transformations, and evaluate each stage.

Context assembly

Select chunks under a token budget, remove redundancy, preserve useful order, and include source identifiers. Position effects can make evidence in the middle less influential. Context compression saves tokens but can remove qualifications or provenance.

Prompt injection in retrieved content is an adversarial input. Separate instructions from evidence, apply source trust policy, constrain tool use, and evaluate injection attacks. The model should cite evidence and abstain when evidence is insufficient.

Evaluation

Decompose end-to-end quality:

- retrieval recall at k for required evidence;
- reranker quality;
- context precision and redundancy;
- answer correctness;
- faithfulness to provided evidence;
- citation correctness and completeness;
- abstention quality;
- latency and cost by stage;
- freshness and ACL correctness.

An answer can be correct despite failed retrieval due to model memory, or incorrect despite good retrieval due to generation. Stage metrics localize the failure.

Serving architecture

Use an API layer, query planner, lexical and vector retrieval, metadata/ACL service, reranker, context builder, generation service, and evaluation/telemetry pipeline. Cache public or tenant-scoped retrieval carefully. Version embeddings and indexes; support dual-read during migration.

For multi-region, decide whether indexes are replicated, partitioned by data residency, or queried remotely. Freshness and deletion propagation may control the topology more than query latency.

RAG DOES NOT GUARANTEE GROUNDING

Providing evidence changes the model input; it does not force the output to follow that evidence. Measure faithfulness, citations, conflict handling, and abstention directly.

Principal Interview Review

- 1 Design a secure RAG system for enterprise documents.
- 2 How do you evaluate retrieval and generation separately?
- 3 What happens when an embedding model changes?
- 4 How do ACL filters interact with approximate search?
- 5 Design defenses against prompt injection in retrieved content.

PART VII - TECHNICAL LEADERSHIP AND THE INTERVIEW MASTERCLASS

Part VII - Technical Leadership and the Interview Masterclass



Principal engineering is the ability to improve decisions beyond the code one person can write. The evidence is not senior-sounding vocabulary. It is durable direction, aligned execution, technical risk retired early, and systems that continue working after attention moves elsewhere.

CHAPTER 36

36

Executive Technical Communication

Executive Technical Communication

Executive communication compresses complexity without hiding uncertainty. It gives each audience the decision, evidence, risk, and requested action at the level they can use.

The decision memo

A useful one-page memo contains:

- 1 **Decision:** one sentence, including scope and timing.
- 2 **Why now:** user, reliability, capacity, or strategic trigger.
- 3 **Evidence:** the few facts that separate options.
- 4 **Options:** credible alternatives, including doing nothing.
- 5 **Recommendation:** choice and assumptions.
- 6 **Risks:** failure modes, mitigations, and reversible boundaries.
- 7 **Plan:** owner, milestone, metric, and next decision date.

The memo is not a transcript of analysis. Put supporting benchmarks and architecture detail in an appendix. If the recommendation cannot be stated before the history, the problem may not be framed.

Communicating uncertainty

Separate **known**, **estimated**, **assumed**, and **unknown**. Attach ranges where they change the choice. Name the observation that would invalidate the recommendation.

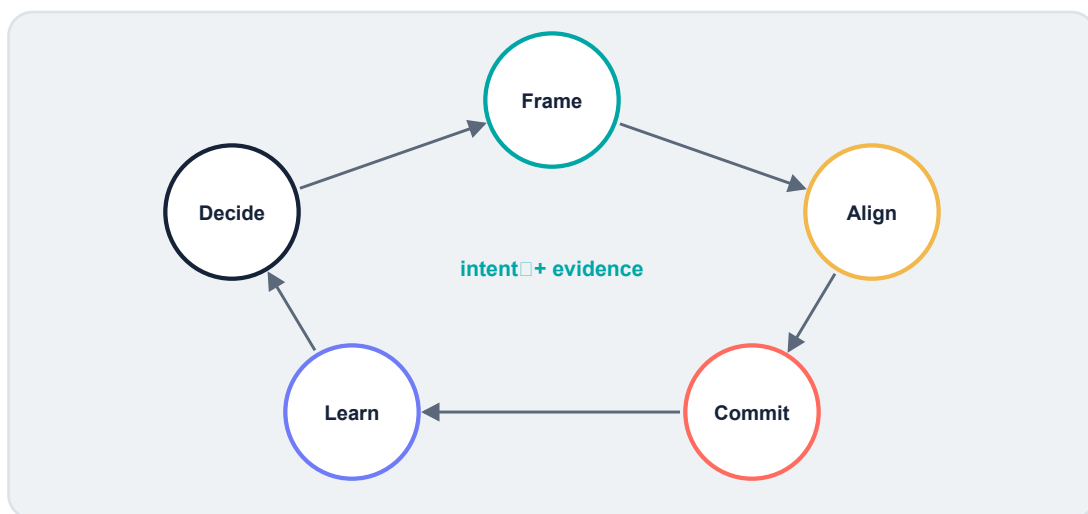
"We do not know" becomes actionable when followed by: "The uncertainty is draft acceptance on code traffic; a 48-hour replay of 100,000 anonymized prompts can bound it to within two points, and the architecture decision flips below 62 percent."

Technical deep dive structure

For an engineering audience:

- state the workload and invariant;
- show the resource model;
- explain the baseline and bottleneck evidence;
- walk the proposed data and control paths;
- compare alternatives;
- demonstrate correctness and performance;
- cover rollout, observability, and ownership.

For executives, lead with outcome, exposure, investment, and decision. Preserve the ability to drill down but do not force everyone through kernel details.



Strong technical leadership repeats a loop: frame with evidence, align stakeholders, commit to a decision, learn from execution, and revise.

Disagreement

Disagreement is valuable before commitment. Identify whether the conflict is about goals, facts, models, risk tolerance, ownership, or incentives. Many arguments persist because participants debate different layers.

Write the shared objective. List assumptions. Seek the smallest experiment that differentiates predictions. Define who decides and by when. After the decision, support execution while preserving a record of risk and trigger conditions.

CONSENSUS IS NOT THE SAME AS ALIGNMENT

Consensus means everyone prefers the choice. Alignment means people understand the decision, rationale, role, and escalation path. High-stakes work often needs the latter before the former is possible.

Influence without authority

Influence grows from credible problem framing, early stakeholder inclusion, useful artifacts, and reducing the cost of adoption. Build a coalition around the user or system outcome. Give partner teams real ownership rather than presenting a completed plan that requires their labor.

Mechanisms scale influence: design-review templates, shared metrics, reference implementations, migration tooling, office hours, and decision logs. A heroic recurring meeting does not scale.

Principal Interview Review

- 1 Turn a complex inference optimization into a one-page executive decision.
- 2 Describe a disagreement where both sides had valid local incentives.
- 3 How do you distinguish missing data from incompatible risk tolerance?
- 4 What mechanism makes a cross-org technical standard durable?
- 5 When should a Principal engineer escalate rather than seek more consensus?

CHAPTER 37

37

Strategy, Vision, and the First 90 Days

Strategy, Vision, and the First 90 Days

Vision names a future state and why it matters. Strategy chooses the few coordinated actions that can move the organization there despite constraints.

A technical vision

A credible vision connects users, system capabilities, and organizational leverage. "Build a world-class platform" is not a vision. A stronger statement is:

Any product team can take a validated model from checkpoint to a safe regional canary in one day, with predictable cost and no engine-specific integration.

The statement implies developer experience, release identity, evaluation gates, capacity, and isolation. It is measurable and invites architectural consequences.

Strategy diagnosis

Diagnose the constraint that prevents the vision. It might be fragmented serving engines, weak evaluation, long data iteration, unreliable checkpoints, lack of ownership, or capacity allocation. Strategy concentrates effort on the controlling constraint instead of launching one initiative per complaint.

Choose a small set of mutually reinforcing moves. For example:

- standardize model-engine release bundles;
- create a common evaluation and canary service;
- expose token-level capacity economics;
- migrate the highest-cost workloads with dedicated support.

Each move should have a leading metric, owner, and condition to stop or change course.

The first 30 days: learn and map

Meet customers and partner teams. Read incident reviews, architecture decisions, roadmaps, cost reports, and reliability metrics. Trace one real request and one model release. Map decision rights and informal influence. Identify which pain is loud and which pain is economically important.

Deliver a small useful artifact - perhaps a capacity model, risk register, or unified metric definition - without pretending to have the full strategy.

Days 31-60: align and test

Share a diagnosis and candidate principles. Form a working group around a concrete decision, not a generic forum. Run one or two experiments that retire major uncertainty. Draft target architecture, ownership, migration path, and success metrics.

Choose an early project that proves the mechanism and reveals integration cost. Avoid a showcase that succeeds only through exceptional manual effort.

Days 61-90: commit and start

Publish a decision memo and roadmap. Secure owners and capacity. Define interfaces, review cadence, rollout gates, and a deprecation plan for legacy paths. Start the pilot and establish transparent reporting.

Prioritization

Score opportunities by user or business impact, risk reduction, strategic leverage, effort, dependency, and reversibility. Do not reduce the decision to one formula; use scoring to expose assumptions.

High-leverage infrastructure often has indirect benefit. Quantify adoption and duplicated work removed. A platform with excellent local performance and no migration path has zero realized leverage.

STRATEGY INCLUDES A SEQUENCE

Order work so early steps generate information and capability for later steps. A list of desirable end states is a roadmap only after dependencies and decision gates are explicit.

Principal Interview Review

- 1 Give a measurable vision for ML inference infrastructure.
- 2 What would you do in the first 90 days at a new organization?
- 3 How do you prioritize reliability work against model capability?

- 4 When is standardization premature?
- 5 How do you measure the leverage of a platform investment?

CHAPTER 38

38

Behavioral Stories at Principal Level

Behavioral Stories at Principal Level

A Principal story demonstrates scope, judgment, and durable influence. It explains the technical mechanism and the organizational mechanism with equal precision.

The SCORE structure

Use **SCORE** rather than a mechanical chronology:

- **Situation:** the user and system context, scale, and stakes.
- **Constraint:** the central technical and organizational tension.
- **Options:** credible alternatives and the evidence that separated them.
- **Response:** your decisions, influence, mechanisms, and execution.
- **Effect:** measurable outcome, second-order impact, and what you learned.

The interviewer should know what *you* did without erasing the team. Use "I" for decisions and actions you owned, "we" for collective execution, and name partners' contributions.

Customer obsession

Weak story: you optimized latency because a dashboard was red. Strong story: you traced how latency disrupted a customer workflow, discovered that the aggregate metric hid long-context users, changed the SLO and scheduler, and created a recurring customer-to-capacity review.

Include a hard tradeoff. Customer obsession is not saying yes to every request; it is understanding the underlying outcome and choosing the most durable response.

Growth mindset

Choose a failure or changed belief with consequence. Explain the evidence that contradicted you, how you created safety while revising course, and which mechanism changed afterward. "I learned to communicate more" is too generic.

Example: a custom kernel project missed launch because the team optimized a microbenchmark rather than engine replay. You stopped the rollout, rebuilt the benchmark contract, introduced shape-distribution review and numerical gates, and later achieved a smaller but real end-to-end gain.

Diversity and inclusion

Use engineering mechanisms, not slogans. Examples include changing design-review participation, reducing timezone bias in decisions, auditing data or evaluation slices, improving interview calibration, sponsoring a colleague into visible ownership, or designing accessibility into a developer tool.

Name the observed barrier, your intervention, the voices included, and measured outcome. Avoid telling another person's private story or positioning yourself as the sole rescuer.

Ambiguity

Ambiguity stories should show how you created clarity without inventing certainty. State what was unknown, how you bounded it, which decisions were reversible, and what you deliberately postponed.

At Principal scope, ambiguity often spans organizations: no shared metric, unclear ownership, or incentives that reward local optimization. The story should show the mechanism that aligned those surfaces.

Hiring and mentorship

Mentorship is not only advice. It creates increasing ownership. Describe how you diagnosed a growth edge, set a stretch assignment with safety, provided feedback, opened stakeholder access, and stepped back. Hiring stories should include role definition, calibrated signal, closing, and the team's capability after hire.

Story bank

Prepare six to eight stories that can flex across prompts:

Story	Primary signal	Useful alternate prompts
Cross-org platform migration	Influence, vision	Conflict, customer, prioritization
Failed optimization and recovery	Learning, judgment	Risk, quality, delivery
Severe production incident	Ownership, calm execution	Ambiguity, communication
Architecture disagreement	Technical depth, alignment	Backbone, tradeoffs
Talent growth	Mentorship	Delegation, inclusion
Product-quality tradeoff	Customer judgment	Data, metrics, ethics

END WITH THE MECHANISM THAT REMAINED

Principal impact is durable. Mention the interface, metric, review, tool, ownership model, or talent growth that continued after the immediate result.

Principal Interview Review

- 1 Tell a customer-obsession story with a non-obvious tradeoff.
- 2 Describe a technical belief you changed after contradictory evidence.
- 3 Show inclusion through an engineering or decision mechanism.
- 4 Explain a cross-org disagreement without making the other side irrational.
- 5 Demonstrate mentorship by the ownership the other person gained.

CHAPTER 39

39

Recruiter-Derived Interview Masterclass

Recruiter-Derived Interview Masterclass

The following drills are the recurring high-signal questions behind the recruiter hints. Each answer is a compact spine that should expand through assumptions, equations, alternatives, and measurement.

1. Design a draft model for speculative decoding

Answer spine: Define target workload and exactness. Match tokenizer. Choose draft architecture from memory and latency budget. Train on production prompts with target logits or verified continuations, emphasizing rejection positions. Jointly tune draft size and proposal length. Evaluate acceptance per microsecond, committed tokens per cycle, target batch capacity, tail latency, and exact output distribution. Roll out by traffic slice with automatic fallback to ordinary decode.

Follow-ups: Why not only teacher outputs? What changes at high temperature? When does a larger draft win? How do you handle domain shift? Can K/V state be shared?

2. Explain forward versus reverse KL

Answer spine: Write both expectations. Forward KL weights error under the teacher and strongly penalizes missing teacher-supported modes. Reverse KL weights under the student and can select a mode when capacity is limited. For token distillation, minimizing teacher cross entropy equals forward KL up to teacher entropy. Temperature reveals non-argmax structure; sequence-level behavior still requires task evaluation.

Follow-ups: What if teacher assigns zero probability? Why T^2 ? When could reverse KL be useful? How does limited student capacity change behavior?

3. Generate training data for reasoning or code

Answer spine: Start from a capability taxonomy and seed distribution. Generate diverse candidates. Verify with independent executable, symbolic, retrieval, or expert checks. Score difficulty and novelty. Deduplicate against train and eval. Balance slices and preserve provenance. Train pilots, analyze failures, and feed verified failures back into generation. Protect private holdouts.

Follow-ups: How much synthetic data? How do you prevent model collapse? What if the verifier is gameable? How do you measure coverage?

4. Evaluate a new data-quality filter

Answer spine: Characterize what the filter removes by source and slice. Measure label precision and bias on adjudicated samples. Train controlled pilots with equal token budgets and matched schedules. Compare downstream capability, safety, memorization, and rare-slice floors. Evaluate whether gains come from quality or distribution change. Run counterfactual weight-versus-filter experiments.

Follow-ups: What if validation loss improves but target tasks regress? How do you test contamination? How do you handle dialect or domain bias?

5. Prefill versus decode

Answer spine: Prefill has many prompt positions, large GEMMs, and creates KV; it is often compute- or attention-IO-bound and controls time to first token. Decode adds one token per sequence, repeatedly reads weights and KV, and is commonly bandwidth- or launch-bound; it controls inter-token latency. Use roofline and byte estimates, then choose phase-specific scheduling and kernels.

Follow-ups: What changes with batch? Why does GQA help decode? When should phases be disaggregated? What is chunked prefill?

6. How many draft tokens should be proposed?

Answer spine: Expected accepted prefix grows with products of conditional acceptance, while draft and verification cost grow with proposal length. Benchmark committed tokens divided by cycle time, including memory and scheduling. Tune by batch, temperature, domain, and request class. Use a conservative adaptive policy if workload variation justifies it.

Follow-ups: Can confidence predict acceptance? Why can a longer proposal hurt even if verification is parallel? What happens under continuous batching?

7. Improve decode efficiency

Answer spine: Quantify weight and KV bytes and observed bandwidth. Increase safe batching, reduce bytes with quantization or GQA, improve KV allocation, fuse launch-bound operations, use graphs, cache prefixes, and consider speculation. Evaluate tail latency and capacity under production shapes. Do not optimize FLOPs if bytes are controlling.

Follow-ups: Why can 4-bit be slower? When does batching violate the product? Which fusion is safe? How do you profile collectives?

8. Parallelize matrix multiplication

Answer spine: Partition output tiles across blocks. Cooperatively load A and B tiles into shared memory with coalesced accesses. Accumulate register fragments, use tensor-core instructions when layout and precision allow, pipeline async copies, and handle edge tiles uniformly. Choose tile size from reuse versus registers, shared memory, and occupancy. Compare achieved compute and bandwidth ceilings.

Follow-ups: Split-K? Bank conflicts? Register spilling? Persistent kernels? Why use a library?

9. Design continuous batching

Answer spine: Maintain per-sequence state and admit/remove work at iteration boundaries. Protect decode deadlines, chunk long prefills, reserve KV pages, group compatible graph shapes, and enforce tenant fairness. Admission uses estimated tokens and state, not request count. Track queue, time to first token, inter-token gaps, utilization, and rejection.

Follow-ups: Starvation? Cancellation? Prefix caching? Speculative sequences that advance unevenly? Overload policy?

10. Walk through a recent ML systems project

Answer spine: State user outcome and scale. Explain the previous system and quantified bottleneck. Present alternatives and why your team chose one. Walk the critical technical mechanism. Show your personal decisions and cross-org influence. Report quality, performance, cost, and reliability outcomes. Close with what changed in the platform or organization and what you would do differently.

Follow-ups: Hardest disagreement? Largest unknown? How did you test? What failed? How would the design change at ten times scale?

11. Design a distributed inference service

Answer spine: Clarify model, traffic, SLO, context, availability, and cost. Estimate weights, KV, prefill, decode, and network. Choose parallel group inside fast topology, replicate for throughput and failure isolation, route by work and cache affinity, schedule continuously, and define admission. Add versioned control plane, observability, canary, rollback, and stateful failover semantics.

12. Design model evaluation

Answer spine: Build a versioned matrix of tasks and critical slices. Separate capability, system, guardrail, and diagnostic metrics. Protect holdouts and provenance. Run unit, offline, replay, shadow, canary, and online stages. Predefine meaningful effect and hard guardrails. Localize regressions by data, model, engine, and policy version.

13. Handle distribution shift

Answer spine: Define source versus target distribution and whether shift affects covariates, labels, or user behavior. Detect through input, representation, calibration, and outcome monitors. Confirm with labeled samples. Mitigate through reweighting, data acquisition, robust objectives, routing, fallback, or retraining. Protect critical slices and avoid self-reinforcing feedback.

14. Explain sampling

Answer spine: Temperature rescales logits. Top-k truncates by count; top-p truncates to a cumulative mass. Greedy takes argmax. Sampling policy changes diversity, calibration, acceptance, and evaluation. Implement stable softmax, deterministic seeded streams where required, distributed top-k merge, and exact speculative correction.

15. Customer obsession at Principal scope

Answer spine: Identify the workflow and consequence behind a request. Segment the metric to reveal affected users. Make a tradeoff that improves the durable outcome, not merely the loud request. Align product and infrastructure owners. Ship with feedback and leave a mechanism that keeps customer evidence in planning.

PRACTICE EXPANSION, NOT MEMORIZATION

For each answer spine, practice a 30-second thesis, a 3-minute structured answer, and a 15-minute deep dive with derivation or code. The structure should remain stable while examples and depth adapt to the interviewer.

Whiteboard closing checklist

- Did I state the workload and objective?
- Did I estimate the controlling bytes, FLOPs, state, or queue?
- Did I name a credible alternative?
- Did I cover correctness and failure, not only speed?
- Did I define offline, online, and guardrail metrics?
- Did I explain rollout and rollback?
- Did I make my own decision and remaining uncertainty clear?

PART VIII - FIELD REFERENCE

Part VIII - Field Reference



The appendices are designed for the final week before an interview and for architecture reviews after it. Each item is a retrieval cue; return to the chapter when the assumptions are not obvious.

CHAPTER 40

40

Formula and Capacity Sheet

Formula and Capacity Sheet

Training

Quantity	Approximation	Use
Global batch	$\text{microbatch} * \text{accumulation} * \text{DP}$	Schedule and optimizer comparisons
Dense training compute	proportional to $\text{parameters} * \text{tokens}$	Budget and scaling estimates
Adam state	weights + gradients + two moments, precision dependent	Device and sharding memory
Pipeline bubble	roughly $(\text{stages} - 1) / \text{microbatches}$ for simple schedule	Pipeline efficiency
Ring all-reduce traffic	$2 * (n - 1) / n * \text{tensor_bytes per rank}$	Network estimate

Always specify which bytes are sharded, replicated, offloaded, or temporarily gathered. Activation memory depends on sequence, microbatch, hidden width, saved intermediates, and checkpointing.

Transformer inference

$$\text{KV_bytes} = 2 * \text{layers} * \text{batch} * \text{sequence} * \text{kv_heads} * \text{head_dim} * \text{bytes_per_element}$$

$$\text{decode_weight_lower_bound} = \text{weight_bytes} / \text{sustainable_bandwidth}$$

$$\text{roofline} = \min(\text{peak_compute}, \text{bandwidth} * \text{FLOPs_per_byte})$$

`attention_scores = 0(sequence^2) values if materialized``

FlashAttention preserves $O(\text{sequence}^2)$ arithmetic for dense attention while reducing HBM materialization.

Speculative decoding

`accept(x) = min(1, p(x) / q(x))`

`correction(x)` proportional to `max(0, p(x) - q(x))`

`Pr(accepted_prefix >= k)` = product of conditional acceptance through `k`

`E[accepted]` = `sum_(k=1..gamma) Pr(accepted_prefix >= k)`

`speedup = committed_target_equivalent_work /`
`total_draft_plus_verify_time`

The final expression must include draft memory, target batch capacity, queueing, and scheduling when making a deployment decision.

Distillation

`KL(p || q) = sum p log(p / q)`

`H(p, q) = H(p) + KL(p || q)`

`softmax_i(z, T) = exp(z_i / T) / sum_j exp(z_j / T)`

Classic logit distillation often uses $T^2 * KL(\text{teacher_T} || \text{student_T})$ plus a hard-label loss.

Streaming

Structure	Update / query	Memory	Contract
Min-heap top-k	$O(\log k)$ update	$O(k)$	Exact for insert-only global top-k
Monotonic deque	amortized $O(1)$	window size worst case	Exact sliding min/max
Reservoir	$O(1)$ expected	$O(k)$	Uniform sample of unknown-length stream
Count-Min Sketch	$O(\text{depth})$	$O(\text{width} * \text{depth})$	One-sided frequency overestimate
HyperLogLog	$O(1)$	fixed registers	Probabilistic cardinality
Bloom filter	$O(\text{hashes})$	bit array	False positives, no false negatives

Queueing and capacity

Little's Law under stable conditions: $L = \lambda W$, where L is average work in system, λ arrival rate, and W average time.

It does not predict tail latency or prove capacity. Use it to check dimensional consistency and connect concurrency to service time.

At high utilization with variable service time, queueing rises nonlinearly. Provision headroom based on burst and failure targets, not average utilization alone.

Precision and quantization

For symmetric signed quantization with scale s :

$$q = \text{clip}(\text{round}(x / s), q_{\min}, q_{\max})$$

$$x_{\text{hat}} = s \cdot q$$

Per-group metadata cost is $\text{number_of_groups} * \text{scale_bytes}$ plus optional zero points. Effective model bytes include packing, alignment, scales, outliers, and any higher-precision fallback.

Communication

Communication time is often modeled as:

$$\text{time} = \text{latency_term} * \text{messages} + \text{bytes} / \text{bandwidth}$$

Collective algorithms alter both. Overlap hides only communication that runs concurrently with independent useful compute; report exposed communication on the critical path.

EVERY FORMULA HAS A BOUNDARY

Write units. Name what is omitted. A lower bound is not a prediction, and a big-O expression is not a performance model.

CHAPTER 41

41

CUDA Interview Checklist

CUDA Interview Checklist

Before writing the kernel

- 1 State input and output shapes, dtype, layout, alignment, and allowed error.
- 2 Write the serial reference and complexity.
- 3 Identify parallel output units and reduction dimensions.
- 4 Estimate bytes and FLOPs.
- 5 Choose block and warp ownership.
- 6 Decide which values live in registers, shared memory, or HBM.
- 7 Plan edge handling and synchronization.

Correctness checks

- Every global load and store is bounds-safe.
- Every block barrier is reached uniformly.
- Warp masks match active lanes.
- Shared-memory producers synchronize before consumers.
- Reduction identities initialize inactive lanes.
- Atomics use the correct type and scope.
- Random state is indexed without overlap.
- Numerical accumulation uses adequate precision.
- Empty, singleton, odd, and maximum shapes are tested.
- Results and gradients match a reference within declared tolerance.

Performance checks

- Global accesses are coalesced or the scatter is unavoidable.
- Shared accesses avoid serious bank conflicts.
- Tiles provide reuse before eviction.
- Register count does not cause damaging spill.
- Shared memory does not eliminate needed residency.
- Launch count and CPU submission are measured.
- Vector width matches alignment and tail policy.
- Tensor-core instructions are actually selected.
- Fusion saves more traffic than it costs in occupancy or flexibility.
- Benchmark covers the production shape distribution.

Matmul drill

Start with one output per thread. Add shared-memory tiling. Add register tiling. Change the inner loop to tensor-core fragments. Pipeline global-to-shared copies. Discuss split-K, persistent scheduling, epilogue fusion, and why a library may still win.

Reduction drill

Reduce within a warp using shuffles. Store one value per warp. Synchronize. Let the first warp reduce partials. State which lane owns the output. Extend to max, argmax, Welford state, and multi-block reduction.

Softmax drill

Find row max, sum shifted exponentials, normalize. Explain stable math, online combination, wide rows, recompute versus store, and fusion into attention.

LayerNorm drill

Compute mean and variance or Welford state, normalize, apply scale and bias. Explain accumulation dtype, row width, vectorized IO, residual fusion, and backward reductions.

Attention drill

Contrast naive score materialization with tiled online softmax. Derive rescaling. Cover causal masking, ragged lengths, GQA, backward recomputation, and tile selection.

Common CUDA interview bugs

Bug	Symptom	Repair
Early return before barrier	Deadlock or partial output	Predicate loads and stores; keep barrier uniform
Missing edge predicate	Illegal access at nonmultiple shapes	Zero-fill input tile and guard output
Lane uses undefined shuffle value	Shape-dependent corruption	Correct active mask and identity values
Excess register tile	Spill and sudden slowdown	Inspect register count; retune tile
Column access in square shared tile	Bank-conflict serialization	Pad or remap layout
Timing without synchronization	Impossible speed	Use events or synchronized benchmark boundary

NARRATE THE RESOURCE LEDGER

As you write, say what each optimization buys and consumes: fewer HBM bytes, more reuse, more registers, more shared memory, fewer launches, or greater specialization.

CHAPTER 42

42

Rapid-Fire Question Bank

Rapid-Fire Question Bank

Training and data

Why AdamW rather than Adam plus L2?

Adam's coordinate-wise scaling also scales an L2 gradient, producing uneven effective shrinkage. AdamW applies weight decay separately from the adaptive gradient update.

Why warm up the learning rate?

Early moment estimates, activation scales, and representations are unstable. Warmup limits destructive updates until optimization statistics become informative.

What is gradient clipping protecting?

It bounds update-driving gradient norm under outliers or instability. It does not repair a systematically bad learning rate or numerical bug.

How do you compare recipes fairly?

Match data, tokenizer, token budget, optimizer-step count or explain the difference, evaluation, precision, and failure handling. Report quality versus tokens, wall-clock, and cost.

Exact versus near deduplication?

Exact hashes remove byte-identical content after normalization. Near deduplication clusters similar documents using shingles, MinHash, structural signatures, or semantic methods.

Why can a quality filter hurt?

It may remove rare domains, informal language, difficult examples, or diversity while improving average fluency. Train controlled pilots and inspect slice distribution.

How do you prevent synthetic collapse?

Use diverse seeds and generators, independent verification, novelty and duplication checks, human or authoritative data anchors, and evaluation outside the teacher distribution.

Model and inference

Why GQA?

It reduces KV heads and cache bandwidth while retaining more K/V diversity than MQA, often improving the quality-throughput tradeoff during decode.

What does FlashAttention approximate?

Nothing in dense attention semantics. It changes the IO schedule using tiling and online normalization.

Why is decode sequential?

The next token distribution depends on the token just sampled. Parallelism comes from batch, model partition, or predicting and verifying multiple candidates.

What limits time to first token?

Queueing, prompt preprocessing, prefill compute and attention IO, model communication, and possibly prefix-cache miss or model cold start.

What limits inter-token latency?

Decode iteration cadence: scheduler wait, weight and KV traffic, collectives, launches, sampling, and streaming.

When does prefix caching fail?

Low prefix reuse, unsafe cross-tenant reuse, frequent model or template changes, large eviction pressure, or key mismatch from tokenization and position state.

Why can quantization improve capacity but not latency?

Unsupported or inefficient kernels, dequantization overhead, small workload, communication dominance, or a new compute bottleneck.

Why does speculation preserve quality?

Exact verification and residual correction reproduce the target distribution. Approximate variants need an explicit quality contract.

Distributed systems

Data versus tensor parallelism?

Data parallelism replicates model compute and communicates gradients. Tensor parallelism partitions each layer and communicates activations or partial results inside the forward and backward critical path.

What does FSDP trade?

Lower replicated state for parameter gather/reduce-scatter traffic, prefetch complexity, and more sensitive execution order.

Why pipeline bubbles?

Stages wait during fill and drain or when work is imbalanced. More microbatches and better schedules reduce but do not erase dependency.

Why topology-aware groups?

Collective frequency and volume differ. Put the most latency- and bandwidth-sensitive communication on the fastest links.

What makes a retry unsafe?

Non-idempotent side effects or missing request identity. Streaming output, data consumption, and artifact publication need deduplication or transactional boundaries.

Leadership

What is a Principal engineer's output?

Better technical decisions and execution across a scope larger than one person's implementation, expressed through architecture, mechanisms, talent, and aligned ownership.

How do you influence without authority?

Frame a shared outcome, include stakeholders early, provide evidence and adoption tooling, give partners ownership, and create mechanisms that reduce ongoing coordination cost.

When do you escalate?

When decision rights are blocked, risk exceeds the local mandate, a deadline requires choice, or incentives cannot be resolved at the working level. Escalate with options and evidence, not surprise.

How do you handle a wrong decision?

Contain harm, state new evidence, reopen the choice, preserve trust by owning the decision, and improve the mechanism that allowed the miss.

What is strategy?

A diagnosis of the controlling challenge plus a small set of coordinated choices that concentrate resources and create leverage toward a defined future state.

CHAPTER 43

43

Glossary and Decision Index

Glossary and Decision Index

Core terms

Arithmetic intensity: useful operations per byte moved across a specified memory boundary.

Continuous batching: scheduler policy that admits and removes sequences at iteration boundaries rather than waiting for an entire static batch.

Control plane: components that decide placement, version, rollout, policy, and capacity rather than executing individual model steps.

Data lineage: the versioned relationship from source through transformations to datasets, checkpoints, evaluations, and releases.

Forward KL: divergence weighted by the reference or teacher distribution, penalizing missing its supported outcomes.

GQA: grouped-query attention, where multiple query heads share a smaller number of key/value heads.

KV cache: stored attention keys and values for prior tokens, avoiding recomputation during autoregressive decode.

Operational intensity: a practical form of arithmetic intensity, sometimes including algorithm- and cache-specific byte assumptions.

Paged attention: attention over KV state managed in non-contiguous fixed-size blocks with logical-to-physical mapping.

Prefill: prompt processing phase that creates KV state and produces first-token logits.

Reverse KL: divergence weighted by the learned distribution, often favoring a supported mode when covering all modes is costly.

Speculative decoding: exact or controlled approximate generation using cheap proposals and expensive parallel verification.

Tensor parallelism: partitioning tensor operations across devices within layers.

Time to first token: latency from the declared request boundary until the first output token is available.

Decision index

If the symptom is...	First model	Likely chapters
Slow first token	Queue plus prefill compute/IO	Prefill, scheduling, serving
Slow token cadence	Weight/KV bytes plus collectives	Decode, quantization, distributed inference
GPU OOM with free fragments	Logical versus physical KV allocation	Paged KV cache
High acceptance, no speedup	Joint draft/verify/scheduler cost	Speculative decoding
Training instability	Update, precision, data, synchronization	Optimization, recipe
Benchmark gain, product loss	Evaluation contract and workload shift	Measurement, serving
Low GEMM throughput	Tile, tensor-core, occupancy, shape	Tiled matrix multiplication
MoE slowdown	All-to-all and expert imbalance	Transformers, distributed systems
Platform not adopted	Migration cost and ownership	Strategy and leadership
Recurring disagreement	Goal, facts, risk, or incentives	Executive communication

Final principle

Begin with the objective, quantify the bottleneck, choose the smallest coherent design, and close the loop with evidence.

That principle is equally useful for a KL objective, a CUDA kernel, a distributed scheduler, and a cross-org strategy. The scale changes. The discipline does not.